R pour l'archéologue

Exploration, visualisation et analyse de données

Nicolas Frerebeau Ben Marwick

2022-06-10

Université Bordeaux Montaigne
University of Washington

Nicolas Frerebeau

Archéosciences Bordeaux (UMR 6034)
Maison de l'Archéologie
Université Bordeaux Montaigne
F-33607 Pessac cedex

Ben Marwick

Department of Anthropology
University of Washington
Seattle, USA

Sommaire

Avant propos **v**
Que contient ce livre ? . . . v
Pour qui ce livre est-il écrit ? . . . vi
Licence . . . vii
Contributions . . . viii
Conventions d'écriture . . . viii
Reproductibilité . . . viii

Introduction **1**
Pourquoi les archéologues devraient-ils apprendre à programmer ? . . . 1
Pourquoi travailler avec R ? . . . 3
Ressources complémentaires . . . 7

Premiers pas avec R **9**

1. Prise en main **11**
1.1. Invite de commande . . . 11
1.2. Objets . . . 13
1.3. Environnement . . . 15
1.4. Fonctions . . . 15
1.5. Packages . . . 17
1.6. Obtenir de l'aide . . . 22

2. Structures de données **25**
2.1. Vecteurs atomiques (`atomic`) . . . 25
2.2. Listes (`list`) . . . 30
2.3. Matrices (`matrix` et `array`) . . . 33
2.4. Tableaux de données (`data.frame`) . . . 35

3. Extraction et remplacement **39**
3.1. Vecteurs atomiques . . . 39
3.2. Listes . . . 41
3.3. Matrices . . . 44
3.4. Tableaux de données . . . 46

4. Opérateurs binaires **49**
4.1. Opérateurs arithmétiques 49
4.2. Opérateurs logiques 51

5. Structures de contrôle **55**
5.1. Alternatives 55
5.2. Boucles 56

6. Écrire des fonctions **63**

Collecter et organiser ses données **65**

7. Introduction **67**

8. Gérer ses fichiers **69**
8.1. Travailler par projets 69
8.2. Organiser ses fichiers 71
8.3. Nommer ses fichiers 76

9. Organiser ses données **79**
9.1. Variables et observations 79
9.2. Enregistrer ses données 84

Préparation des données **87**

10. Introduction **89**

11. Importer et exporter **91**

12. Traitement des données **93**
12.1. Nettoyer 93
12.2. Selectionner 93
12.3. Filtrer 93
12.4. Transformer 93
12.5. Fusionner 93

Éléments de statistiques **95**

13. Introduction **97**

14. Statistique descriptive **99**
14.1. Indicateurs de position 99
14.2. Indicateurs de dispersion 99

15. Relations entre variables **101**
15.1. Covariance 101
15.2. Corrélation 101
15.3. Régression 101

Visualisation de données **103**

16. Introduction **105**
16.1. Recherche empirique en matière de visualisation de données 105

17. Représentations **109**
17.1. Représenter des quantités 109
17.2. Représenter des propotions 112
17.3. Représenter des distributions 112
17.4. Réprésenter des relations entre variables 112
17.5. Représenter des séries temporelles 112
17.6. Représenter des tendances 112
17.7. Diagrammes 3D 112

Données de comptage **113**

18. Introduction **115**

Données de composition **117**

19. Introduction **119**
19.1. Étendue du problème 119
19.2. Définitions 119
19.3. Principes 121

20. Transformations de données **125**

21. Détection des valeurs aberrantes **127**

Chronologie **133**

22. Introduction **135**

23. Chronologie qualitative **137**

24. Chronologie quantitative **139**

Cartographie 141

25. Introduction 143

26. Analyse de données vectorielles 145

27. Analyse de données raster 147

Annexes 149

A. Installation 151
A.1. Installer R 151
A.2. Installer RStudio 151

B. Aide mémoire 157
B.1. Séquences 157
B.2. Pipes 157

C. Bonnes pratiques 159

D. Style de programmation 161
D.1. Dénominations 161
D.2. Syntaxe 164
D.3. Commentaires 170

Bibliographie 173

Avant propos

Ce livre s'adresse aux personnes qui souhaitent analyser, visualiser et modéliser des données archéologiques à l'aide du langage R (R Core Team, 2022). Cet ouvrage doit vous permettre d'acquérir les connaissances et les compétences nécessaires pour aborder un large éventail de problèmes dans le cadre des sciences archéologiques. Par science archéologique, nous entendons ici une recherche systématique, objective et empirique sur les comportements humains et les sociétés du passé, à partir de données issues de vestiges et de traces matérielles.

Ce livre est en cours de rédaction.

Que contient ce livre ?

Le but n'est pas d'être exhaustif, mais de se concentrer sur ce que nous pensons être les compétences essentielles à tout archéologue souhaitant utiliser R. Certains archéologues utilisent déjà R de manière *ad hoc*, pour un tracer un graphique par-ci, ou ajuster un modèle linéaire par-là. Si vous êtes l'une de ces personnes, vous serrez sans doute déjà familier de certains aspects développés dans ce livre. Notre objectif est que vous y trouviez également, quelque soit votre pratique, de nouvelles idées en vous montrant comment R peut être au centre de votre travail de recherche, de l'acquisition des données brutes jusqu'à la finalisation de votre thèse, rapport ou manuscrit.

Bien que des notions élémentaires soient rappelées au fil de l'ouvrage, ce livre n'est pas un manuel de statistiques. Il ne contient pas (ou peu) de discussions détaillées sur la théorie statistique, et le lecteur sera invité à consulter à d'autres textes concernant les détails techniques des algorithmes, etc. Le lecteur trouvera ainsi des suggestions de lecture complémentaires à la fin de l'introduction générale et au début de chaque partie du livre[1].

Ce livre se veut une introduction pratique au langage R pour la réalisation efficaces de tâches courantes en archéologie. Sa lecture doit servir de tremplin vers une utilisation de R plus avancée. Les principaux sujets abordés dans ce livre sont les suivants :

[1] L'ensemble des références citées dans cet ouvrage est disponible sous la forme d'une bibliographie Zotero

- Le chapitre XX vous permettra de vous familiariser avec le langage R et d'acquérir les notions élémentaires qui vous permettrons de progresser dans la lecture de l'ouvrage. Ce premier chapitre constitue également un aide-mémoire, vers lequel vous tourner au cours de la lecture des différents chapitres.
- Le chapitre XX expose les motivations et modalités conduisant à mettre en œuvre une recherche ouverte et reproductible en archéologie (une bonne pratique pour la science en général, et pour votre productivité individuelle en particulier).
- Le chapitre XX propose un certain nombre de recommandations quant à l'organisation de votre travail en amont de l'analyse pour en faciliter la réalisation.
- Le chapitre X doit vous permettre de travailler avec des structures de données courantes en archéologie, les lire avec R et les manipuler pour les préparer en vue d'être analysées.
- Le chapitre XX
- Le chapitre XX
- Le chapitre XX Travailler avec des données de comptage d'artéfacts ou de restes de faune ou de flore.
- Le chapitre XX : biométrie.
- Le chapitre XX vous guidera dans l'univers très contraint de l'analyse des données de composition.
- Le chapitre XX détaille comment travailler avec des données chronologiques (relatives et absolues) en analysant des données stratigraphiques, en réalisant des sériations, et en calibrant, analysant et visualisant des datation radiocarbone.
- Le chapitre XX est une introduction à l'analyse de données spatiales et à la cartographie.
- Le chapitre XX : simplifier la recherche collaborative grâce au contrôle de version.

Pour qui ce livre est-il écrit ?

Ce livre s'adresse à tout étudiant, doctorant, post-doctorant ou archéologue professionnel souhaitant découvrir le langage R et s'y initier pour l'analyse de données archéologiques. Il s'adresse ainsi aux archéologues désireux d'élargir leurs horizons analytiques et d'accéder à de nouvelles méthodes et à des moyens plus efficaces d'organiser leur processus de recherche.

Il est écrit pour des utilisateurs débutants et aucune connaissance préalable de R, de l'informatique ou de la programmation n'est attendue. Cependant, un peu de curiosité pour les statistiques sera nécessaire, ainsi que la volonté de lire au-delà de ce livre, pour prendre judicieusement des décisions quant à la pertinence d'une méthode statistique au regard d'un problème spécifique.

La plupart des méthodes présentées ici sont destinées à des données tabulaires, c'est-à-dire des données qui sont généralement saisies dans des feuilles de calcul et des bases de données. Plus précisément, le livre s'adresse aux archéologues qui utilisent des données

organisées en lignes (c'est-à-dire des variables) et en colonnes (c'est-à-dire des spécimens, des échantillons ou des observations). De nombreux jeux de données archéologiques sont organisés différemment, comme les images ou les textes. Bien que R soit également utile pour ces derniers, nous ne les abordons pas en détail ici. Les données tabulaires sont la forme la plus courante en archéologie, comme dans de nombreux autres domaines, c'est pourquoi nous nous concentrons sur celles-ci ici.

Ce livre se veut utile pour les archéologues travaillant avec des données de petite taille (c'est-à-dire des tableaux de moins de 1 million de lignes, et de moins de 2 Go par fichier) qui peuvent être stockées sur votre ordinateur (plutôt que sur un serveur distant). De notre expérience, la majorité des archéologues travaillent avec des données à cette échelle, les méthodes présentées ici devraient donc être utiles à la majorité des applications archéologiques. Si vos données sont plus grandes que cela, vous faites probablement quelque chose de très spécialisé et vous aurez des défis particuliers à relever pour rendre votre recherche reproductible. Cependant, vous pourrez toujours utiliser R et les méthodes décrites ici pour travailler sur des sous-ensembles de votre grand jeu de données.

Licence

Ce livre est mis à disposition selon les termes de la licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 4.0 International[2].

Le code contenu dans ce livre est simultanément disponible sous la licence MIT[3].

Une entrée BibTeX pour citer ce document :

```
@Book{rarcheo,
  title = {R pour l'archéologue: Exploration, visualisation et analyse de données},
  author = {Nicolas Frerebeau and Ben Marwick},
  year = {2021},
  publisher = {Université Bordeaux Montaigne},
  address = {Pessac, France},
  url = {https://rstats.archaeo.science},
}
```

[2]Cela signifie que vous êtes libre de copier, distribuer et communiquer ce livre par tous moyens et sous tous formats, de modifier, transformer et créer à partir du matériel de ce livre. En contrepartie, vous devez créditer la source d'origine en précisant si vous l'avez modifiée, vous devez partager sous la même licence et vous ne pouvez pas en faire un usage commercial.

[3]Cela signifie que vous êtes libre de l'utiliser dans vos propres projets, à condition de citer la source.

Contributions

Le code source de ce livre est disponible sur GitHub. N'hésitez pas à y ouvrir un ticket pour signaler toutes les erreurs que vous pourrez constater ou proposer des modifications.

Conventions d'écriture

Par la suite, on écrira `foo()` pour désigner une fonction, `bar` pour désigner les variables et les paramètres des fonctions et `baz/` pour désigner les chemins. Les noms de *packages* sont indiqués en italique.

Les commentaires dans le code sont précédés par le signe `#`. Les blocs de code alternent l'entrée et la sortie. La sortie est commentée (précédée par `#>` pour la distinguer des commentaires standards) de sorte que vous pouvez facilement copier et coller les exemples dans R.

Reproductibilité

Ce livre a été écrit dans RStudio avec le package *bookdown* (Xie, 2016). Le site web est hébergé par GitHub Pages et automatiquement mis à jour après chaque commit par GitHub Actions (cette version a été compilée le 10 Jun 2022).

Les données utilisées dans les exemples sont issues des packages *datasets* (R Core Team, 2022) et *folio* (Frerebeau, 2022). Les exemples reposant sur la génération de nombre aléatoires sont reproductibles en utilisant `set.seed(12345)`.

Cette version du livre a été réalisée avec R version 4.2.0 (2022-04-22) et pandoc version 2.14.2. Les packages R utilisés sont résumés dans le tableau 1.

Tab. 1. : Liste des packages utilisés.

Package	Version	Source	Référence
bookdown	0.26	CRAN (R 4.2.0)	Xie (2022a)
bslib	0.3.1	CRAN (R 4.2.0)	Sievert et Cheng (2021)
car	3.0-13	CRAN (R 4.2.0)	Fox et al. (2022)
desc	1.4.1	CRAN (R 4.2.0)	Csárdi et al. (2022)
DiagrammeR	1.0.9	CRAN (R 4.2.0)	Iannone (2022)
downlit	0.4.0	CRAN (R 4.2.0)	Wickham (2021)
folio	1.1.0	CRAN (R 4.2.0)	Frerebeau (2022)
ggplot2	3.3.6	CRAN (R 4.2.0)	Wickham et al. (2022)
ggraph	2.0.5	CRAN (R 4.2.0)	Pedersen (2021)

Package	Version	Source	Référence
kableExtra	1.3.4	CRAN (R 4.2.0)	Zhu (2021)
knitr	1.39	CRAN (R 4.2.0)	Xie (2022b)
markdown	1.1	CRAN (R 4.2.0)	Allaire et al. (2019)
MASS	7.3-57	CRAN (R 4.2.0)	Ripley (2022)
mime	0.12	CRAN (R 4.2.0)	Xie (2021)
miniCRAN	0.2.16	CRAN (R 4.2.0)	de Vries (2022)
rmarkdown	2.14	CRAN (R 4.2.0)	Allaire et al. (2022)
scatterplot3d	0.3-41	CRAN (R 4.2.0)	Ligges et al. (2018)
sessioninfo	1.2.2	CRAN (R 4.2.0)	Wickham, Chang et al. (2021)
tidygraph	1.2.1	CRAN (R 4.2.0)	Pedersen (2022)
webshot	0.5.3	CRAN (R 4.2.0)	Chang (2022)
xml2	1.3.3	CRAN (R 4.2.0)	Wickham, Hester et al. (2021)

Introduction

Pourquoi les archéologues devraient-ils apprendre à programmer ?

Ce livre vise à résoudre un problème spécifique : la majorité des archéologues reçoivent peu ou pas de formation en programmation scientifique pour l'analyse, la modélisation et la visualisation des données, bien qu'il s'agisse de tâches courantes en archéologie. Un programme à interface en ligne de commande tel que R est idéal pour accomplir ce type de travail et, pourtant, il est peu familier et exotique pour la plupart des archéologues. Une interface de ligne de commande permet d'interagir avec un logiciel en envoyant des instructions au programme sous forme de lignes de texte brut. Au lieu de cela, la plupart des archéologues utilisent Microsoft Excel, SPSS ou un logiciel commercial similaire de type "pointer-cliquer" (à interface graphique). Ceci pose quatre problèmes.

1. Il en résulte généralement beaucoup de répétitions fastidieuses, comme le copier-coller entre des feuilles de calcul, ou entre un tableur et un logiciel de traitement de texte. Ces dernières années, les biologistes ont vu le volume des données génomiques augmenter considérablement en raison des progrès des méthodes de séquençage. Cela les a amenés à rechercher des moyens plus efficaces d'analyser leurs données et d'automatiser leurs analyses. Ils ont notamment constaté que les tableurs n'offraient pas suffisamment de souplesse pour mener à bien leurs travaux de recherche. Par conséquent, beaucoup se sont tournés vers R, Python et d'autres langages de programmation similaires pour surmonter les limites et l'inefficacité des tableurs. Le détail clé ici est le passage de l'interface graphique d'un tableur à l'interface en ligne de commande d'un langage de programmation, où des instructions sont fournies à l'ordinateur en texte brut.

2. Cela limite le développement de nouvelles méthodes, car l'utilisateur est limité à ce qui est disponible dans le logiciel commercial. Avec la plupart des logiciels commerciaux, vous êtes limité à l'ensemble des fonctions que l'éditeur choisit de mettre à votre disposition. Avec un langage de programmation sous licence libre (), vous et toute autre personne êtes libres de mettre en œuvre de nouvelles méthodes. R dispose de fonctions étendues pour l'analyse des données, mais plus important encore, R dispose d'un large éventail d'extensions () pour l'analyse quantitative, la visualisation, l'import et la manipulation des données… Pour la plupart des archéologues, quelle que soit la tâche que vous cherchez à réaliser, il y a de fortes chances que quelqu'un l'ait déjà réalisée, ou quelque chose de très similaire, et ait rendu

le code disponible. R est également la *lingua franca* des chercheurs en statistiques, qui publient souvent une extension à R pour accompagner leurs articles scientifiques. Cela signifie souvent un accès immédiat aux toutes dernières techniques et implémentations statistiques.

3. Les logiciels commerciaux (et même certains logiciels libres) rendent difficile la démonstration et la reproductibilité en raison de la nature éphémère des interfaces graphiques. Ces interfaces sont familières et faciles à utiliser pour la plupart des gens, car elles sont associées au développement de l'informatique domestique. Il est cependant très difficile d'enregistrer efficacement une séquence de clics complexes afin qu'une autre personne (ou vous-même à l'avenir) puisse répéter la procédure sans ambiguïté. Il est donc difficile de rendre vos analyses reproductibles si tout votre travail est effectué avec une souris. Il n'est pas pratique d'abandonner complètement l'utilisation de la souris lorsqu'on utilise un ordinateur, mais en utilisant un programme en ligne de commande tel que R, il devient possible d'enregistrer en texte brut les étapes les plus importantes de l'analyse. Nous disposons alors d'un enregistrement détaillé des étapes de notre analyse : nous pouvons facilement la partager avec d'autres et la réutiliser nous-mêmes, des mois ou des années plus tard.

4. Les logiciels commerciaux limitent la transparence de la recherche, car les algorithmes qui sous-tendent les fonctions utilisées ne peuvent pas être inspectés et les modifications ne sont pas autorisées. Les statisticiens ont observés depuis longtemps qu'Excel comporte des procédures statistiques inexactes (McCullough & Heiser, 2008 ; Yalta, 2008), et les manuels d'introduction aux statistiques avertissent généralement les étudiants de ne pas utiliser Excel lorsque les résultats sont importants (voir par exemple Keller, 2000). Avec un logiciel libre tel que R, vous pouvez facilement inspecter et modifier les algorithmes derrière chaque opération. R, en particulier, présente l'avantage supplémentaire d'être l'un des logiciels les plus précis pour l'analyse statistique (Almiron et al., 2009 ; Keeling & Pavur, 2007).

L'utilisation d'un programme en ligne de commande tel que R contribue pour beaucoup à la résolution de ces quatre problèmes.

Ce livre est complémentaire aux principaux manuels d'archéologie quantitative. Ces ouvrages sont de très bon supports pour les aspects théoriques, les discussions détaillées et la pertinence des exemples qu'ils proposent : ce livre ne les remplacera pas sur votre étagère. Cependant, ces livres ont trois limites que nous développerons dans les chapitres suivants.

1. La première de ces limites est l'absence d'exemples prêts à l'emploi, ce qui oblige le lecteur à déployer des efforts considérables pour mettre en œuvre une méthode décrite à l'aide de formules et cré de nombreuses possibilités d'erreur. VanPool et Leonard (2010) demandent ainsi à leurs lecteurs de faire leurs analyses statistiques à la main, une recommandation peu pratique et dépassée alors que les

ordinateurs sont facilement disponibles. La quantité d'efforts requis par ces textes est prohibitive pour l'exploration rapides de données et l'expérimentation à l'aide de nouvelles méthodes. Le présent livre comprend des exemples reproductibles utilisant des jeux de données réels, de sorte que vous pourrez facilement procéder à l'analyse pour explorer les effets de différents paramètres et les modifier pour utiliser vos propres données.

2. Les ouvrages existants n'accordent que peu de place aux méthodes intuitives et méthodologiquement plus robustes, telles que le rééchantillonnage et les analyses bayésiennes, privilégiant les méthodes paramétriques traditionnelles. Cela reflète une époque où la puissance de calcul était une ressource rare et où les calculs étaient souvent effectués à la main ou à l'aide d'une calculatrice (voir par exemple Fletcher & Lock, 1991). Les ordinateurs modernes peuvent maintenant facilement gérer le rééchantillonnage et les méthodes bayésiennes, et R est unique en ce qu'il possède un ensemble mature de méthodes pour calculer les tests courants dans ces situations. Ces méthodes sont également de plus en plus courantes dans la littérature scientifique. Ce livre présente ces approches alternatives afin de vous permettre de développer votre boîte à outils analytique.

3. Les ouvrages actuellement disponibles sont muets sur les aspects pratiques de nombreuses tâches d'analyse et de visualisation de données, courantes en archéologie bien qu'elles ne soient traditionnellement pas considérées comme des statistiques. Il s'agit notamment de la visualisation et de l'analyse d'informations stratigraphiques issues de fouilles et de données spatiales provenant d'enquêtes. L'apprentissage d'outils pour ces tâches se fait traditionnellement en autodidacte et souvent à grands frais avec des logiciels propriétaires. Ce livre essaie ainsi de répondre à un besoin de formation à l'usage d'outils libres pour la réalisation de tâches courantes en archéologie. Pour ce faire, il présente un environnement de travail reproductible afin de montrer comment R et ses extensions peuvent être utilisés pour mener de bout en bout sa recherche à partir de données de prospection, de fouille ou issues du laboratoire.

Pourquoi travailler avec R ?

Si vous êtes novice en matière de R, vous vous demandez peut-être quel est l'attrait de ce logiciel, surtout quand il existe tant d'autres programmes et langages couramment utilisés. Voici quelques-unes des caractéristiques qui nous ont incité à apprendre R :

R est libre et gratuit Il n'est pas nécessaire d'acheter une licence ou un abonnement pour utiliser R. Le développement de R a débuté en 1991 à l'initiative de Ross Ihaka et Robert Gentleman au sein du département de statistiques de l'université d'Auckland (Nouvelle-Zélande). En 1995 la première version officielle est publiée et R devient un logiciel libre, distribué sous licence GNU. Le code source de l'ensemble du système R est ainsi accessible à tous, en garantissant quatre libertés essentielles :

- La liberté de faire fonctionner le programme comme vous voulez, pour n'importe quel usage (liberté 0) ;
- La liberté d'étudier le fonctionnement du programme, et de le modifier pour qu'il effectue vos tâches informatiques comme vous le souhaitez (liberté 1) ; l'accès au code source est une condition nécessaire ;
- La liberté de redistribuer des copies, donc d'aider les autres (liberté 2) ;
- La liberté de distribuer aux autres des copies de vos versions modifiées (liberté 3) ; en faisant cela, vous donnez à toute la communauté une possibilité de profiter de vos changements ; l'accès au code source est une condition nécessaire.

R est modulable Les libertés offertes par la licence GNU et la structure modulaire de R permettent à toute personne de développer des extensions (appelées *packages*) et de les mettre à disposition de la communauté des utilisateurs. De nombreux packages fournissent des interfaces entre le langage R et d'autres langages ou logiciels libres, ce qui en fait un outil polyvalent, une passerelle permettant d'utiliser de nombreuses méthodes et algorithmes spécialisés. Ces packages peuvent être publiés et installés depuis le *Comprehensive R Archive Network*[4] (CRAN) qui distribue également le code source de R et sa documentation. Fin 2021, plus de 22500 packages étaient disponibles sur le CRAN (fig. 1).

R est robuste R est un langage spécialisé pour l'analyse statistique et la visualisation de données. R a été développé à l'origine par des statisticiens et continue d'être largement utilisé par des statisticiens professionnels et universitaires. Cela signifie que le langage entretient des liens forts avec la littérature scientifique en statistique. Nombre de ses algorithmes ont été validés par des publications et testés de manière approfondie par une utilisation intensive. Il est facile de passer de la théorie statistique à la pratique en utilisant R.

R facilite la reproductibilité R est un langage de programmation qui fonctionne à l'aide de scripts. Ces scripts permettent de définir une suite d'instructions à exécuter et contiennent ainsi l'ensemble des étapes de l'analyse. Il est possible de modifier ou de rééxecuter une analyse pour mettre à jour des résultats ou de réutiliser un même script avec des données différentes. Ces scripts peuvent être archivés, partagés, soumis à l'examen des pairs et publiés au bénéfice d'une science ouverte, transparente et reproductible. R est disponible pour tous les principaux types d'ordinateurs. Par conséquent, si vous effectuez votre analyse avec R, il y a de fortes chances pour que toute personne disposant d'un ordinateur puisse facilement la reproduire.

R facilite la publication R propose des outils flexibles pour communiquer vos résultats, en permettant de produire facilement des documents complexes comprenant du texte, des tableaux et des figures, le tout généré à partir du code (comme ce livre !). C'est un gain de temps considérable, surtout lorsque des semaines ou des

[4] Il existe d'autres archives, parfois spécialisées comme Bioconductor pour la biologie.

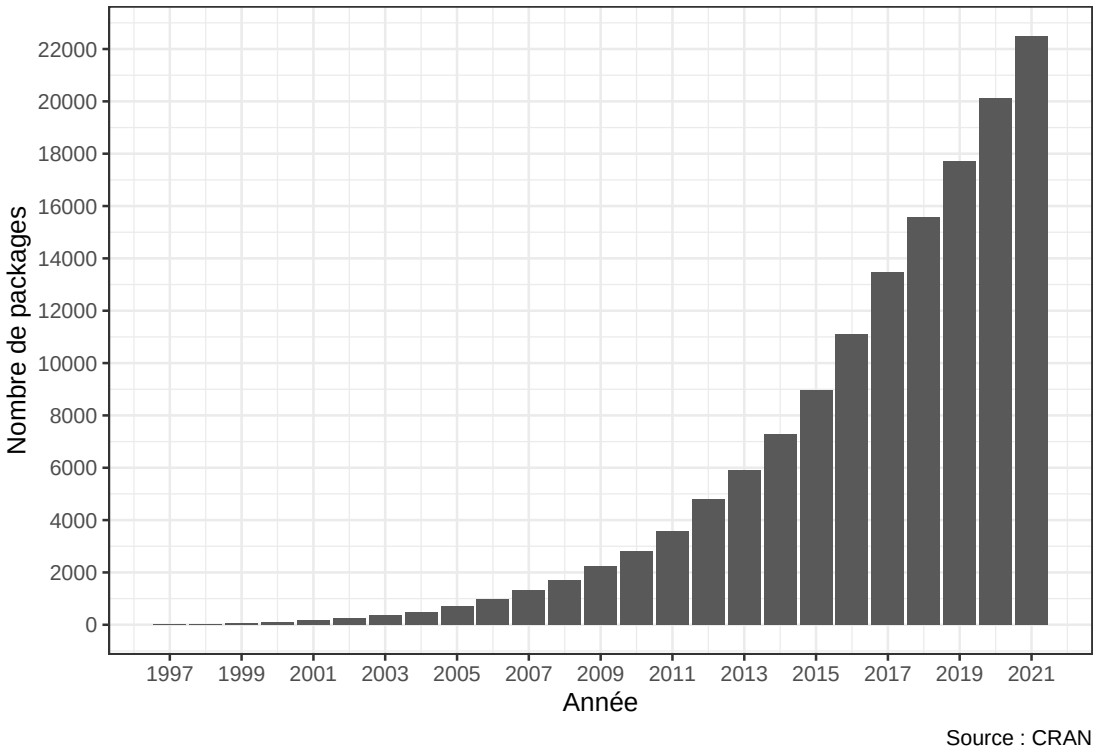


FIG. 1. : Évolution du nombre de packages disponibles sur le CRAN par année. Cette croissance exponentielle n'est pas sans poser de question quant au futur du CRAN (THIEME, 2018).

mois s'écoulent entre deux projets et que le souvenir des détails du dernier travail s'estompe. En consultant mon code R, je peux rapidement me souvenir de mon plan d'analyse et reprendre là où je me suis arrêté.

R est collaboratif R s'est progressivement imposé comme un outil de référence pour l'analyse de données dans de nombreuses disciplines, notamment en archéologie (fig. 2). Si l'apprentissage du langage peut paraître difficile pour qui n'a jamais programmé, la documentation est abondante et il est toujours possible de trouver de l'aide en ligne ou par le biais de nombreux groupes d'utilisateurs locaux. R bénéficie d'une communauté étendue d'utilisateurs, de contributeurs et de développeurs. Cette communauté est très active, solidaire et généralement progressiste. Les auteurs d'extensions sont souvent heureux de voir que leur travail est utilisé par d'autres et sont prêts à répondre à des questions sur leur utilisation.

Enfin, l'usage de R, comme de tout autre langage des programmation, conduit très souvent à généraliser un cas particulier. C'est ainsi un très bon moyen pour améliorer sa réflexion et, éventuellement, revoir la façon dont les problèmes sont posés.

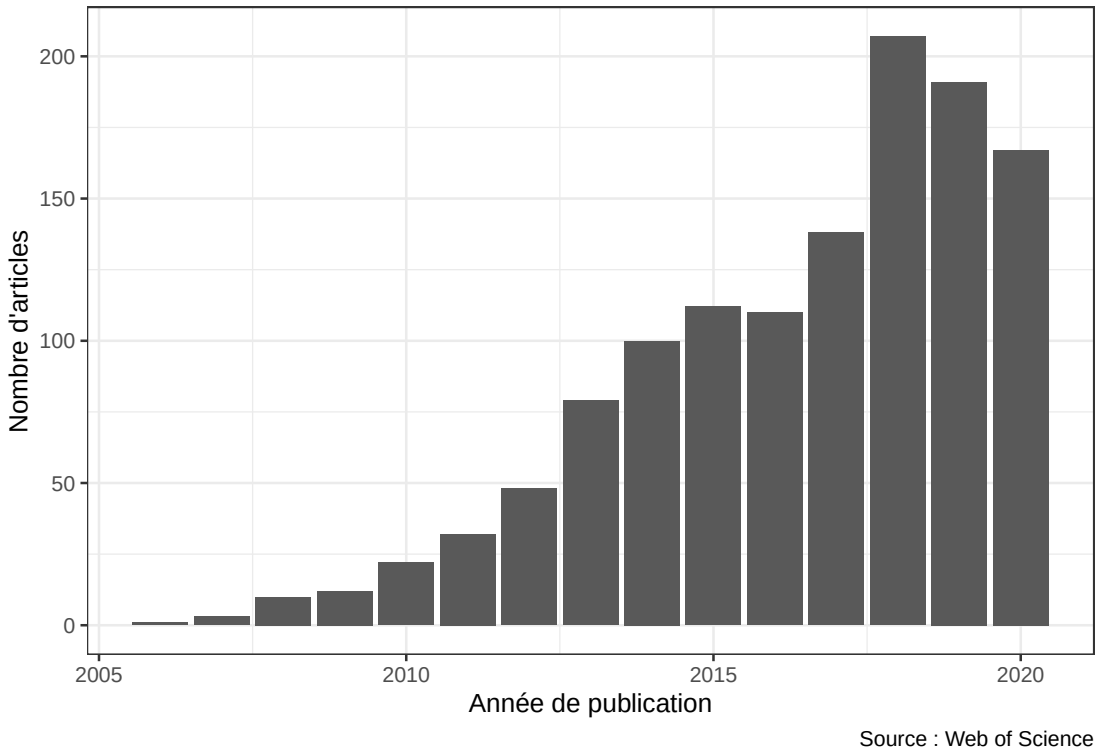


Fig. 2. : Nombre d'articles d'archéologie citant R publié chaque année (répliqué d'après Schmidt & Marwick, 2020).

Ressources complémentaires

Éléments de statistiques

(Baxter 2003 ; 1994, Van Pool et Leonard 2011, Drennan 2009, Shennan 1997)

Utiliser R

En complément de la documentation officielle pour démarrer avec R et comprendre la structure des données, il existe de très nombreux ouvrages dont la plupart sont consultables en ligne. *R in a Nutshell* de Adler (2010), *Hands-On Programming with R* de Grolemund (2015) et *R cookbook* de Long (2019) couvrent les usages élémentaires du langage (plus qu'ils ne sont traités ici) et sont d'une aide précieuse pour un usage courant.

L'ouvrage de référence de Murrell (2019), *R graphics*, offre une vue d'ensemble des fonctionnalités graphiques de R et de leur usage avancé. *R Graphics Cookbook* (Chang, 2012) constitue également une référence indispensable et fournit des recettes clé en main pour la création de graphiques. Quiconque s'attachant à la visualisation de données consultera avec intérêt *Fundamentals of Data Visualization* de Wilke (2019).

L'utilisateur confirmé trouvera dans les ouvrages *Advanced R* (Wickham, 2019) et *Efficient R programming* (Gillespie, 2016) de quoi approfondir sa compréhension du langage et développer sa pratique. *R packages* de Wickham (2015) est l'ouvrage idéal pour tout utilisateur souhaitant écrire son premier package et se former aux outils de développement (*devtools*, *roxygen2*, *testthat*...)[5].

Quelques ouvrages thématiques susceptibles d'intéresser l'archéologue[6] :

- *Comprendre et réaliser les tests statistiques à l'aide de R* (Millot, 2014), un ouvrage détaillé expliquant comment choisir un test statistique et présentant toutes les subtilités.
- *Geocomputation with R* (Lovelace, 2020), un livre sur l'analyse, la visualisation et la modélisation des données géographiques.
- *Text Mining with R* (Silge, 2017), un livre sur l'analyse textuelle avec R.

Enfin, on trouvera sur R-bloggers (un agrégateur de blogs) l'actualité du langage et de nombreux tutoriels.

[5] Voir également la documentation officielle pour le développement d'extensions de R, la description du processus de création des modules complémentaires, l'écriture de la documentation, les interfaces et l'API : *Writing R Extensions*.

[6] On trouvera sur le site du package bookdown et sur le site du Big Book of R de nombreux ouvrages supplémentaires, librement consultables en ligne. Le site du projet R propose également une liste d'ouvrages dédiés à R et ses applications.

Conseil

Le site collaboratif Rzine a pour objectif de favoriser la diffusion et le partage de connaissances sur la pratique de R en SHS :

> Rzine.fr s'efforce de dessiner les contours de la communauté R en SHS. Il donne aux débutant · e · s un accès simple à l'information et orienté vers une utilisation autonome. Il offre aux utilisateur · rice · s de niveau intermédiaire la possibilité d'étendre leurs pratiques et de s'ouvrir à d'autres méthodes parfois issues d'autres disciplines. Enfin, il s'agit d'un espace de diffusion pour les utilisateur · rice · s avancé · e · s, auteur · e · s de développements ou de documentation. — Rzine

Rzine recense de nombreuses ressources, fiches et manuels, *open source* et rédigés en français. Le site du projet permet également suivre l'actualité du langage et des évènements liés à la pratique de R en SHS.

Pour aller plus loin

Si les enjeux de reproductibilité et de science ouverte vous intéressent, il vous faudra sans doute compléter votre maîtrise de R par des outils supplémentaires, notamment les logiciels de contrôle de version (Bryan, 2021).

Remarque

Le tidyverse est un ensemble de packages partageant "une philosophie de conception, une grammaire et des structures de données" dont les principes sont détaillés par Wickham et al. (2019). Tout comme le tidyverse revendique un "parti pris" (*opinionated*), ce livre revendique un choix : celui de ne pas utiliser les packages du tidyverse (à l'exception de *ggplot2*), malgré leur popularité au sein de la communauté des utilisateurs de R.
Ce choix repose sur un constat simple : l'usage de ces packages tend à considérablement complexifier l'apprentissage de R par les débutants et à les enfermer dans un écosystème particulier. Au contraire, l'utilisateur qui maîtrise les bases de R peut faire face à une multitudes de situations (il s'agit ici d'apprendre les bases d'un langage, plutôt que celles d'un dialecte particulier[a]).
Ceci étant dit, le lecteur qui le souhaite trouvera dans *Introduction à R et au tidyverse* (Barnier, 2021), *R for Data Science* (Wickham, 2016) et *STAT 545* (Bryan, 2020) une introduction à l'analyse de données avec les packages du tidyverse.

[a] Voir Matloff (2019) pour une discussion détaillée.

Première partie

Premiers pas avec R

1. Prise en main

Maintenant que nous avons établi les motivations principales justifiant l'apprentissage de R et son usage en archéologie, il est temps d'introduire certains termes et concepts clés qui apparaissent fréquemment dans ce livre. Se familiariser avec ces termes vous aidera à trouver votre chemin lorsque vous commencerez à utiliser R.

Jusque là, nous avons utilisé le mot code sans vraiment le définir. Le **code** désigne tout ce qui est écrit dans un langage de programmation, d'un simple mot à une fonction de plusieurs milliers de lignes. Un **script** est un fichier qui contient du code, les noms de fichiers de scripts R se terminent par l'extension `.R`. Vous pouvez ouvrir ces fichiers avec n'importe quel éditeur de texte.

Le code contient souvent des **commentaires**, que vous pouvez reconnaître dans le langage R par un symbole `#` (dièse) en début de ligne. Le symbole `#` indique à l'interpréteur R d'ignorer tout ce qui se trouve à droite de ce symbole et de passer à la ligne de code suivante. Il n'y a pas de règles strictes sur le nombre de commentaires que vous devez inclure dans votre code, mais un bon point de départ serait une ligne de commentaire pour chaque 10 lignes ou section majeure de code. Les commentaires de code sont importants car ils vous aident, ainsi que d'autres personnes, à comprendre l'objectif du code. Les commentaires doivent expliquer le pourquoi, et non le comment (il suffit de lire le script pour cela) de votre code : ils doivent expliquer l'intention générale du code. Plus généralement, vos commentaires doivent vous dire ce que vous devrez savoir dans quelques semaines ou mois, lorsque vous reprendrez le travail après une pause et que vous ne vous souviendrez plus des détails.

1.1. Invite de commande

R se présente sous la forme d'une interface en ligne de commande, permettant de donner des instructions à l'ordinateur. Dans RStudio, cette interface est accessible dans l'onglet *Console*.

Lorsque R est en cours d'exécution, l'interface signifie qu'elle est prête à recevoir des instructions par une invite de commande (*prompt*) indiquée par un chevron `>`.

Il est alors possible d'entrer une **expression**, qui va être **évaluée** par R avant d'en retourner le résultat :

```
1 + 1
#> [1] 2
```

Sans surprise, le résultat de l'addition est 2. R donne une information supplémentaire, le `[1]` devant `2` précise qu'il s'agit du premier (dans le cas présent, du seul) élément retourné. Certaines commandes peuvent retourner plusieurs valeurs. Par exemple, l'opérateur `:` (deux-points) permet de construire des séquences de nombres entiers :

```
0:100
#>   [1]   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
#>  [17]  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
#>  [33]  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47
#>  [49]  48  49  50  51  52  53  54  55  56  57  58  59  60  61  62  63
#>  [65]  64  65  66  67  68  69  70  71  72  73  74  75  76  77  78  79
#>  [81]  80  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95
#>  [97]  96  97  98  99 100
```

La commande `0:100` retourne 101 valeurs qui sont affichées sur plusieurs lignes : le nombre entre crochet indique l'**indice** de la valeur par laquelle débute chacune des lignes (la deuxième ligne commence par la 17e valeur, etc.).

Important

Contrairement à Python, dans R les indices commencent à 1 et non à 0.

Si une commande incomplète est transmise, l'invite de commande signale par un `+` que la suite des instructions peut être saisie sur plusieurs lignes :

```
1 -
1
#> [1] 0
```

Conseil

Il est possible d'interrompre la saisie ou d'annuler une commande avec `ctrl/cmd + c`.

Si une commande erronée est transmise, R s'interrompt et retourne un message d'erreur (plus ou moins explicite) :

```
x
#> Error in eval(expr, envir, enclos): objet 'x' introuvable
```

Conseil

Il est possible de naviguer dans l'historique des commandes précédemment exécutées à l'aide des flèches `haut` et `bas` du clavier.

1.2. Objets

Il est possible d'associer un nom à une valeur et ainsi de créer des **variables** (au sens informatique) qui peuvent ensuite être appelées dans l'invite de commande. Une variables n'est rien d'autre qu'un emplacement de la mémoire de l'ordinateur, réservé pour stocker une valeur.

Cette **affectation** (ou assignation, par anglicisme) est réalisée à l'aide de l'opérateur `<-` (flèche) :

```
x <- 1 # Le résultat d'une affection n'est pas affiché dans la console

(x <- 1) # Avec des parenthèses, le résultat est affiché dans la console
#> [1] 1

x
#> [1] 1
```

Il est possible de copier une variable :

```
y <- x
y
#> [1] 1
```

Une fois copié, il n'existe aucun lien entre l'objet initial et sa copie. Dans l'exemple précédent, si `x` est modifié après avoir été copié dans `y`, la valeur de `y` ne sera pas modifiée.

Il est possible d'affecter plusieurs variables en même temps :

```
i <- j <- 1
```

Les variables ainsi crées peuvent être réutilisées :

```
x + 1
#> [1] 2
i + j
#> [1] 2
```

La réutilisation d'un nom de variable existant entraîne le remplacement de la valeur précédente (sans avertissement) :

```
i <- 1
i
#> [1] 1

i <- 2
i
#> [1] 2
```

Important

- Un nom de variable peut uniquement être composé de lettres, de chiffres et des caractères `.` ou `_`.
- Un nom de variable doit toujours commencer par une lettre (ou par un point suivi d'une lettre).
- Les noms de variables sont sensibles à la casse (`x` et `X` sont deux variables différentes).
- Il existe des mots réservés qui ne peuvent être utilisés comme nom de variable.

Dans R les variables stockent des **objets**[1]. Ainsi, le code `x <- 1` peut se lire "créer un objet nommé x dont la valeur est 1"[2].

Un objet possède un type et une structure de données particulière. Le type d'un objet est lié au type d'information qu'il contient et à la façon dont il est stocké dans la mémoire[3]. La structure de données d'un objet correspond à la manière dont sont organisées les données.

Ainsi, pour conserver les résultats d'un calcul, nous affectons la sortie d'une instruction à une variable. Ce processus d'affectation enregistre la sortie dans notre **environnement** afin qu'elle soit disponible pour une réutilisation ultérieure dans notre session. L'affectation n'enregistre pas les données dans un fichier qui peut être utilisé ailleurs : l'export de données est un processus distinct que nous aborderons plus tard.

[1] En informatique, un objet est une représentation d'une entité du monde réel et de ses attributs, auxquels sont associés des traitements particuliers.

[2] Il s'agit cependant d'une simplification, voir la section "Binding basics" dans Wickham (2019) pour plus de détails.

[3] Votre système d'exploitation alloue la mémoire en fonction du type de donnée et décide de ce qui peut être stocké dans la mémoire réservée lors de la création d'une variable.

1.3. Environnement

1.4. Fonctions

Il existe des objets particuliers, qui permettent d'agir sur d'autres objets : les **fonctions**. Une fonction désigne un bloc de code qui exécute des instructions pour effectuer une tâche précise. Par exemple, la fonction `mean()` permet de calculer la moyenne d'un ensemble de nombres. Dans ce livre, vous serez toujours en mesure de reconnaître une fonction car elle sera toujours suivie d'une paire de parenthèses, avec souvent du code entre les parenthèses. Une fonction est un moyen efficace d'organiser le code, car une seule fonction peut entraîner l'exécution de centaines de lignes de code pour produire un résultat. Plutôt que d'exécuter ces centaines de lignes, une par une, encore et encore, nous pouvons simplement taper le nom de la fonction qui les invoque et nous épargner beaucoup de saisies. Minimiser la saisie est une bonne chose : cela permet de gagner du temps et de réduire les risques d'erreurs en faisant des fautes de frappe.

Les fonctions peuvent accepter une ou plusieurs valeurs (ou objets) en entrée, appelés **arguments**, et retournent un objet au terme de leur exécution. Les arguments permettent de modifier le comportement d'une fonction.

R fournit de très nombreuses fonctions, mais pour réaliser des taches très spécifiques il est possible d'écrire ses propres fonctions ou d'installer des packages supplémentaires.

Par exemple, la fonction `round()` permet d'arrondir une valeur numérique au nombre de décimales spécifié. Par défaut, la fonction `round()` réalise un arrondi à zéro chiffre après la virgule :

```
round(3.141593)
#> [1] 3
```

La fonction `args()` permet de connaitre les différents arguments d'une fonction et les valeurs par défaut qui peuvent leur être associées :

```
args(round)
#> function (x, digits = 0)
#> NULL
```

La fonction `round()` accepte ainsi deux arguments, la valeur à arrondir et le nombre de décimales à conserver (0 par défaut).

Remarque

Les arguments qui possèdent une valeur par défaut sont des **arguments facultatifs** : ils peuvent être omis lorsque la fonction est appelée (la valeur par défaut sera utilisée). À l'inverse, les arguments sans valeurs par défaut doivent être spécifiés : ce sont des **arguments obligatoires**.

Les différents arguments d'une fonction sont séparés par une virgule et sont nommés. Lors de l'appel d'une fonction, si les arguments ne sont pas spécifiés par leur nom, R va associer les valeurs correspondantes dans l'ordre. Dans l'exemple suivant, la valeur `2` est associée au second argument (`digits`) :

```
round(3.141593, 2)
#> [1] 3.14
```

Pour éviter les erreurs et pour faciliter la lecture du code, dans un appel de fonction, spécifiez toujours les arguments par leur nom (à l'exception du premier argument). Les arguments obligatoires doivent être placés en premier, suivis des arguments facultatifs :

```
round(3.141593, digits = 2)
#> [1] 3.14
```

Conseil

Le nom des arguments d'une fonction, leurs rôles et leurs éventuelles valeurs par défaut sont détaillés dans l'aide.

Par exemple, on peut vouloir calculer la moyenne des longueurs d'un assemblage d'artefacts. Cet exemple montre l'utilisation de deux fonctions simples pour calculer une valeur moyenne :

```
x <- c(4, 7, 12) # Combine la longueur de trois artefacts
y <- mean(x)     # Calcule la longueur moyenne
```

La fonction `c()` permet de combiner ("c" pour combiner) nos mesures d'artefacts dans un objet appelé vecteur. Un vecteur est une séquence simple d'éléments du même type. Dans notre exemple, tous les éléments sont des nombres. Il existe deux autres vecteurs couramment utilisés dans R : un vecteur de chaînes de caractères où tous les éléments peuvent être des lettres ou des mots, et un vecteur logique où tous les éléments sont des constantes logiques, soit `TRUE` (vrai) ou `FALSE` (faux). Le vecteur est un type de données fondamental en R que nous utiliserons très souvent. Dans l'exemple ci-dessus, nous avons stocké le résultat de la fonction `c()` dans un objet que nous appelons `x` (nous pouvons l'appeler comme bon nous semble, et nous n'avons pas besoin de créer `x` à l'avance).

La deuxième ligne de code permet de calculer la moyenne de 4, 7 et 12 (c'est-à-dire $(4 + 7 + 12)/3$), puis d'affecter, ou stocker, le résultat dans un objet appelé `y`. Nous pouvons ensuite utiliser `x` et `y` plus tard dans notre flux de travail pour d'autres tâches. C'est utile car cela signifie que nous n'avons pas à recalculer la moyenne à plusieurs reprises.

Les fonctions, telles que `c()` et `mean()` dans l'exemple ci-dessus, sont au cœur du travail avec R, et expliquent en partie la grande polyvalence de R. Elles permettent de gagner beaucoup de temps en minimisant la saisie et le copier-coller, il est donc utile d'investir quelques efforts pour apprendre à utiliser les fonctions, et à écrire vos propres fonctions.

1.5. Packages

1.5.1. Installer et utiliser des packages

Les fonctions sont organisées en **packages**, que vous pouvez télécharger pour étendre l'utilisation de R. Lors d'une première installation de R, un ensemble de packages contenant les fonctions fondamentales est installé :

- Les packages essentiels : `base`, `compiler`, `datasets`, `graphics`, `grDevices`, `grid`, `methods`, `parallel`, `splines`, `stats`, `stats4`, `tcltk`, `tools`, `translations`, `utils`.
- Les packages recommandés : `boot`, `class`, `cluster`, `codetools`, `foreign`, `KernSmooth`, `lattice`, `MASS`, `Matrix`, `mgcv`, `nlme`, `nnet`, `rpart`, `spatial`, `survival`.

L'installation d'un package permet de bénéficier de fonctionnalités supplémentaires, généralement dédiées à une tâche bien spécifique. Pour ne pas se perdre dans la multitude de ressources disponibles, le CRAN propose des répertoires de packages par domaine (*Task Views*) pour faciliter le choix des packages pour une analyse spécifique. Il existe ainsi des répertoires pour les sciences sociales, l'analyse de données environnementales ou encore l'analyse de données spatiales. Pour l'archéologie, il existe une *Task View* non officielle, maintenue par Ben Marwick.

Lorsqu'un package est disponible sur le CRAN, il peut aisément être installé à l'aide de la fonction `install.packages()`[4]. Par exemple, si vous exécutez `install.packages("binford")`, vous allez télécharger automatiquement le package *binford* depuis le CRAN (vous devez être connecté à Internet) et l'installer dans votre bibliothèque. Le package *binford* contient les données de son livre de *Constructing Frames of Reference : An Analytical Method for Archaeological Theory Building Using Ethnographic and Environmental Data Sets* (2001).

Essayez vous-même en exécutant les commandes suivantes pour installer certains des packages que nous utiliserons dans les chapitres suivants. Vous ne devez utiliser

[4] `install.packages()` permet également d'installer des packages publiés sur d'autres dépôts que le CRAN, comme Bioconductor.

`install.packages()` qu'une seule fois (par ordinateur et par utilisateur) : vous n'avez pas besoin de le faire à chaque nouvelle session de travail. Remarquez comment nous utilisons la fonction `c()` ici pour créer un vecteur de noms de packages (chaînes de caractères) sur lequel la fonction `install.packages()` peut travailler :

```
# Installer un seul package
install.packages("folio")

# Installer plusieurs packages à la fois
pkg <- c("dplyr", "ggplot2", "knitr", "readr", "rmarkdown", "scales", "stringr", "tidyr")
install.packages(pkg)
```

Conseil

La plupart des packages publiés sur le CRAN font l'objet de mises à jour régulières. Pour maintenir à jour les packages installés sur votre ordinateur, la fonction `old.packages()` permet de lister les packages pour lesquels il existe une nouvelle version et `update.packages()` permet de télécharger et d'installer les nouvelles versions.

Installer un nouveau package est une condition nécessaire, mais pas suffisante, pour pouvoir l'utiliser. Au lancement, R ne charge pas tous les packages installés, mais uniquement les packages de base[5]. Dans l'exemple suivant, si on utilise la fonction `data()` pour charger le jeux de données `intcal20` du package *folio* sans que ce dernier soit chargé. R ne sait pas où chercher l'objet et retourne un avertissement :

```
data(intcal20)
#> Warning in data(intcal20): jeu de données 'intcal20' introuvable
```

Lorsque R installe un package, il le télécharge dans votre bibliothèque. Vous devez utiliser la fonction `library()` pour mettre le contenu du package à la disposition de votre session R actuelle. Chaque fois que vous lancez R, vous devrez exécuter `library()` pour utiliser les fonctions des package que vous avez précédemment installés. Une bonne pratique consiste à placer les appels à la fonctions `library()` parmi les premières lignes de votre script, afin que les autres utilisateurs puissent rapidement voir quels paquets ils devront avoir pour exécuter votre code.

Il est ainsi nécessaire de charger un package à l'aide de la fonction `library()` avant de pouvoir l'utiliser[6] :

[5] La liste des packages chargés par défaut au lancement de R peut être obtenue avec `getOption('defaultPackages')`.

[6] Dans le cas présent, une alternative serait de spécifier l'argument `package` de la fonction `data()`.

```
library("folio")
data(intcal20)
```

L'une des forces de R, la communauté dynamique de chercheurs-développeurs, est aussi l'une de ses faiblesses. En effet, cela signifie que certains packages sont mis à jour fréquemment, et d'autres non. Parfois, ces mises à jour peuvent modifier le fonctionnement de votre code, ou l'empêcher complètement de fonctionner. Nous aborderons des solutions détaillées à ce problème par la suite. Pour l'instant, nous noterons simplement que c'est une bonne pratique d'inclure la sortie de `sessionInfo()` dans les résultats de votre analyse, car elle vous indique les numéros de version spécifiques de tous les paquets utilisés. Cela signifie que s'il y a des changements drastiques dans certains des packages que vous utilisez au cours de la vie de votre projet, vous avez un enregistrement de la dernière version des packages qui a fonctionné pour vous.

1.5.2. Limiter les dépendances

Si les packages de base de R offrent de nombreuses possibilités, il est courant d'avoir besoin de fonctionnalités supplémentaires au cours d'une étude. Pour une analyse spécifique, il est très probable qu'il existe déjà un ou plusieurs packages offrant les fonctionnalités recherchées et installable depuis le CRAN. Cette offre pléthorique a cependant un revers : à chaque package supplémentaire utilisé dans votre projet, vous augmentez le risque de voir apparaître des problèmes liés à ces dépendances[7].

Par exemple, *FactoMineR* est sans doute le package le plus utilisé pour l'analyse de données multivariées. *FactoMineR* possède 15 dépendances directes : d'autres packages dont il utilise les fonctionnalités. Cependant, chacune de ces dépendances est susceptible d'avoir elle même des dépendances, et ainsi de suite, si bien que *FactoMineR* a en réalité une longue chaîne de 104 (!) dépendances (fig. 1.1).

Qu'arrivera-t-il alors si une seule des ces dépendances change drastiquement, arrête de fonctionner ou disparaît tout simplement (fig. 1.2) ? Pour réduire ce risque et sortir de cet enfer des dépendances :

- Évitez d'utiliser un package particulier quand la même tâche peut être réalisée en R basique (écrivez vos propres fonctions !).
- Quand cela est possible, préférez les packages qui n'ont pas (ou peu) de dépendances.
- N'utilisez pas la version de développement d'un package, mais installez toujours la version stable depuis le CRAN.

Le tableau n'est cependant pas totalement noir. Les packages publiés sur le CRAN doivent se conformer à des règles strictes et sont continuellement testés dans différentes

[7] Sur le sujet, voir les billets regroupés sur le site du tinyverse.

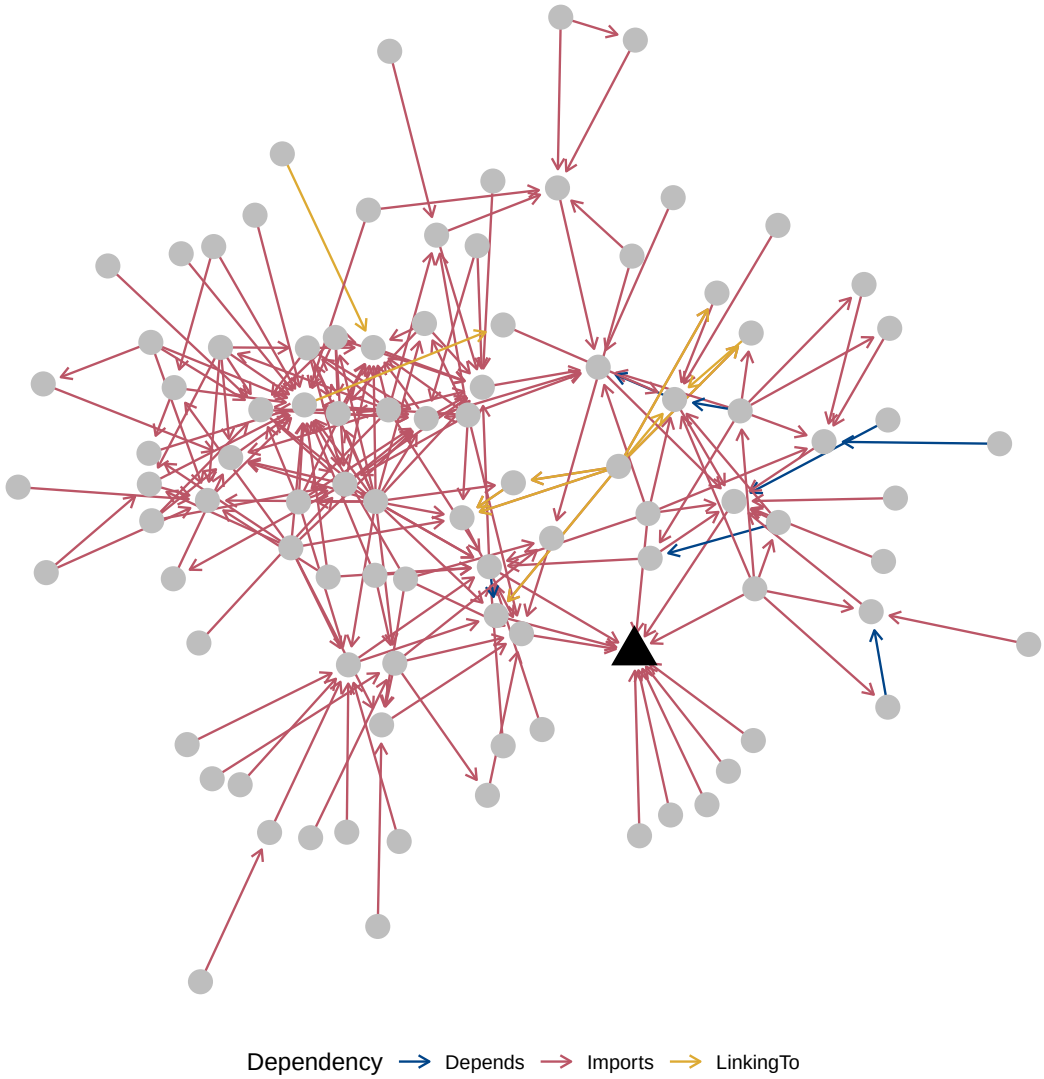


FIG. 1.1. : Réseau des dépendances du package *FactoMineR* (hors packages de base). Les noms des packages ont été omis pour faciliter la lecture (*FactoMineR* est représenté par un triangle noir, les autres packages sont représentés par des points gris).

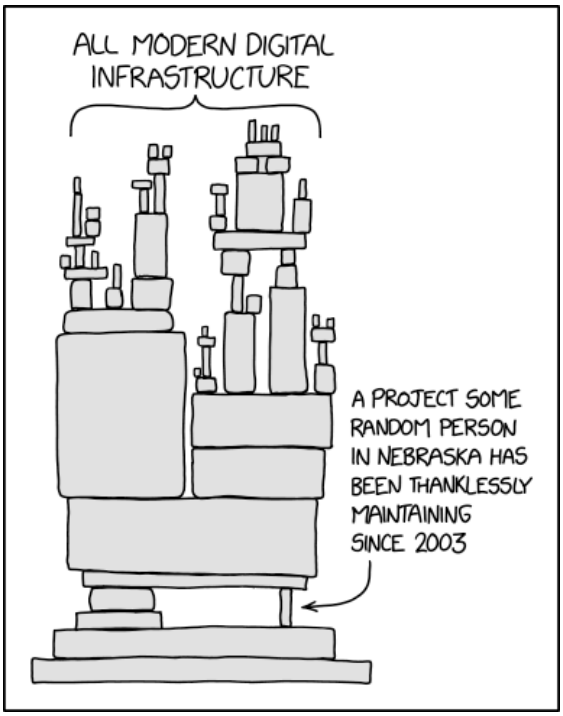


FIG. 1.2. : Dependency. "Someday ImageMagick will finally break for good and we'll have a long period of scrambling as we try to reassemble civilization from the rubble." (CC BY-NC 2.0 MUNROE, 2020).

configurations (systèmes d'exploitation et versions de R), obligeant les développeurs à réagir rapidement lorsqu'un bug est observé. De plus, lorsqu'un package n'est plus disponible sur le CRAN, les versions antérieures sont archivées et restent disponibles au téléchargement. Enfin, des initiatives comme rOpenSci œuvrent pour garantir un écosystème fonctionnel, en favorisant l'évaluation et la maintenance des packages.

1.6. Obtenir de l'aide

1.6.1. Utiliser la documentation

La maîtrise d'une langue étrangère comme l'allemand ou le chinois demande du temps, de la pratique et des hauts et des bas. L'apprentissage d'un langage de programmation comme R est un processus similaire, et vous devez vous attendre à des hauts et des bas à mesure que R devient un élément de plus en plus central de votre flux de travail. R est fourni avec une documentation intégrée très complète, très structurée et contenant très souvent des exemples de code que vous pouvez exécuter pour explorer le fonctionnement d'une fonction.

Lorsque vous ne savez pas comment utiliser une fonction particulière (difficile de mémoriser les 1184 fonctions des packages de base), votre premier réflexe doit être de consulter la documentation. La documentation couvre l'ensemble des fonctions, jeux de données et autres objets présents dans les différents packages.

Pour un objet donné, on peut consulter la documentation associée en utilisant soit la fonction `help()`, soit l'opérateur `?`. Par exemple, pour accéder à l'aide de la fonction `round()` :

```
help(round)

?round
```

Si l'objet recherché appartient à un package qui n'est pas chargé, R ne saura pas où chercher la documentation. Dans l'exemple suivant, la fonction `lda()` appartient au package *MASS* qui n'est pas chargé :

```
help(lda)
```

Pour accéder à la documentation, il faudra alors charger le package au préalable ou le spécifier dans la fonction `help()` :

```
help(lda, package = "MASS")
```

Si on ignore le nom de la fonction à utiliser, deux approches permettent d'explorer la documentation :

- On peut parcourir la documentation d'un package particulier, dont on sait qu'il doit contenir la fonction recherchée (par exemple, `help(package = "MASS")` permet d'afficher le sommaire de la documentation du package *MASS*).
- On peut rechercher dans l'aide par mot-clé à l'aide de l'opérateur `??` ou de la fonction `help.search()`.

Par exemple, pour chercher toutes les pages de la documentation contenant le terme "correlation"[8] :

```
help.search("correlation")

??correlation
```

Toutes les pages d'aide sont structurées de la même façon. Généralement, on y trouve au moins les sections suivantes :

Description Une courte description de ce que fait la fonction.
Usage Un exemple d'utilisation de la fonction présentant l'ensemble des arguments et leurs éventuelles valeurs par défaut.
Arguments La liste des arguments de la fonction précisant leur nom, leur rôle et la valeur ou le type d'objet qui est attendu.
Details Une explication détaillée du comportement de la fonction.
Value Une description de ce qui est retourné lorsque la fonction est exécutée.
References Les références bibliographiques associées à la fonction.
See Also Une suggestion de fonctions similaires.
Examples Quelques exemples illustrant différents cas d'utilisation de la fonction. Ces exemples peuvent être exécutés directement avec la fonction `example()`.

Pour exécuter les exemples illustrant la documentation de la fonction `mean()` :

```
example(mean)
#>
#> mean> x <- c(0:10, 50)
#>
#> mean> xm <- mean(x)
#>
#> mean> c(xm, mean(x, trim = 0.10))
#> [1] 8.75 5.50
```

[8] Attention, la documentation est écrite en anglais. Pour chercher les pages de la documentation concernant les méthodes de régression linéaire il vous faudra utiliser `??"linear model"` (ou similaire).

Conseil

Si vous faites face à une difficulté, il fort probable quelqu'un se soit déjà posé la même question que vous et que la réponse se trouve sur Stack Overflow ou sur les listes de diffusions R-help et R-devel (actives depuis 1996). Dans le cas contraire, vous pourrez y exposer votre problème, mais il vous faudra apprendre à écrire des exemples reproductibles.

1.6.2. Écrire des exemples reproductibles

Si vous souhaitez obtenir de l'aide en ligne auprès de la communauté des utilisateurs de R, sur Stack Overflow[9] ou sur les listes de diffusion, la seule description de votre difficulté ne sera pas suffisante. Vous devez permettre aux autres de reproduire le problème sur leur machine pour qu'ils puissent vous proposer une solution. Pour cela, inutile de diffuser l'intégralité de votre code et de vos données, préparez un exemple qui soit :

- Minimal : utilisez le moins de code possible tout en produisant le même problème.
- Complet : fournissez tous les éléments (version de R, packages utilisés, etc.) dont un tiers a besoin pour reproduire votre problème. Utilisez les données d'exemple de R.
- Reproductible : le code que vous vous apprêtez à fournir doit reproduire le problème.

Ces trois aspects sont détaillés dans l'aide de Stack Overflow, il existe également une question dédié à l'écriture d'un exemple reproductible (*reprex*) avec R.

Comme le souligne Wickham (2016), la plupart du temps, l'écriture d'un exemple reproductible vous permettra d'identifier et de résoudre vous-même le problème.

Conseil

L'utilisation du package *reprex*[a] peut vous aider à écrire un exemple reproductible.

[a] Voir https://reprex.tidyverse.org/articles/articles/learn-reprex.html.

[9] Avant de poster votre question sur Stack Overflow, prenez le temps de bien lire les recommandations d'écriture.

2. Structures de données

Le vecteur (`vector`) est la structure de base de R, à partir de laquelle toutes les autres sont construites. Un vecteur est simplement un ensemble d'**éléments**.

Un vecteur peut être un vecteur atomique (`atomic`), si tous ses éléments sont de même **type**, ou une liste (`list`), si ses éléments sont de types différents (tab. 2.1). Un vecteur peut également avoir des **attributs** (sorte de métadonnées).

Tab. 2.1. : Structures de données des objets de R.

Dimensions	Objet homogène	Objet hétérogène
1	`atomic`	`list`
2	`matrix`	`data.frame`
n	`array`	-

2.1. Vecteurs atomiques (`atomic`)

Un vecteur atomique possède deux propriétés : son type et sa longueur (le nombre d'éléments qu'il contient ; fig. 2.1). Les quatre principaux types de R sont les suivants :

- `logical` : booléen.
- `integer` : nombre entier.
- `double` : nombre réel, avec ou sans partie décimale.
- `character` : chaîne de caractère.

Fig. 2.1. : Un vecteur est un ensemble d'éléments de même type.

2.1.1. Création

Chacun de ces types possède une syntaxe particulière pour former des vecteur contenant un unique élément (scalaires) :

```
x <- TRUE          # logical
x <- 1L            # integer
x <- 10.5          # double
x <- "archéologie" # character
```

- Un vecteur logique ne peut pendre que deux valeurs `TRUE` (vrai) ou `FALSE` (faux). `TRUE` et `FALSE` sont des mots réservés dans R : ils ne peuvent pas être utilisés comme identifiants pour nommer des variables ou des fonctions.
- Un nombre réel sans partie décimale sera toujours interprété comme étant de type `double` et non de type `integer`.
- Un vecteur de type `integer` s'écrit avec une syntaxe particulière : les nombres doivent être suivis d'un `L` pour les distinguer du type `double` et ne doivent pas avoir de partie décimale.
- Une chaîne de caractère doit toujours être encadrée par des guillemets simples (`'`) ou doubles (`"`). Si les guillemets sont manquantes, R l'interprétera comme un nom d'objet (et cherchera l'objet correspondant).

La fonction `c()` (*combine*) permet de construire des vecteurs longs :

```
vec_log <- c(TRUE, FALSE, TRUE) # logical
vec_int <- c(1L, 2L, 3L)        # integer
vec_num <- c(1.2, 2.1, 3.2)     # double
vec_cha <- c("A", "B", "C")     # character
```

2.1.2. Description

La fonction `typeof()` permet de connaitre le type d'un vecteur et la fonction `length()` sa longueur :

```
typeof(vec_cha)
#> [1] "character"

length(vec_cha)
#> [1] 3
```

La famille de fonctions `is.*()` permet de tester si un vecteur est d'un type particulier :

```
is.logical(vec_log)
#> [1] TRUE

is.integer(vec_int)
#> [1] TRUE
```

```
is.double(vec_num)
#> [1] TRUE

is.character(vec_cha)
#> [1] TRUE
```

2.1.3. Conversion

Tous les éléments d'un vecteur atomique sont du même type. Lors de la combinaison d'éléments de types différents, R va réaliser une conversion implicite pour tenter de représenter tous les éléments du vecteur de manière raisonnable. Le type du vecteur final est alors déterminé par la hiérarchie `logical` < `integer` < `double` < `character`.

Un vecteur contenant au moins une chaîne de caractères sera donc de type `character` :

```
x <- c(1, "B", 3)
typeof(x)
#> [1] "character"

x <- c(TRUE, "B", TRUE)
typeof(x)
#> [1] "character"
```

Un vecteur contenant au moins un nombre réel sera de type `double` (même s'il ne possède pas de partie décimale) :

```
x <- c(1L, 2.1, 3L)
typeof(x)
#> [1] "double"

x <- c(TRUE, 2.1, TRUE)
typeof(x)
#> [1] "double"
```

La création d'un `integer` avec une partie décimale sera automatiquement converti en type `double` avec un avertissement :

```
x <- 10.5L
typeof(x)
#> [1] "double"
```

Il est possible de convertir explicitement un vecteur d'un type à l'autre à l'aide de la famille de fonctions `as.*()` :

```
as.integer(c(1.2, 2.1, 3.2))
#> [1] 1 2 3

as.numeric(c(TRUE, FALSE, TRUE))
#> [1] 1 0 1

as.character(c(1.2, 2.1, 3.2))
#> [1] "1.2" "2.1" "3.2"
```

Les valeurs d'un vecteur numérique peuvent être converties en valeurs logiques : les valeurs égales à 0 deviennent `FALSE`, tout autre valeur non nulle devient `TRUE` :

```
as.logical(c(1, 0, 3))
#> [1]  TRUE FALSE  TRUE
```

En cas de conversion impossible, des valeurs manquantes sont générées et un avertissement est retourné :

```
as.numeric(c("A", "B", "C"))
#> Warning: NAs introduits lors de la conversion automatique
#> [1] NA NA NA
```

2.1.4. Attributs

Les attributs d'un vecteur peuvent être vus comme des métadonnées : des données qui fournissent des informations sur les données du vecteur.

Les attributs peuvent être spécifiés individuellement à l'aide de la fonction `attr()`, sous la forme de paires associant un nom et une valeur. Dans l'exemple suivant, deux attributs nommés `a` et `b` sont associés au vecteur `x` avec pour valeurs `"xyz"` et `1 2 3`, respectivement :

```
x <- 1:5

attr(x, "a") <- "xyz"
attr(x, "a")
#> [1] "xyz"

attr(x, "b") <- 1:3
attr(x, "b")
#> [1] 1 2 3
```

Remarque

La syntaxe particulière `attr(x, "a") <- "xyz"` indique qu'il s'agit d'une fonction de remplacement. Ces fonctions particulières modifient en place l'objet correspondant au premier argument (`x`).

L'utilisation des attributs requiert un peu de prudence, car à quelques exceptions près les attributs ne sont pas conservés lors d'opérations sur les vecteurs (comme lors de l'extraction de sous-ensembles). Il existe cependant deux attributs particuliers qui sont généralement conservés : les noms (`names`) et la dimension (`dim`, qui transforme un vecteur en matrice).

Il est possible d'associer des noms aux différents éléments d'un vecteur, soit au moment de sa création, soit *a posteriori* avec la fonction `names()` (qui permet également d'extraire les noms) :

```
# Un nom est associé à chaque élément
x <- c(a = 1, b = 2, c = 3, d = 4, e = 5)
x
#> a b c d e
#> 1 2 3 4 5

# Les noms sont spécifiés à l'aide d'un vecteur
names(x) <- c("f", "g", "h", "i", "j")
x
#> f g h i j
#> 1 2 3 4 5

# Extraire les noms des différents éléments
names(x)
#> [1] "f" "g" "h" "i" "j"

# Supprimer les noms
names(x) <- NULL
names(x)
#> NULL

x
#> [1] 1 2 3 4 5
```

Remarque

L'utilisation des attributs permet de créer des vecteurs particuliers, appelés `factor`, qui représentent des variables qualitatives (nominales ou ordinales).

2.2. Listes (`list`)

Une liste est un vecteur particulier, dont les éléments peuvent être de n'importe quel type (fig. 2.2).

FIG. 2.2. : Une liste est un vecteur dont les éléments peuvent être de différents types.

Une liste peut être créée avec avec la fonction `list()` :

```
# Création d'une liste contenant 4 éléments
x <- list(
  TRUE,
  1L,
  c(1.2, 2.1),
  c("X", "Y", "Z")
)
x
#> [[1]]
#> [1] TRUE
#>
#> [[2]]
#> [1] 1
#>
#> [[3]]
#> [1] 1.2 2.1
#>
#> [[4]]
#> [1] "X" "Y" "Z"

# Une liste est un objet de type 'list'
typeof(x)
#> [1] "list"

# Longueur de la liste (nombre d'éléments)
```

```
length(x)
#> [1] 4

# Longueur des différents éléments de la liste
lengths(x)
#> [1] 1 1 2 3

# Les éléments de la liste peuvent être nommés
names(x) <- c("a", "b", "c", "d")
x
#> $a
#> [1] TRUE
#>
#> $b
#> [1] 1
#>
#> $c
#> [1] 1.2 2.1
#>
#> $d
#> [1] "X" "Y" "Z"
```

Les listes permettent ainsi de créer et de manipuler des objets complexes, qui peuvent contenir d'autres listes (on parle alors de vecteurs récursifs) :

```
x <- list(
  list(
    c(TRUE, FALSE),
    c("X", "Y", "Z")
  ),
  c(1, 2)
)
x
#> [[1]]
#> [[1]][[1]]
#> [1]  TRUE FALSE
#>
#> [[1]][[2]]
#> [1] "X" "Y" "Z"
#>
#>
#> [[2]]
#> [1] 1 2
```

La fonction `c()` permet de combiner plusieurs listes en une seule :

```
x <- c(
  list(
    c(TRUE, FALSE),
    c("X", "Y", "Z")
  ),
  list(1, 2)
)
x
#> [[1]]
#> [1]  TRUE FALSE
#>
#> [[2]]
#> [1] "X" "Y" "Z"
#>
#> [[3]]
#> [1] 1
#>
#> [[4]]
#> [1] 2
```

Un vecteur atomique est converti en liste avant d'être combiné :

```
x <- c(
  list(
    c(TRUE, FALSE),
    c("X", "Y", "Z")
  ),
  c(1, 2)
)
x
#> [[1]]
#> [1]  TRUE FALSE
#>
#> [[2]]
#> [1] "X" "Y" "Z"
#>
#> [[3]]
#> [1] 1
#>
#> [[4]]
#> [1] 2
```

2.3. Matrices (matrix et array)

Un vecteur peut être transformé en un tableau à deux (`matrix`) ou plusieurs (`array`) dimensions, en spécifiant l'attribut `dim`. Une matrice, au sens mathématique, est un tableau rectangulaire à deux dimensions : une matrice possède m lignes et n colonnes et contient $m \times n$ éléments du même type (fig. 2.3).

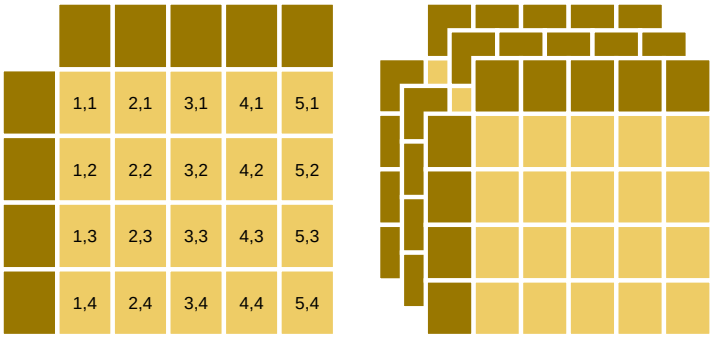


FIG. 2.3. : Une matrice (gauche) est un tableau à deux dimensions dont tous les éléments sont de même type. Un array (droite) peut être vu comme un empilement de matrices (formant ainsi un cube de données).

```
# Soit un vecteur contenant 12 éléments
x <- 1:12
x
#>  [1]  1  2  3  4  5  6  7  8  9 10 11 12

# Le vecteur est transformé en une matrice de 2 lignes et 6 colonnes
dim(x) <- c(2, 6)
dim(x)
#> [1] 2 6
x
#>      [,1] [,2] [,3] [,4] [,5] [,6]
#> [1,]    1    3    5    7    9   11
#> [2,]    2    4    6    8   10   12
```

Les caractéristiques d'une matrice peuvent être retrouvées :

```
typeof(x) # Type des éléments
#> [1] "integer"
length(x) # Nombre d'éléments (valeurs)
#> [1] 12
nrow(x)   # Nombre de lignes
```

```
#> [1] 2
ncol(x)   # Nombre de colonnes
#> [1] 6
```

La fonction `matrix()` permet de créer des matrices en spécifiant le nombre de lignes et de colonnes :

```
# Par défaut, une matrice est remplie colonne par colonne
x <- matrix(1:12, nrow = 3, ncol = 4)
x
#>      [,1] [,2] [,3] [,4]
#> [1,]    1    4    7   10
#> [2,]    2    5    8   11
#> [3,]    3    6    9   12

# Ce comportement peut être modifié
y <- matrix(1:12, nrow = 3, ncol = 4, byrow = TRUE)
y
#>      [,1] [,2] [,3] [,4]
#> [1,]    1    2    3    4
#> [2,]    5    6    7    8
#> [3,]    9   10   11   12
```

Plusieurs matrices peuvent être combinées en une seule, par lignes (`rbind()`) ou par colonnes (`cbind()`) :

```
# Combine les lignes de deux matrices
rbind(x, y)
#>      [,1] [,2] [,3] [,4]
#> [1,]    1    4    7   10
#> [2,]    2    5    8   11
#> [3,]    3    6    9   12
#> [4,]    1    2    3    4
#> [5,]    5    6    7    8
#> [6,]    9   10   11   12

# Combine les colonnes de deux matrices
cbind(x, y)
#>      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8]
#> [1,]    1    4    7   10    1    2    3    4
#> [2,]    2    5    8   11    5    6    7    8
#> [3,]    3    6    9   12    9   10   11   12
```

Enfin, il est possible de spécifier/extraire les noms de lignes et de colonnes avec les fonctions `rownames()` et `colnames()` ou avec la fonction `dimnames()` :

```
rownames(x) <- c("X", "Y", "Z")
colnames(x) <- c("A", "B", "C", "D")
dimnames(x)
#> [[1]]
#> [1] "X" "Y" "Z"
#>
#> [[2]]
#> [1] "A" "B" "C" "D"

x
#>   A B C  D
#> X 1 4 7 10
#> Y 2 5 8 11
#> Z 3 6 9 12

dimnames(y) <- list(c("X", "Y", "Z"), c("A", "B", "C", "D"))
rownames(y)
#> [1] "X" "Y" "Z"
colnames(y)
#> [1] "A" "B" "C" "D"

y
#>   A  B  C  D
#> X 1  2  3  4
#> Y 5  6  7  8
#> Z 9 10 11 12
```

2.4. Tableaux de données (`data.frame`)

Le `data.frame` est probablement le concept le plus important de R : il permet de représenter un tableau de données rectangulaire dont les colonnes peuvent contenir des informations de différentes natures. Un `data.frame` est un cas particulier de `list` dont tous les éléments sont nommés et ont la même longueur : chaque élément correspond à une colonne du tableau[1]. Chaque colonne d'un `data.frame` peut ainsi être d'un type différent (contrairement aux matrices dont tous les éléments sont du même type ; fig. 2.4).

[1] Ainsi, la fonction `names()` est équivalente à `rownames()` et la fonction `length()` retourne le nombre de colonne du tableau (équivalent à `ncol()`).

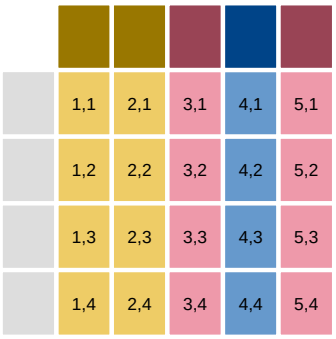


FIG. 2.4. : Un data.frame est un tableau à deux dimensions dont les colonnes peuvent être de différents types.

Un tableau de données peut être créé avec la fonction `data.frame()` en spécifiant les différentes colonnes (si les arguments sont nommés, ils seront utilisés comme noms de colonne) :

```
x <- data.frame(
  a = 1:3,
  b = c("x", "y", "z")
)
x
#>   a b
#> 1 1 x
#> 2 2 y
#> 3 3 z
```

Un `data.frame` est une liste :

```
typeof(x) # Type list
#> [1] "list"
length(x) # Nombre d'éléments (colonnes)
#> [1] 2
```

Un `data.frame` est un tableau à deux dimensions :

```
nrow(x) # Nombre de lignes
#> [1] 3
ncol(x) # Nombre de colonnes
#> [1] 2
```

Lignes et colonnes d'un `data.frame` sont nommées :

```
rownames(x) # Noms de lignes
#> [1] "1" "2" "3"
colnames(x) # Noms de colonnes
#> [1] "a" "b"
```

Remarque

Sauf instruction spécifique lors de la création d'un `data.frame`, les noms de lignes sont générés automatiquement à partir d'une séquence de nombres entiers. Malgré les apparences, il s'agit bien de noms de lignes (de type `character`) et ils ne doivent pas être confondu avec les indices des lignes.

Comme pour les matrices, il est possible de combiner les lignes ou les colonnes de plusieurs `data.frame` en un seul tableau de données avec `rbind()` et `cbind()`. Dans le cas de la combinaison de colonnes, il pourra être préférable de réaliser des jointures.

3. Extraction et remplacement

R possède un mécanisme très puissant pour naviguer parmi les valeurs d'un objet, afin d'en extraire ou d'en remplacer tout ou partie. Ce mécanisme obéit cependant à des règles parfois complexes et peut aboutir à des résultats inattendus si elles ne sont pas bien comprises.

La façon la plus simple de sélectionner ou d'extraire des valeurs consiste à utiliser les opérateurs `[` ou `[[` : `[` permet d'extraire un sous-ensemble, tandis que `[[` permet d'extraire un unique élément. Cette distinction correspond aux relations d'appartenance et d'inclusion de la théorie des ensembles en mathématiques[1] (cette distinction peut paraître un peu obscure au premier abord, mais prend tout son sens lors de la manipulation de listes).

Ces opérateurs peuvent être utilisés conjointement avec l'affectation (`<-`) pour remplacer des valeurs. On écrira ainsi : `x[i] <- y` où `i` correspond à l'indice (position) des valeurs à remplacer dans le vecteur `x` et `y` correspond au vecteur de nouvelles valeurs.

3.1. Vecteurs atomiques

Il existe différentes façons d'extraire un sous-ensemble d'éléments au sein d'un vecteur à l'aide de l'opérateur `[` :

```
x <- c(1.3, 2.4, 3.5, 4.6, 5.7)
```

- Il est possible de spécifier les indices (positions) des éléments à conserver ou à enlever avec des nombres entiers :

```
# Sélection du premier, troisième et cinquième élément
x[c(1, 3, 5)]
#> [1] 1.3 3.5 5.7

# Suppression du second et du quatrième élément
x[-c(2, 4)]
#> [1] 1.3 3.5 5.7

# Zéro est un cas particulier
```

[1] Par exemple, si M est un ensemble (collection) tel que $M = \{1, 2, 3\}$, alors 1, 2 et 3 sont des éléments de M, tandis que $\{1, 2\}$ ou $\{3\}$ sont des sous-ensembles de M.

```
# (retourne un vecteur de longueur nulle)
x[0]
#> numeric(0)
```

- Il est possible de spécifier si oui (`TRUE`) ou non (`FALSE`) on souhaite conserver des éléments :

```
x[c(TRUE, FALSE, TRUE, FALSE, TRUE)]
#> [1] 1.3 3.5 5.7
```

- Si rien n'est spécifié, le vecteur d'origine est retourné :

```
x[]
#> [1] 1.3 2.4 3.5 4.6 5.7
```

- Si les éléments sont nommés, on peut les sélectionner par leur nom :

```
names(x) <- c("a", "b", "c", "d", "e")
x[c("a", "c", "e")]
#>   a   c   e
#> 1.3 3.5 5.7
```

L'utilisation de `[[` permet d'extraire un seul et unique élément au lieu d'un sous-ensemble. Dans le cas des vecteurs atomiques, la différence entre `[` et `[[` n'est pas évidente[2] et est sans doute mieux illustrée avec un exemple :

```
x[1] # L'attribut names est conservé (sous-ensemble)
#>   a
#> 1.3

x[[1]] # L'attribut names n'est pas conservé (élément)
#> [1] 1.3
```

Les éléments sélectionnés peuvent être remplacés en affectant de nouvelles valeurs :

```
# Remplacement du premier et du troisième élément
x[c(1, 3)] <- c(100, 300)
x
#>     a     b     c     d     e
#> 100.0   2.4 300.0   4.6   5.7
```

Lors de l'affectation de nouvelles valeurs dans un vecteur existant, il faut veiller à plusieurs aspects :

[2] Hormis qu'une tentative de sélection de plusieurs éléments avec `[[` produira une erreur.

- La duplication d'indices :

```
x <- 1:5
x[c(1, 1, 1)] # Duplique les valeurs
#> [1] 1 1 1
```

- L'ordre des indices :

```
x <- 1:5
x[c(5, 3, 1)] # Change l'ordre des valeurs
#> [1] 5 3 1
```

- Les mécanismes de conversions implicites :

```
x <- 1:5
typeof(x)
#> [1] "integer"

x[c(1, 3)] <- c(100.1, 300.3)
typeof(x)
#> [1] "double"
```

- La taille du vecteur de remplacement. Si ce dernier est trop petit, il sera **recyclé** : ses valeurs seront réutilisées autant que nécessaire :

```
x <- 1:5
x[c(1, 2, 3, 4)] <- c(100, 200)
x
#> [1] 100 200 100 200   5
```

3.2. Listes

L'extraction de sous-ensembles dans une liste s'opère de la même façon que pour un vecteur atomique :

```
x <- list(
  a = c(TRUE, FALSE),
  b = c(1, 2, 3, 4, 5),
  c = c("X", "Y", "Z")
)
x
#> $a
#> [1]  TRUE FALSE
#>
#> $b
```

```
#> [1] 1 2 3 4 5
#>
#> $c
#> [1] "X" "Y" "Z"

# Sélection du premier et du troisième élément
x[c(1, 3)] # Par position
#> $a
#> [1]  TRUE FALSE
#>
#> $c
#> [1] "X" "Y" "Z"

x[c("a", "c")] # Par nom
#> $a
#> [1]  TRUE FALSE
#>
#> $c
#> [1] "X" "Y" "Z"
```

L'opérateur `[` retourne toujours une liste, y compris lorsqu'un unique élément est sélectionné. Pour extraire un unique élément *hors* de la liste, il est possible d'utiliser l'opérateur `[[` :

```
y <- x[1]
y
#> $a
#> [1]  TRUE FALSE
typeof(y)
#> [1] "list"

z <- x[[1]]
z
#> [1]  TRUE FALSE
typeof(z)
#> [1] "logical"
```

Si les éléments de la liste sont nommés, on peut extraire un unique élément en le sélectionnant par son nom avec `[[` ou avec l'opérateur `$` (dollar) :

```
x[["a"]]
#> [1]  TRUE FALSE
```

```
x$a
#> [1]  TRUE FALSE
```

Cette distinction entre `[` et `[[` est particulièrement importante lorsqu'on souhaite remplacer des valeurs dans une liste (puisque `[` retourne toujours une liste, la valeur de remplacement doit également être une liste) :

```
x[1] <- list(1:10)
x
#> $a
#>  [1]  1  2  3  4  5  6  7  8  9 10
#>
#> $b
#> [1] 1 2 3 4 5
#>
#> $c
#> [1] "X" "Y" "Z"

x[[1]] <- c(TRUE, FALSE)
x
#> $a
#> [1]  TRUE FALSE
#>
#> $b
#> [1] 1 2 3 4 5
#>
#> $c
#> [1] "X" "Y" "Z"
```

L'opérateur `[[` permet également de supprimer un élément dans une liste en lui affectant la valeur `NULL` :

```
x[[2]] <- NULL
x
#> $a
#> [1]  TRUE FALSE
#>
#> $c
#> [1] "X" "Y" "Z"
```

3.3. Matrices

La sélection d'un sous-ensemble dans un tableau à deux dimensions (`matrix` ou `array`) est une généralisation de la sélection dans un vecteur. Il suffit de spécifier les indices des lignes et des colonnes, séparés par une virgule :

```
x <- matrix(1:12, nrow = 3, ncol = 4)
rownames(x) <- c("x", "y", "z")
colnames(x) <- c("A", "B", "C", "D")
x
#>   A B C  D
#> x 1 4 7 10
#> y 2 5 8 11
#> z 3 6 9 12

# Sélection des lignes 2 à 3 et des colonnes 1 à 3
x[2:3, 1:3]
#>   A B C
#> y 2 5 8
#> z 3 6 9

x[-1, -4]
#>   A B C
#> y 2 5 8
#> z 3 6 9

x[c(FALSE, TRUE, TRUE), c("A", "B", "C")]
#>   A B C
#> y 2 5 8
#> z 3 6 9
```

Si rien n'est spécifié, toutes les lignes ou toutes les colonnes sont sélectionnées :

```
# Sélection de toutes les lignes et des colonnes 1 à 3
x[, 1:3]
#>   A B C
#> x 1 4 7
#> y 2 5 8
#> z 3 6 9

# Sélection de toutes les colonnes et des lignes 2 à 3
x[2:3, ]
#>   A B C  D
```

```
#> y 2 5 8 11
#> z 3 6 9 12
```

Par défaut, `[` **simplifie** les résultats à la plus petite dimension possible. La sélection d'une unique valeur, d'une unique ligne ou d'une unique colonne, ne retourne donc pas une matrice mais un vecteur. La structure de l'objet d'origine peut cependant être conservée à l'aide de l'argument `drop` :

```
# Sélection de la première colonne
x[, 1] # Retourne un vecteur
#> x y z
#> 1 2 3

x[, 1, drop = FALSE] # Retourne une matrice
#>   A
#> x 1
#> y 2
#> z 3
```

Avertissement

Un vecteur sans attribut `dim` n'a pas de dimension et ne doit pas être confondu avec une matrice à une seule ligne ou une seule colonne !

Les matrices sont des vecteurs possédant un attribut particulier (`dim`), il est donc également possible de sélectionner un sous-ensemble à l'aide d'un unique vecteur (souvenez-vous que les matrices sont remplies par colonnes) :

```
# Sélection des première et troisième valeurs
x[c(1, 3)]
#> [1] 1 3

# Remplacement des première et troisième valeurs
x[c(1, 3)] <- 0
x
#>   A B C  D
#> x 0 4 7 10
#> y 2 5 8 11
#> z 0 6 9 12
```

Enfin, il est possible d'utiliser une matrice pour sélectionner un sous-ensemble d'un tableau à deux dimensions ou plus. Cette matrice doit contenir les coordonnées (lignes et colonnes) des valeurs à sélectionner. La première colonne doit ainsi contenir les indices

des lignes à sélectionner, la seconde colonne les indices des colonnes et ainsi de suite pour les cas de dimensions supérieures (`array`) :

```
# On veut sélectionner les cellules (1,2), (2,3) et (3,4)
y <- matrix(c(1, 2, 3, 2, 3, 4), ncol = 2)

# La première colonne contient les indices des lignes
# La seconde colonne contient les indices des colonnes
y
#>      [,1] [,2]
#> [1,]    1    2
#> [2,]    2    3
#> [3,]    3    4

x[y]
#> [1]  4  8 12
```

3.4. Tableaux de données

Pour extraire ou remplacer des valeurs dans un `data.frame`, on peut utiliser deux stratégies. Les `data.frame` ont en effet les caractéristiques des listes et des matrices :

```
x <- data.frame(
  a = c(TRUE, FALSE, TRUE, TRUE),
  b = c(1, 2, 3, 4),
  c = c("A", "B", "C", "D")
)

# Comme une liste !
x[c("a", "b", "c")] # Colonnes nommées "a", "b" et "c"
#>       a b c
#> 1  TRUE 1 A
#> 2 FALSE 2 B
#> 3  TRUE 3 C
#> 4  TRUE 4 D
x[[2]]              # Deuxième colonne uniquement
#> [1] 1 2 3 4
x$b                 # Colonne nommée "b"
#> [1] 1 2 3 4

# Comme une matrice !
x[, c("a", "b", "c")]
```

```
#>       a b c
#> 1  TRUE 1 A
#> 2 FALSE 2 B
#> 3  TRUE 3 C
#> 4  TRUE 4 D
x[, 2]
#> [1] 1 2 3 4
x[, "b", drop = FALSE]
#>   b
#> 1 1
#> 2 2
#> 3 3
#> 4 4
```

4. Opérateurs binaires

4.1. Opérateurs arithmétiques

Les opérateurs arithmétiques (tab. 4.1) permettent de réaliser les opérations élémentaires :

```
x <- 1
y <- 2

# Addition
x + y
#> [1] 3

# Soustraction
x - y
#> [1] -1

# Multiplication
x * y
#> [1] 2

# Exponentiation
x^y
#> [1] 1
```

Tab. 4.1. : Les principaux opérateurs arithmétiques de R.

Opérateur	Description
`+`	addition
`-`	soustraction
`*`	multiplication
`/`	division
`^`	exponentiation

Une des particularités des R est de *vectoriser* de nombreuses opérations. Cela signifie que les opérations mathématiques sont réalisées élément par élément :

```
x <- 1:5
y <- 6:10

# Addition des éléments de x et y, un à un
# Équivaut à réaliser 1 + 6, 2 + 7, 3 + 8, 4 + 9 et 5 + 10
x + y
#> [1]  7  9 11 13 15
```

Cet aspect est important dans le cas d'opérations sur des matrices (souvenez-vous, une matrice est simplement un vecteur avec un attribut de dimension et est remplie colonne par colonne) :

```
# Une matrice de 5 lignes et 2 colonnes
x <- matrix(1:10, ncol = 2)
x
#>      [,1] [,2]
#> [1,]    1    6
#> [2,]    2    7
#> [3,]    3    8
#> [4,]    4    9
#> [5,]    5   10

# Un vecteur de 10 éléments
y <- 1:10
y
#>  [1]  1  2  3  4  5  6  7  8  9 10

# Les éléments de x et y sont additionnés un à un
x + y
#>      [,1] [,2]
#> [1,]    2   12
#> [2,]    4   14
#> [3,]    6   16
#> [4,]    8   18
#> [5,]   10   20
```

Si deux vecteurs n'ont pas la même longueur, R va **recycler** le plus court des deux et le réutiliser autant que nécessaire. Si la taille de l'objet le plus long n'est pas multiple de la taille de l'objet le plus court, un avertissement sera également retourné :

```
# Un vecteur de 5 éléments
i <- 1:5
i
```

```
#> [1] 1 2 3 4 5

# Les éléments de x et i sont additionnés un à un
# i est recyclé silencieusement
x + i
#>      [,1] [,2]
#> [1,]    2    7
#> [2,]    4    9
#> [3,]    6   11
#> [4,]    8   13
#> [5,]   10   15

# Un vecteur de 3 éléments
j <- 1:3
j
#> [1] 1 2 3

# Les éléments de x et j sont additionnés un à un
# j est recyclé avec un message d'avertissement
x + j
#> Warning in x + j: la taille d'un objet plus long n'est pas multiple
#> de la taille d'un objet plus court
#>      [,1] [,2]
#> [1,]    2    9
#> [2,]    4    8
#> [3,]    6   10
#> [4,]    5   12
#> [5,]    7   11
```

4.2. Opérateurs logiques

Les opérateurs logiques (tab. 4.2) permettent comparer des valeurs. Ces comparaisons se présentent sous la forme d'un test (la condition est-elle vraie ou fausse ?) et retournent donc un vecteur de type `logical` :

```
# x contient un unique élément
x <- 1

# La valeur de x est-elle égale à 0 ?
x == 0
#> [1] FALSE
```

```
# La valeur de x est-elle différente de 1 ?
x != 1
#> [1] FALSE

# La valeur de x est-elle supérieure à 0 ?
x > 0
#> [1] TRUE

# La valeur de x est-elle inférieure à 2 ?
x < 2
#> [1] TRUE
```

Tab. 4.2. : Les opérateurs logiques de R.

Opérateur	Comparaison
`>`	strictement supérieur
`>=`	supérieur ou égal
`<`	strictement inférieur
`<=`	inférieur ou égal
`==`	égal
`!=`	différent
`&`	ET
`|`	OU

Les opérateurs `&` (ET) et `|` (OU) permettent de combiner plusieurs conditions et de réaliser des comparaisons plus complexes. Dans ce cas, chaque condition est évaluée séparément, puis les résultats sont comparés :

```
# La valeur de x est-elle comprise entre 0 et 2 ?
x > 0 & x < 2
#> [1] TRUE

# La valeur de x est-elle égale à 0 ou 1 ?
x == 0 | x == 1
#> [1] TRUE

# ET
TRUE & TRUE
#> [1] TRUE
TRUE & FALSE
#> [1] FALSE
FALSE & FALSE
```

```
#> [1] FALSE

# OU
TRUE | TRUE
#> [1] TRUE
TRUE | FALSE
#> [1] TRUE
FALSE | FALSE
#> [1] FALSE

# NEGATION
!TRUE
#> [1] FALSE
!FALSE
#> [1] TRUE
```

Souvenez-vous, R vectorise de nombreuses opérations. La comparaison de deux objets est réalisée élément par élément :

```
x <- c(1, 4, 3, 6, 5)
y <- c(3, 2, 5, 4, 7)

# Comparaison des éléments de x et y, un à un
# Équivaut à tester si 1 < 3, 2 < 4, 3 < 5, 4 < 6 et 5 < 7
x < y
#> [1]  TRUE FALSE  TRUE FALSE  TRUE

# Toutes les valeurs de x sont-elles inférieures à y ?
all(x < y)
#> [1] FALSE

# Au moins une des valeurs de x est-elle inférieure à y ?
any(x < y)
#> [1] TRUE
```

Avertissement

Prenez garde aux valeurs manquantes ! Si une comparaison est réalisée avec une valeur manquante, le résultat sera une valeur manquante uniquement s'il est ambigu :

```
TRUE & NA
#> [1] NA
FALSE & NA
#> [1] FALSE

TRUE | NA
#> [1] TRUE
FALSE | NA
#> [1] NA
```

Utilisés conjointement avec `[` et `<-`, les opérateurs logiques permettent de filtrer et nettoyer facilement ses données :

```
x <- 0:10
x
#>  [1]  0  1  2  3  4  5  6  7  8  9 10

# Extraire les valeurs supérieures à 5
x[x > 5]
#> [1]  6  7  8  9 10

# Remplacer les 0 par des valeurs manquantes
x[x == 0] <- NA
x
#>  [1] NA  1  2  3  4  5  6  7  8  9 10
```

5. Structures de contrôle

5.1. Alternatives

Les alternatives permettent d'exécuter différentes instructions en fonction du résultat d'un test logique.

5.1.1. Test "si..."

Fig. 5.1. : Diagramme d'un test "si..."

L'instruction `if` permet d'exécuter un bloc de code, uniquement *si* une condition est vraie (fig. 5.1) :

```
x <- 1

# On définit un test logique
# On encadre les instructions à exécuter avec des accolades
if (x > 0) {
  print("Texte affiché si la condition est vraie.")
}
#> [1] "Texte affiché si la condition est vraie."
```

5.1.2. Test "si... sinon..."

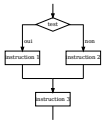


Fig. 5.2. : Diagramme d'un test "si... sinon..."

Il est possible de définir plusieurs choix en utilisant alternativement les instructions `if` et `else` (fig. 5.2) :

```
x <- 1

# On définit un test logique
if (x < 0) {
  print("Texte affiché si la condition est vraie.")
} else {
  print("Texte affiché si la condition est fausse.")
}
#> [1] "Texte affiché si la condition est fausse."
```

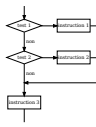


Fig. 5.3. : D
d'un test "si

Plusieurs alternatives peuvent être définies en utilisant conjointement `else` et `if`. On peut alors enchaîner autant de clauses "sinon si" que nécessaire et, éventuellement, définir une clause finale qui sera exécutée uniquement si aucune autre clause n'a été vérifiée (fig. 5.3) :

```
x <- "chien"

if (x == "chat") {
  # Si la première condition est vraie
  message("x est un chat.")
} else if (x == "chien") {
  # Si la première condition est fausse et la seconde est vraie
  message("x est un chien.")
} else {
  # Si toutes les conditions sont fausses
  message("x est un animal inconnu.")
}
#> x est un chien.
```

5.1.3. Test "selon..."

L'instruction `switch` permet de choisir un bloc de code *selon* la valeur d'une variable.

L'exemple précédent peut être réécrit comme suit :

```
x <- "chien"

switch (
  x,
  chat = message("x est un chat."),
  chien = message("x est un chien."),
  message("x est un animal inconnu.")
)
#> x est un chien.
```

5.2. Boucles

Les boucles sont un moyen simple pour répéter des instructions en faisant varier un paramètre.

5.2.1. Boucles itératives

Une boucle itérative est contrôlée un compteur : une variable qui va prendre une valeur différente à chaque itération de la boucle. Le nombre d'exécutions de la boucle est connu à l'avance.

L'instruction `for` permet d'écrire des boucles en définissant un compteur et en spécifiant les instructions à répéter :

```
# On définit une variable i (le nom est libre)
# On précise que i va prendre successivement les valeurs 1 à 5
# On encadre les instructions à répéter avec des accolades
for (i in 1:5) {
  print(i) # A chaque itération, on affiche la valeur de i
}
#> [1] 1
#> [1] 2
#> [1] 3
#> [1] 4
#> [1] 5
```

Dans l'exemple suivant, on cherche à extraire et à afficher successivement chaque élément d'un vecteur :

```
# Création d'un vecteur contenant 5 éléments
x <- c(1.2, 6.5, 6.1, 5.9, 6.4)

# Faire varier i de 1 à 5
for (i in 1:5) {
  # Extraire la i-ème valeur de x
  # Stocker cette valeur dans l'objet k
  k <- x[i]
  # Afficher la valeur de k
  print(k)
}
#> [1] 1.2
#> [1] 6.5
#> [1] 6.1
#> [1] 5.9
#> [1] 6.4
```

Dans l'exemple précédent, la valeur de `k` a été remplacée à chaque itération de la boucle. Si on veut récupérer les résultats des différentes étapes, il va falloir au préalable créer un objet vide susceptible de recevoir progressivement les différentes valeurs. La définition

de cet objet implique d'anticiper les résultats de la boucle pour qu'il ait la structure adéquate :

```
# Créer un vecteur vide pour récupérer les résultats
# Le vecteur à la même longueur que le nombre d'itérations
k <- vector(mode = "numeric", length = 5)
k
#> [1] 0 0 0 0 0

# Faire varier i de 1 à 5
for (i in 1:5) {
  # Extraire la i-ème valeur de x
  # Affecter cette valeur à la i-ème position de k
  k[i] <- x[i]
}
k
#> [1] 1.2 6.5 6.1 5.9 6.4
```

Dans le cas d'un tableau à deux dimensions, écrire une même série d'instructions pour chaque ligne ou colonne individuellement peut rapidement s'avérer fastidieux. On peut ainsi mettre à profit les boucles pour n'écrire qu'une seule fois ces instructions et les appliquer à toutes les lignes ou toutes les colonnes.

Imaginons que l'on souhaite connaitre le nombre total de tessons de différents types de céramiques retrouvés sur un ensemble de sites archéologiques. Si nos données sont correctement structurées, on peut calculer cette somme pour chacune des colonnes d'un tableau de données[1] :

```
# On charge les données d'exemple
# help(compiegne, package = "folio")
data(compiegne, package = "folio")

# Le tableau possède 5 lignes et 16 colonnes
dim(compiegne)
#> [1]  5 16

# On crée un vecteur vide pour récupérer les résultats
# (16 valeurs sont attendues : la somme de chaque colonne)
resultats <- vector(mode = "numeric", length = 16)

# On fait varier i de 1 à 16
```

[1] Il s'agit d'un exemple : dans le cas présent une boucle n'est pas nécessaire, R possède deux fonctions qui permettent de calculer la somme des lignes (`rowSums()`) ou des colonnes (`colSums()`) d'un tableau.

```
for (i in 1:16) {
  # On extrait la i-ème colonne
  colonne_i <- compiegne[, i]
  # On calcule la somme de la colonne
  total_i <- sum(colonne_i)
  # On stocke la somme dans la i-ème position du vecteur de résultats
  resultats[i] <- total_i
}
resultats
#>  [1] 53200 12380 35300  8115  4825  7360  2775 10700   990  5425
#> [11] 18910 11155  2410  2835  1965  1185
```

5.2.2. Boucles de parcours

R possède une famille de fonctions qui permettent d'appliquer facilement des instructions aux différents éléments d'un objet. Les principales fonctions de cette famille sont :

- `apply()` : applique une fonction sur les lignes ou les colonnes d'une matrice (ou sur les dimensions supérieures d'un `array`).
- `lapply()` et `vapply()` : appliquent une fonction sur les différents éléments d'un vecteur ou d'une liste.
- `tapply()` : applique une fonction sur différents groupes de valeurs.

L'exemple précédent peut ainsi être simplifié avec `apply()` :

```
# Inutile de créer un objet vide au préalable
somme <- apply(
  X = compiegne,
  MARGIN = 2, # Appliquer aux lignes (1) ou aux colonnes (2)
  FUN = sum
)
somme
#>     A     B     C     D     E     F     G     H     I     J     K
#> 53200 12380 35300  8115  4825  7360  2775 10700   990  5425 18910
#>     L     M     N     O     P
#> 11155  2410  2835  1965  1185
```

Avertissement

La fonction `apply()` est conçue pour être utilisée sur une matrice (ou un `array`). Lorsqu'elle est utilisée sur un `data.frame`, elle va donc le convertir en `matrix`. Les éléments d'une matrice sont tous du même type : si les colonnes du `data.frame` sont de types différents, les règles de conversions implicites s'appliquent et peuvent entraîner des erreurs (par exemple lorsque les colonnes numériques sont transformées en chaînes de caractères, empêchant ainsi l'exécution d'une fonction mathématique).

Dans le cas d'un `data.frame`, il est préférable d'utiliser `lapply()` lorsqu'on souhaite appliquer une même fonction aux différentes colonnes :

```
x <- data.frame(
  a = c(3.6, 7.5, 7.2, 7.7, 4.8),
  b = c(5.8, 7.5, 1.5, 5.7, 2.3),
  c = c(7.2, 6.4, 3.3, 3.1, 8.1)
)

# Convertir toutes les colonnes en integer
y <- lapply(X = x, FUN = as.integer)
y
#> $a
#> [1] 3 7 7 7 4
#>
#> $b
#> [1] 5 7 1 5 2
#>
#> $c
#> [1] 7 6 3 3 8
```

`lapply()` retourne une simple liste, mais en utilisant une sélection vide, il est possible de conserver la structure du `data.frame` initial :

```
# Convertir toutes les colonnes en integer
# Conserve la structure du data.frame
# (mais remplace les données d'origine)
x[] <- lapply(X = x, FUN = as.integer)
x
#>   a b c
#> 1 3 5 7
#> 2 7 7 6
#> 3 7 1 3
```

```
#> 4 7 5 3
#> 5 4 2 8
```

`vapply()` est similaire à `lapply()`, mais peut être plus pratique à utiliser. `vapply()` simplifie le résultat final et retourne un vecteur ou, si cela est approprié, une matrice (comme pour `apply()`, faites attention aux règles de conversions implicites). De plus, le comportement de `vapply()` est prédictible : il est nécessaire de spécifier avec l'argument `FUN.VALUE` le type de résultat retourné.

```
# On spécifie le résultat de l'application de FUN aux éléments de X
# Celui-ci doit être un vecteur de type integer et de longueur 5
z <- vapply(X = x, FUN = as.integer, FUN.VALUE = integer(5))
z
#>      a b c
#> [1,] 3 5 7
#> [2,] 7 7 6
#> [3,] 7 1 3
#> [4,] 7 5 3
#> [5,] 4 2 8
```

Si le résultat de l'application de la fonction `FUN` à chaque élément de `X` ne correspond pas à la spécification, l'exécution du code est interrompue et une erreur est retournée :

```
# La longueur du vecteur retourné par FUN ne correspond pas
vapply(X = x, FUN = as.integer, FUN.VALUE = integer(3))
#> Error in vapply(X = x, FUN = as.integer, FUN.VALUE = integer(3)): les valeurs doivent être d'une long
#>  mais FUN(X[[1]]) a une longueur 5

# Le type du vecteur retourné par FUN ne correspond pas
vapply(X = x, FUN = as.integer, FUN.VALUE = character(5))
#> Error in vapply(X = x, FUN = as.integer, FUN.VALUE = character(5)): les valeurs doivent être de type
#>  mais FUN(X[[1]]) est de type 'integer'
```

`tapply()` permet d'appliquer une même fonction à différents sous-ensembles d'un objet. Cette fonction est particulièrement pratique pour calculer des indicateurs statistiques par groupes :

```
# On dispose de 5 mesures d'un paramètre quelconque
valeurs <- c(6.8, 6.4, 5.8, 8.5, 8.8)
# Chaque mesure appartient à un groupe particulier (A ou B)
groupes <- c("A", "A", "B", "A", "B")
```

```
# On calcule la moyenne de chaque groupe
moy <- tapply(X = valeurs, INDEX = groupes, FUN = mean)
moy
#>        A        B
#> 7.233333 7.300000
```

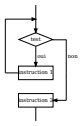


Fig. 5.4. : Diagramme d'une boucle à précondition.

5.2.3. Boucles à précondition

Une dernière possibilité, d'un usage peut-être moins courant, consiste à exécuter des instructions tant qu'une condition est vérifié. Cette boucle peut être considérée comme la répétition d'une instruction `if` *tant que* la valeur d'un test est vraie. Ainsi, une condition est vérifiée avant chaque itération de la boucle : si la condition est vraie (`TRUE`) les instructions sont exécutées, si la condition est fausse (`FALSE`) la boucle s'arrête sans exécuter les instructions (fig. 5.4).

```
# On définit la valeur initiale d'une variable
i <- 0

# On répète les instructions tant que i est inférieur à 5
while (i < 5) {
  print(i)   # Afficher la valeur de i
  i <- i + 1 # Incrémenter la valeur i
}
#> [1] 0
#> [1] 1
#> [1] 2
#> [1] 3
#> [1] 4
```

Remarque

Il convient d'être prudent avec l'usage de l'instruction `while`, car une mauvaise définition de la condition d'arrêt peut créer une boucle infinie.

6. Écrire des fonctions

Comme son nom le laisse deviner, fonction `function()` permet de créer ses propres fonctions.

```
addition <- function(x, y) {
  # On définit les instructions à exécuter
  z <- x + y
  # On précise le résultat à retourner au terme des calculs
  return(z)
}

addition(1, 2)
#> [1] 3
```

Il est possible de définir des arguments facultatifs et ainsi de spécifier le comportement par défaut d'une fonction.

La fonction suivante permet d'élever un nombre à la puissance désirée. En spécifiant la valeur du second argument (qui correspond à l'exposant) dans la définition de la fonction, on peut prédéfinir son comportement : par défaut, notre fonction va élever le nombre désiré au carré :

```
# Définition de la fonction
# Par défaut, la valeur de y est 2
puissance <- function(x, y = 2) {
  z <- x^y
  return(z)
}

# L'utilisateur ne change pas la valeur par défaut
puissance(2) # 2 au carré
#> [1] 4

# L'utilisateur change explicitement la valeur par défaut
puissance(2, 3) # 2 au cube
#> [1] 8
```

Deuxième partie

Collecter et organiser ses données

7. Introduction

L'objectif de ce chapitre est d'expliquer comment des décisions prises avant et pendant la collecte des données peuvent permettre d'économiser du temps et des efforts plus tard, pendant le processus d'analyse et de visualisation des données, en utilisant R ou tout autre outil. Le chapitre est organisé de manière hiérarchique, en commençant par l'organisation des fichiers au niveau du projet, puis en descendant pour discuter de l'organisation efficace des feuilles de calcul (l'un des outils les plus courants pour collecter, partager et stocker les données archéologiques), et enfin au niveau très granulaire de la cellule individuelle dans une feuille de calcul.

Le gain de temps et d'efforts est bien sûr une motivation importante pour modifier vos pratiques en matière de collecte et d'organisation des données, mais il existe un objectif plus large. Cet objectif est double :

- D'une part, cela permet participe à améliorer la reproductibilité de la recherche. Lorsqu'un projet est bien organisé, il est plus facile de trouver les fichiers clés pour vérifier les résultats obtenus.
- D'autre part, cela doit conduire à la production de jeux de données interopérables à grande échelle. De sorte que les données de divers projets puissent être facilement combinées pour répondre à des questions qu'aucun projet ne peut résoudre seul.

Cette double approche de l'organisation des données repose sur d'autres aspects, telles que la volonté de partager les données et de les rendre accessibles au public, ainsi qu'un consensus sur ce qu'il faut mesurer et comment nommer les variables. Si le partage de vos données est la première étape pour permettre leur réutilisation — ce qui n'est pas encore courant en archéologie — elles doivent également être faciles à comprendre pour que d'autres puissent facilement les réutiliser (White et al., 2013).

Une grande partie de ce chapitre ne présente pas de complexité particulière et peut sembler relever du bon sens. Cependant, les pratiques décrites ici font une grande différence quant à la réutilisation des données. Il nous semble donc important de les discuter directement et explicitement, plutôt que de supposer que tout un chacun les assimilera en cours de route sans conseils ou instructions spécifiques. Ce chapitre s'appuie sur des documents du domaine public, préparés à l'origine par Jenny Bryan, Karl Broman et d'autres pour le programme science reproductible de Data Carpentry, ainsi que sur des sources provenant d'autres disciplines, comme Borer et al. (2009), Sutter et al. (2015), Volk et al. (2014), Hampton et al. (2013), Strasser et Hampton (2012).

8. Gérer ses fichiers

Le principe fondamental à suivre pour améliorer l'organisation de ses dossiers et fichiers est simplement d'avoir une stratégie et de s'y tenir. L'une des principales motivations pour bien organiser ses fichiers est de faciliter leur utilisation, à la fois pendant la durée de l'analyse et à l'avenir, lorsque vous ou d'autres personnes les utiliserez. Si vos fichiers sont bien organisés, une personne qui n'est pas familière avec votre projet (ce qui pourrait aussi être vous dans le futur) devrait être capable de les regarder et de comprendre rapidement en détail ce que vous avez fait et pourquoi (Noble, 2009). Une deuxième motivation réside dans le caractère souvent itératif de l'analyse de données : tout ce que vous faites pendant votre analyse, vous aurez probablement à le refaire. Nous explorons généralement de nombreuses options analytiques avec nos données, de sorte que le processus d'analyse ressemble davantage à un arbre ramifié qu'à une séquence unique d'actions consécutives. Nous revenons souvent à une étape antérieure de l'analyse pour étudier l'effet de la sélection de données particulières ou de l'ajustement des paramètres d'un modèle. Si nous avons organisé et documenté clairement notre travail, il sera beaucoup plus facile de répéter les étapes précédentes de l'analyse avec des variations mineures.

8.1. Travailler par projets

Une bonne pratique liée à l'utilisation de R consiste ainsi à organiser l'ensemble de son analyse en un projet, c'est à dire un dossier autonome contenant l'ensemble des fichiers nécessaires à l'analyse (dont les données brutes et les fichiers générés en cours d'analyse). Une telle organisation doit permettre de déplacer le dossier sur son ordinateur ou sur un autre ordinateur, tout en maintenant le bon fonctionnement du code (absence de liens brisés, de dépendances manquantes…). Ce dossier, appelé répertoire de travail (*working directory*), doit être **portable**.

Le répertoire de travail est un concept important à comprendre, car il diffère de la façon dont on travaille avec un tableur. Le répertoire de travail est le répertoire (ou dossier) de votre ordinateur dans lequel R va travailler. Il s'agit de l'emplacement sur votre ordinateur où R cherchera lorsque vous lirez des fichiers ou enregistrerez des résultats. Par défaut, le répertoire de travail a probablement été défini dans un répertoire peu pratique lors de l'installation de R, mais il est facile de le modifier.

Le répertoire de travail peut être défini ou retrouvé à l'aide des fonctions `setwd()` et `getwd()`, respectivement :

```
# Définition du répertoire de travail
setwd("/chemin/vers/mon/dossier")
```

Une fois le répertoire de travail défini, toute référence à un fichier en utilisant un simple nom de fichier ou un chemin relatif sera interprétée relativement au répertoire de travail.

Cependant, cette approche n'est pas recommandée car le risque est de se retrouver dans la situation suivante :

```
# Définition du répertoire de travail
setwd("/chemin/uniquement/valide/sur/mon/ordinateur")
```

Si vous utilisez la commande `setwd()` pour définir le répertoire de travail, votre code ne fonctionnera que sur votre ordinateur, dans sa configuration actuelle de fichiers et de dossiers. Cela signifie que votre code n'est pas portable et n'est pas robuste face au changement. Si vous réorganisez vos fichiers ou si vous donnez votre code à un collaborateur, le répertoire de travail réel où se trouvent vos fichiers ne correspondra plus à ce qui est écrit dans le code. Cela peut entraîner des erreurs et de la frustration. Ainsi, bien qu'il soit important de connaître le concept de répertoire de travail pour travailler avec R, vous ne devez pas inclure sa définition dans votre code R.

Pour pallier ce problème qui rend difficilement déplaçable un dossier de projet, il existe différentes solutions (utilisable conjointement) parmi lesquelles :

- L'utilisation du package *here*,
- L'utilisation de la gestion de projet dans RStudio.

Conseil

RStudio dispose d'un mécanisme permettant de créer facilement de tels projets. Il est possible de créer un projet dans RStudio à partir du menu déroulant situé en haut à droite de l'interface ou en cliquant sur *New Project…* depuis le menu *File*. Le projet peut alors être créé soit dans un nouveau dossier, soit en transformant un dossier existant (fig. 8.1).
Un projet créé par RStudio est reconnaissable par la présence d'un fichier `.Rproj`. Ce fichier marque le dossier de plus haut niveau au sein d'un projet (répertoire de travail), à partir duquel des chemins d'accès relatifs peuvent être utilisés pour lire ou écrire des fichiers. Le répertoire de travail est automatiquement définit lors de l'ouverture d'un projet RStudio : il n'est plus nécessaire d'utiliser `setwd()`.

Il peut être nécessaire que votre code fasse référence à d'autres répertoires par rapport au répertoire principal de votre projet (par exemple, un répertoire appelé "data" qui contient des feuilles de calcul, etc.). L'organisation des fichiers et des sous-dossiers au

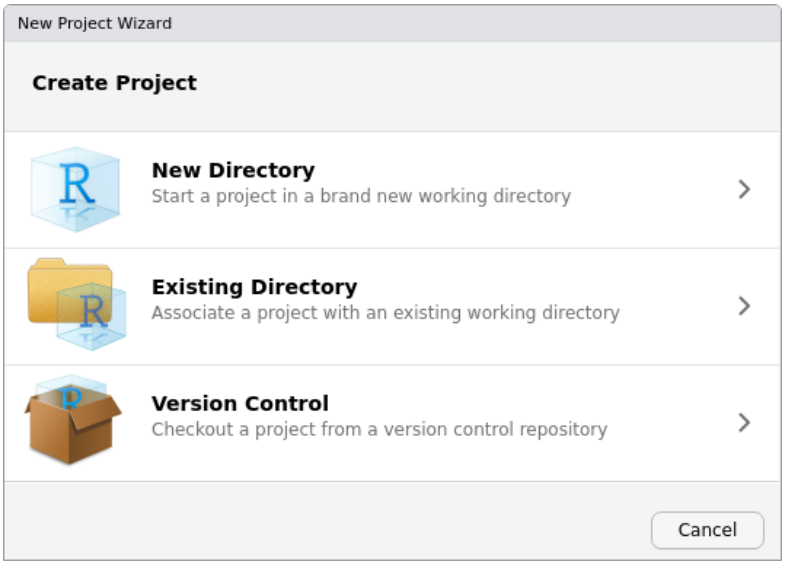


FIG. 8.1. : Création d'un projet avec RStudio.

sein d'un projet relève des habitudes de travail de chacun. S'il n'existe pas de consensus sur la manière d'organiser les fichiers à l'intérieur d'un projet, il peut cependant être avantageux de suivre certaines conventions (fig. 8.2).

8.2. Organiser ses fichiers

Au-delà d'avoir une stratégie pour organiser ses dossiers et fichiers, il existe plusieurs lignes directrices qui ont été proposées par des biologistes et des écologistes et qui sont applicables à la recherche archéologique (NOBLE, 2009 ; WHITE et al., 2013). Ces recommandations reposent sur un principe élémentaire qui veut que l'organisation des fichiers indique les entrées et les sorties, ainsi que le flux d'informations pendant l'analyse des données. Les recommandations les plus importantes sont que (1) les données brutes doivent être séparées des données dérivées, et (2) les données doivent être séparées du code.

Les **données brutes** désignent les fichiers de données tels qu'ils étaient lorsque vous les avez reçus à l'origine, par exemple transcrits à partir d'un cahier, saisis directement dans une base de données ou un tableur, ou téléchargés à partir d'un instrument tel qu'une station totale ou un analyseur XRF. Les **données dérivées** sont les données qui résultent des actions que vous entreprenez après avoir reçu les fichiers de données. Il s'agit de manipulations manuelles, telles que des modifications dans une feuille de calcul et de manipulations programmatiques qui se produisent lorsque vous utilisez R pour produire une version actualisée de vos données. Les manipulations manuelles sont à éviter autant que possible, car elles ne laissent aucune trace — contrairement à une manipulation

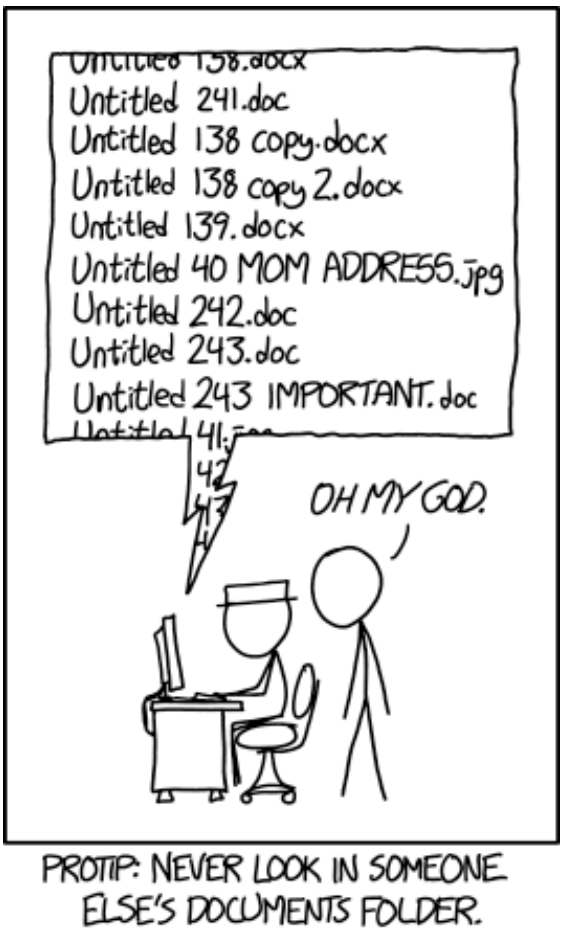


Fig. 8.2. : Documents. “Copy of Untitled.doc” (CC BY-NC 2.0 Munroe, 2014).

effectuée par un script — ce qui les rend difficiles à défaire si vous changez d'avis. Si une manipulation manuelle est nécessaire dans vos données, vous devriez conserver un fichier de texte brut avec le fichier de données et y consigner un bref récit des modifications apportées afin d'en avoir une trace. Une façon simple de séparer les données brutes des données dérivées consiste à avoir un dossier appelé `data/` qui contient deux répertoires, `raw_data/` et `derived_data/` (voir le schéma ci-dessous). L'utilisation d'une structure simple comme celle-ci signifie que vous n'avez besoin que d'une seule copie des fichiers de données originaux dans votre projet, plutôt que de les dupliquer à travers toute votre structure de fichiers (ce qui peut être une source de confusion importante pour savoir quel fichier est le bon).

```
├── data/              # Données brutes et dérivées
|   ├── raw_data/      # Données brutes collectées avant l'analyse
|   └── derived_data/  # Données générées au cours de l'analyse
```

Cette pratique consistant à conserver les données brutes isolées de tout le reste signifie que vous êtes toujours en mesure de revenir aux premières étapes de votre analyse (lecture des données brutes) au cas où vous auriez besoin d'explorer une autre voie ou de vérifier vos résultats. Ce point est essentiel pour garantir la reproductibilité de votre recherche et pour avoir l'esprit tranquille en sachant que vous pouvez vérifier vos résultats. Si vous apportez de nombreuses modifications à vos fichiers de données brutes au cours de votre analyse, vous ne pourrez peut-être jamais revenir au début de votre analyse, vous n'aurez pas la possibilité de vérifier vos résultats et votre travail sera impossible à reproduire. Le fait de conserver les données brutes intactes offre également aux autres chercheurs davantage d'options lorsqu'ils réutilisent vos données, ce qui augmente la valeur de votre travail pour l'ensemble de la communauté scientifique.

Le second principe d'organisation des fichiers dont nous pouvons nous inspirer des biologistes et des écologistes est étroitement lié au premier : les données sont séparées du code. Utiliser un tableur est une façon très peu naturelle de travailler. Dans une feuille de calcul, les formules qui calculent à partir les données sont situées directement à côté des cellules qui contiennent les données (par exemple, dans les lignes à droite ou sous les données). Bien que cela puisse être pratique pour travailler avec de petits tableaux de données, il est rapidement difficile de distinguer les données brutes des données dérivées et de distinguer les données et les méthodes (c'est-à-dire les fonctions dans les cellules de la feuille de calcul). Ce mélange des données et des méthodes est un obstacle majeur à la reproductibilité et à l'ouverture de la recherche, car la séquence des décisions prises au cours de l'analyse n'est pas explicite.

Votre code changera fréquemment pendant que vous travaillez, mais vos données brutes ne devraient être modifiées. Si vous conservez ces fichiers dans des dossiers séparés, vous serez moins tenté de modifier les données brutes à la main pendant que vous écrivez du code. Il s'agit d'un principe de base de la conception de logiciels, qui permet de minimiser la confusion lors de la navigation dans les fichiers d'un projet. Une raison

supplémentaire devant conduire à séparer le code et les données est que vous pouvez ne pas être en mesure de partager vos données brutes (par exemple, elles peuvent contenir des informations sensibles sur l'emplacement de sites archéologiques), mais vous pouvez être disposé à partager votre code. Lorsque ces deux composantes du projet sont bien séparées, il est facile de contrôler ce qui peut être rendu public et ce qui doit rester privé. Voici un exemple de structure de dossier de projet qui montre comment le code et les données peuvent être séparés :

```
├── data/              # Données brutes et dérivées
|   ├── raw_data/      # Données brutes collectées avant l'analyse
|   └── derived_data/  # Données générées au cours de l'analyse
|
├── scripts/           # Tout code, par exemple des fichiers de script R
```

Une troisième recommandation consiste à garder les fichiers associés à la production de manuscrits ou de rapports séparés de tout le reste. Dans l'exemple ci-dessous, cela est représenté par le répertoire `paper/`. Ce dossier contient un (ou plusieurs) fichiers R Markdown qui documentent l'analyse sous une forme exécutable. Cela signifie qu'il devrait être possible de convertir le document R Markdown en un fichier HTML/PDF/Word présentant les principales méthodes, décisions et résultats de l'analyse. Ce fichier R Markdown peut, par exemple, importer le code d'un script présent dans dans le répertoire `scripts/` en utilisant la fonction `source()`. Cet exemple montre également comment organiser les figures et autres images :

```
├── data/              # Données brutes et dérivées
|   ├── raw_data/      # Données brutes collectées avant l'analyse
|   └── derived_data/  # Données générées au cours de l'analyse
|
├── scripts/           # Tout code, par exemple des fichiers de script R
|
├── figures/           # Figures produites au cours de l'analyse
|
├── paper/             # Rédaction du manuscrit
|   ├── paper.Rmd      # Un ou plusieurs fichier R markdown
|   └── references.bib # Fichier bibliographique
```

Une dernière recommandation est d'avoir un répertoire `scratch/` (brouillon) dans lequel expérimenter. Ce répertoire doit contenir des bribes de code et des résultats produits lors de courtes excursions sur des voies alternatives lors de l'analyse (lorsque vous voulez explorer certaines idées, mais que vous n'êtes pas sûr qu'elles seront pertinentes pour l'analyse principale). Si l'expérimentation s'avère utile, vous pouvez alors l'incorporer à l'endroit approprié dans les dossiers du projet principal. Si l'expérience n'est pas directement pertinente, vous pouvez la laisser dans le répertoire `scratch/` ou la supprimer

(tout ce qui se trouve dans ce répertoire doit pouvoir être supprimé à tout moment sans aucune conséquence pour votre projet). En général, le répertoire `scratch/` est supprimé après la soumission ou la publication de l'article.

Enfin, une recommandation supplémentaire peut être formulée, bien qu'elle ne relève pas de la même logique. Elle consiste à inclure un fichier texte simple nommé "README" (par exemple "README.txt" ou "README.md") à la racine de l'arborescence des fichiers d'un projet. Ce fichier est destiné à documenter quelques détails de base sur le projet. Cela peut inclure une douzaine de lignes de texte fournissant le nom du projet, les noms des personnes impliquées, une brève description du projet et un bref résumé du contenu des dossiers du projet. Vous pouvez également inclure des fichiers README similaires dans d'autres dossiers de votre projet, par exemple pour documenter les paramètres des instruments qui sont importants pour l'utilisation des données brutes. Vous trouverez ci-dessous un exemple de projet comprenant un répertoire brouillon et un fichier README :

```
├── data/              # Données brutes et dérivées
|   ├── raw_data/      # Données brutes collectées avant l'analyse
|   └── derived_data/  # Données générées au cours de l'analyse
|
├── scripts/           # Tout code, par exemple des fichiers de script R
|
├── scratch/           # Expériementations (peut être supprimé)
|
├── figures/           # Figures produites au cours de l'analyse
|
├── paper/             # Rédaction du manuscrit
|   ├── paper.Rmd      # Un ou plusieurs fichier R markdown
|   └── references.bib # Fichier bibliographique
|
├── README.txt         # La description du projet et de son contenu
```

Ces recommandations sont largement applicables à la plupart des projets archéologiques, quelle que soit leur échelle ou leurs méthodes. Certains ajustements peuvent être nécessaires pour les jeux de données volumineux, distant ou contenant des informations sensibles (de même dans les situations où les calculs prennent beaucoup de temps). Dans tous les cas, le principe général de séparation doit gouverner les décisions prises en matière d'organisation des fichiers.

Conseil

Le package *rrtools* permet de structurer rapidement son travail en suivant certaines conventions d'organisation des fichiers.

8.3. Nommer ses fichiers

En tant qu'archéologues, nous sommes habitués à nommer les choses, lorsque nous faisons face à des objets qui ne correspondent pas exactement aux typologies existantes ou lorsque nous identifions des fragments d'objets qui n'ont pas d'attributs distinctifs (Ferris, 1999 ; Kleindienst, 2006). Nous faisons souvent attention lorsque nous nommons des objets, car les noms sont une forme de communication et privilégient certaines interprétations et significations par rapport à d'autres. Souvent, en tant qu'archéologues, lorsque nous attribuons un nom, nous donnons naissance à quelque chose et délimitons ses frontières. Nous discutons de la signification des noms, tels que "Paléolithique moyen" ou "Acheuléen".

Les nombreux produits numériques de notre travail analytique sont rarement rendus publics, si bien que nous ne mettons pas autant de soin et d'efforts à les nommer. De nombreux archéologues considèrent que les fichiers créés dans le cadre de l'analyse des données d'un projet sont leur propriété privée et peuvent être utilisés en l'état. Ils ne pensent donc pas à les nommer de manière à gagner du temps et à faciliter la recherche de ces fichiers. Ceci est peut-être moins vrai dans les grands projets, mais par expérience, en travaillant avec une douzaine de collaborateurs ou plus, le nommage des fichiers est souvent chaotique. Voici un exemple de noms de fichiers mal choisis :

```
resumé.docx
figure 1.png
Fig 2.png
article FINAL v.3 Jan22 introcorrigée.pdf
```

Pour remédier à ce chaos et rendre les fichiers plus faciles à (ré)utiliser, nous pouvons suivre un principe élémentaire pour nommer les fichiers : les noms de fichiers doivent lisibles à la fois par les machines et par les humains. Rendre les noms de fichiers lisibles par les humains signifie inclure des informations sur le contenu du fichier dans son nom. Lorsqu'un nom de fichier contient des informations sur son contenu, il est facile pour les autres utilisateurs de comprendre ce qu'est le fichier et de décider comment l'utiliser. Voici les fichiers précédents renommés en suivant cette directive :

```
2022-01-17_resume_eaa.docx
fig01_dispersion-lithique-silex.png
fig02_histogramme-lithique-silex-eclats.png
```

Une attention particulière doit ainsi être portée à trois éléments. Le premier point consiste à ne pas utiliser d'espaces, de signes de ponctuation (à quelques exceptions près) ou de caractères exotiques tels que les accents. Ceci en veillant à un usage cohérent de la casse des caractères, car pour une machine les majuscules et les minuscules d'une même lettre sont presque aussi différentes que A et Z le sont pour nous.

Le second point consiste à nommer les fichiers de sorte qu'il est possible d'en extraire de l'information. De tels noms de fichiers utilisent délibérément des délimiteurs tels que le trait d'union (-) et le trait de soulignement (_). Dans les exemples ci-dessus, les traits d'union sont utilisés pour délimiter les chiffres et les mots dans le nom des fichiers. Le trait de soulignement est, quant à lui, utilisé pour délimiter les unités de métadonnées, par exemple pour séparer la date de la description, ou le numéro de figure de sa description. Cette utilisation des délimiteurs permet d'extraire des métadonnées des noms de fichiers. Par exemple, on peut obtenir des données pertinentes pour notre analyse à partir des noms de fichiers suivants :

```
# Lister les fichiers
(fichiers <- list.files("data/raw_data/"))
#> [1] "2017-03-03_Zone-3_Unite-1_lithique.csv"
#> [2] "2017-03-03_Zone-3_Unite-2_lithique.csv"
#> [3] "2017-03-03_Zone-3_Unite-3_lithique.csv"
#> [4] "2017-03-03_Zone-3_Unite-4_lithique.csv"
#> [5] "2017-03-03_Zone-4_Unite-1_lithique.csv"
#> [6] "2017-03-03_Zone-4_Unite-2_lithique.csv"

# Découper les noms de fichiers
# (récupère les informations contenues dans les noms)
info <- strsplit(fichiers, split = "_")

# Construire un tableau de métadonnées
as.data.frame(do.call(rbind, info))
#>           V1     V2      V3           V4
#> 1 2017-03-03 Zone-3 Unite-1 lithique.csv
#> 2 2017-03-03 Zone-3 Unite-2 lithique.csv
#> 3 2017-03-03 Zone-3 Unite-3 lithique.csv
#> 4 2017-03-03 Zone-3 Unite-4 lithique.csv
#> 5 2017-03-03 Zone-4 Unite-1 lithique.csv
#> 6 2017-03-03 Zone-4 Unite-2 lithique.csv
```

L'utilisation judicieuse des délimiteurs peut alors rendre le travail avec un grand nombre de fichiers très aisé. En effet, il alors possible utiliser R pour rechercher des fichiers ou des groupes de fichiers spécifiques et extraire les informations contenues dans les noms de fichiers, comme illustré ci-dessus.

Le troisième point consiste à s'assurer que les noms de fichiers vont être automatiquement triés de manière pertinente. La façon la plus simple de le faire est d'ajouter un préfixe numérique, comme une date ou une autre séquence de chiffres.

Conseil

Pour classer des fichiers par date vous devez utiliser la norme ISO 8601 (`AAAA-MM-JJ`), afin d'obtenir un tri par année, mois puis jour.
Par défaut, la plupart des systèmes d'exploitation considèrent les chiffres dans les noms de fichiers comme des caractères. Ainsi, des fichiers préfixés par `1_`, `2_`, `3_`… seront triés de manière non naturelle, car `10_` sera placé entre `1_` et `2_`, plutôt qu'après `9_`, comme vous vous y attendriez. Pour obtenir un tri naturel, vous devez donc ajouter des zéro non significatifs à gauche, par exemple : `001_`, `002_`, `003_`, etc. (les valeurs 10 à 19 seront ainsi reconnues par votre ordinateur comme supérieures à 2). Ce principe est également valable lorsque les noms de fichiers sont préfixés par des caractères, par exemple : `Fig-01_`, `Fig-02_`, `Fig-03_`, etc.

9. Organiser ses données

9.1. Variables et observations

Les feuilles de calcul sont omniprésentes en archéologie, comme dans de nombreux autres domaines. Pour de nombreux chercheurs, la feuille de calcul est le pivot de leurs activités d'analyse de données. Ils y collectent des données, analysent et visualisent les résultats et ils copient et collent les données de la feuille de calcul vers leurs rapports et manuscrits pour la publication. Les tableurs sont utiles pour collecter des données, mais ils sont inefficaces pour l'analyse (lorsque vous souhaitez modifier une variable ou effectuer une analyse avec un nouveau jeu de données, vous devez généralement tout refaire à la main), et ils rendent votre travail analytique difficile à suivre. En effet, les tableurs fonctionnent principalement par des opérations de glisser-déposer effectuées à l'aide de la souris et n'imposent donc pas un ordre linéaire facile à suivre.

En bref, l'utilisation de feuilles de calcul pour l'analyse des données est néfaste pour la reproductibilité et la transparence de votre recherche. Dans cette section, nous allons passer en revue quelques règles d'organisation des données dans les feuilles de calcul afin de faciliter leur exploitation. Ces règles sont valables pour tout langage de programmation, que vous utilisiez R, Python ou Julia.

Lors de l'acquisition puis lors de la préparation de vos données en vue d'une étude, toutes vos actions doivent être guidées par un impératif : structurer au mieux vos jeux de données pour en faciliter l'analyse (Wickham, 2014). Les principes régissant cette étape de structuration (*data tidying*) sont exposés par Wickham (2014), qui propose ainsi une approche pour lier "la structure d'un jeu de données (sa mise en forme) avec sa dimension sémantique (sa signification)."

Des jeux de données bien structurés se présentent sous la forme de tableaux à deux dimensions (lignes et colonnes) contenant des valeurs. Chaque valeur correspond à une observation d'une variable particulière, en conséquence :

- Chaque variable doit correspondre à une colonne du tableau.
- Chaque observation doit correspondre à une ligne du tableau.
- Un tableau doit correspondre à une unique unité d'observation[1].

[1] On reconnait ici la troisième forme normale, dans le cas des bases de données relationnelles (Wickham, 2014).

Conseil

Dans le cas de données tabulaires, utilisez le format CSV.

Les règles sont les suivantes : premièrement, placez toutes vos variables dans leurs propres colonnes (une variable est la caractéristique mesurée, comme la masse ou la longueur) et, deuxièmement, placez chaque observation dans sa propre ligne. La violation la plus courante de ces règles consiste à ne pas placer chaque variable dans sa propre colonne, de sorte qu'une même colonne représente plus d'une variable.

Par exemple, le tableau ci-dessous est partiellement structuré. Chaque observation (c'est-à-dire chaque site archéologique) correspond bien à une ligne et le tableau dans son ensemble décrit une unique unité d'observation. Cependant, c'est un exemple typique de violation de la première règle.

secteur	materiau	total
A	cuivre	5
A	ceramique cuivre	9
A	ceramique os	30
B	ceramique os	12
B	ceramique	40
B	ceramique os cuivre	32

La colonne *materiau*, contient le type de matériau observé sur chaque site. Sous cette forme, il est difficile de regrouper les sites par caractéristiques, par exemple pour n'extraire que les sites contenant des ossements. Une approche plus structurée serait d'avoir une colonne pour la présence de céramique, une colonne pour la la présence d'alliages cuivreux, et ainsi de suite. Chacune de ces colonnes peut alors être exploitée individuellement, et nous avons un contrôle plus fin sur les données.

secteur	ceramique	cuivre	os	total
A	0	1	0	5
A	1	1	0	9
A	1	0	1	30
B	1	0	1	12
B	1	0	0	40
B	1	1	1	32

Il est souvent tentant de combiner plusieurs informations dans une seule cellule, car cela permet d'accélérer la collecte des données en réduisant le temps passé à naviguer entre les champs. S'il vous est impossible d'éviter de combiner des informations dans une seule cellule, ou si les données ont été collectées par quelqu'un d'autre et qu'elles présentent

déjà ce problème, il existe dans R des méthodes de nettoyage et de séparation que nous explorerons au chapitre 12. L'essentiel ici est de nettoyer et d'ordonner les données avant de les analyser et avant de les archiver pour que d'autres puissent les utiliser.

La deuxième caractéristique des données ordonnées (chaque ligne du tableau correspond à une observation), est facile à réaliser lorsque l'on enregistre les données d'un ensemble de spécimens, tels que des artefacts ou des restes de faune. Dans cette situation, il est naturel qu'un artefact soit représenté par une ligne du tableau. Il est moins naturel de suivre cette règle lorsque, par exemple, chaque ligne représente un site, et que certaines des colonnes sont des comptages de types de poteries. Dans ce cas, la forme propre et ordonnée serait une ligne pour chaque combinaison site/type de poterie, mais cette forme est très peu pratique pour la collecte de données. Ainsi, pendant l'acquisition des données, nous pouvons tolérer un peu de désordre afin d'optimiser la vitesse et la commodité de la saisie des données. Dans le chapitre 12, nous explorerons les méthodes permettant de rendre les données collectées sur le terrain vraiment ordonnées et adaptées à leur exploitation.

En revanche, la troisième caractéristique des données ordonnées (un tableau doit correspondre à une unique unité) est simple à mettre en œuvre. Au sein d'une feuille de calcul, chaque feuille ne doit contiennir qu'un seul tableau. Chaque feuille doit comporter un seul rectangle contigu de données. Il faut donc résister à la tentation d'organiser une grille de petits tableaux sur une seule feuille (fig. 9.1). Bien qu'il s'agisse d'une stratégie courante pour organiser les données, car elle permet de jeter un coup d'œil sur les petits tableaux pour les comparer, elle pose un sérieux problème : chaque ligne de cette feuille contient plus d'une observation, une ligne couvrant plusieurs petits tableaux. De même, les noms des colonnes sont susceptibles d'être dupliqués car ils apparaissent dans chacun des petits tableaux. Ces complications rendent plus difficile le nettoyage et l'organisation de vos données sous une forme utile pour l'analyse et la visualisation. La règle est de créer un tableau par feuille, et si vous suivez cette règle, l'organisation finale de vos données pour l'analyse sera beaucoup plus simple et rapide. En apportant de petits changements à la façon dont vous organisez vos données dans les feuilles de calcul, vous pouvez avoir un impact considérable sur l'efficacité et la fiabilité des étapes d'analyse.

Au niveau le plus granulaire de la collecte de données, on trouve la cellule individuelle d'une feuille de calcul (ou le champ de saisie des données). De mauvaises habitudes dans la saisie des données peuvent nécessiter un temps considérable pour les corriger avant que ces dernières ne soient adaptées à l'analyse, et peuvent conduire à des erreurs, quel que soit le logiciel que vous utilisez. Nous passons ici brièvement en revue certaines des erreurs courantes d'organisation des cellules dans l'espoir que si vous en êtes conscient, vous serez peut-être motivé pour les éviter dans votre collecte de données.

Le choix d'une valeur nulle adaptée nécessite une réflexion approfondie. Lorsque vous savez que la valeur est égale à zéro, vous devez bien sûr entrer zéro. Cependant, lorsque la valeur n'a pas été mesurée, ou n'a pas pu être mesurée, vous ne voulez pas entrer zéro car cela implique que vous savez quelque chose sur la valeur. Vous ne voulez pas non plus laisser ce champ vide, car cela pourrait être interprété comme une omission

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
1	Ceramic data											lithic data												
2	Trench	Square	EU	Context	Unit	Counts	Mass	Recorder	Date			Trench	Square	EU	Unit	Lithic_Type	Counts	Mass	Recorde	Date	Context			
3	A	SW1	1	1	1	14	27.2	CC	28-Jul-08			A	SW1	1	1	Flake	1	0.3	CC	28-Jul-08	1			
4	A	SW1	1	1	1	1	5.8	CC	28-Jul-08			A	SW2	2	2	Flake	1	18.4	HVV	28-Jul-08	1			
5	A	SW1	1	1	1	1	0.5	CC	28-Jul-08			A	SW5	5	9	Flake	20	73.6	CC	28-Jul-08	4			
6	A	SW1	1	1	1	1	0	CC	28-Jul-08			A	SW6	6	12	Flake	3	6.1	HVV	28-Jul-08	5			
7	A	SW1	1	1	1	1	0.6	CC	28-Jul-08			A	SW6	6	13	Flake	8	13.3	CC	28-Jul-08	5			
8	A	SW1	1	1	1	1	0.6	CC	28-Jul-08			A	SW9	9	16	Flake	2	2.7	HVV	28-Jul-08	6			
9	A	SW1	1	1	1	1	0.5	CC	28-Jul-08			A	SW11	11	17	Flake	2	6.3	HVV	28-Jul-08	7U			
10	A	SW1	1	1	1	1	1	CC	28-Jul-08			A	SW14	14	19	Flake	1	0.9	HVV	28-Jul-08	8			
11	A	SW1	1	1	1	1	0.4	CC	28-Jul-08			A	SW15	15	19	Flake	2	2.6	HVV	28-Jul-08	8			
12	A	SW2	2	1	2	1	5.1	HVV	28-Jul-08			A	SW/SE17	18	24	Flake	4	41	CC	28-Jul-08	7L			
13	A	SW2	2	1	2	51	72.1	HVV	28-Jul-08			A	SW19	19	27	Flake	2	9.7	HVV	28-Jul-08	7L			
14	A	SW2	2	1	2	1	3.5	HVV	28-Jul-08			A	SW20	20	27	Flake	2	7.1	HVV	28-Jul-08	7L			
15	A	SW2	2	1	2	1	6.9	HVV	28-Jul-08															
16	A	SW2	2	1	2	1	3.4	HVV	28-Jul-08															
17	A	SW2	2	1	2	1	1.5	HVV	28-Jul-08									Mollusc data						
18	A	SW2	2	1	2	1	1.7	HVV	28-Jul-08			Trench	Unit	Context	Taxon	Element	Mass (g)	Count	Max L/W	Burnt/Not B	Cutmark/	Recorder	NOTES	Date
19	A	SW2	2	1	2	1	6.3	HVV	28-Jul-08			A	SW2	2	Unknown	Bivalve	0.9	1	NA	N	N	HVV		9-Jul-12
20	A	SW2	2	1	2	1	3.5	HVV	28-Jul-08			Trench	Unit	Context	Taxon	Element	Mass (g)	Count	Max L/W	Burnt/Not B	Cutmark/	Recorder	NOTES	Date
21	A	SW2	2	1	2	1	1.4	HVV	28-Jul-08			A	SW4	5	F. Vivipar	Gastropod	1.3	1	NA	N	N	HVV		9-Jul-12
22	A	SW2	2	1	2	1	3	HVV	28-Jul-08			A	SW4	5	F. Arcida	Bivalve	0.8	1	NA	N	N	HVV		9-Jul-12
23	A	SW2	2	1	2	1	2.3	HVV	28-Jul-08			A	SW4	5	F. Muricic	Gastropod	1.5	1	NA	N	N	HVV		9-Jul-12
24	A	SW2	2	1	2	1	4.6	HVV	28-Jul-08			A	SW4	5	F. Potam	Gastropod	16.7	8	NA	N	N	HVV		9-Jul-12
25	A	SW2	2	1	2	1	3.1	HVV	28-Jul-08			A	SW4	5	Unknown	Gastropod	3.2	8	NA	N	N	HVV		9-Jul-12
26	A	SW2	2	1	2	1	1.8	HVV	28-Jul-08			A	SW4	5	Unknown	Bivalve	1.4	12	NA	N	N	HVV		9-Jul-12
27	A	SW2	2	1	2	1	1.2	HVV	28-Jul-08			A	SW4	5	Unknown	Bivalve	3.7	4	NA	N	N	HVV		9-Jul-12
28	A	SW2	2	1	2	1	3.2	HVV	28-Jul-08			A	SW4	5	F. Cyclop	Gastropod	1	2	NA	N	N	HVV		9-Jul-12
29	A	SW2	2	1	2	1	1.1	HVV	28-Jul-08			Trench	Unit	Context	Taxon	Element	Mass (g)	Count	Max L/W	Burnt/Not B	Cutmark/	Recorder	NOTES	Date
30	A	SW2	2	1	2	1	1.3	HVV	28-Jul-08			A	SW5	9	F. Cyclop	Gastropod	1.3	1	21.16	N	N	CC	F-100-01	10-Jul-12
31	A	SW2	2	1	2	1	0.8	HVV	28-Jul-08			A	SW5	9	F. Neritid	Gastropod	1.7	1	25.06	N	N	CC	F-100-01	10-Jul-12
32	A	SW2	2	1	2	1	1.1	HVV	28-Jul-08			A	SW5	9	F. Cyclop	Gastropod	2.7	1	28.24	N	N	CC	F-100-01	10-Jul-12
33	A	SW2	2	1	2	1	1	HVV	28-Jul-08			A	SW5	9	F. Cyclop	Gastropod	2.5	1	27.34	N	N	CC	F-100-01	10-Jul-12
34	A	SW2	2	1	2	1	1.5	HVV	28-Jul-08			A	SW5	9	F. Vivipar	Gastropod	4.5	2	NA	N	N	CC		10-Jul-12
35	A	SW2	2	1	2	1	1	HVV	28-Jul-08			A	SW5	9	F. Camas	Gastropod	3.2	1	NA	N	N	CC		10-Jul-12
36	A	SW2	2	1	2	1	0.9	HVV	28-Jul-08			A	SW5	9	F. Cyclop	Gastropod	2.5	3	NA	N	N	CC		10-Jul-12

Fig. 9.1. : Cette capture d'écran montre des données archéologiques désordonnées : plusieurs tableaux sur une feuille, plusieurs valeurs dans une colonne, plusieurs lignes d'en-tête pour un tableau, etc.

de l'enregistrement de cette donnée. Les cellules vides sont ambiguës : le collecteur de données a-t-il omis ce champ par accident, ou n'a-t-il pas été en mesure d'enregistrer une valeur pour ce champ, ou encore la valeur était-elle nulle ? Le meilleur choix dans cette situation est d'utiliser `NA` (*Not Available*) ou `NULL` (White et al., 2013). `NA` est un bon choix, car il s'agit d'un mot spécial (ou réservé) dans le langage R (R peut donc gérer les `NA` facilement). `NULL` peut être préférable si vos données utilisent `NA` comme abréviation, par exemple "Nouvelle-Aquitaine" (`NULL` est également un mot spécial dans R).

Un autre point problématique (néanmoins courant) lors du passage de la collecte à l'analyse des données concerne l'utilisation de formatage dans les feuilles de calcul pour transmettre des informations. Par exemple, lorsqu'une cellule comporte un texte en couleur ou est surlignée pour indiquer une information supplémentaire sur ce spécimen ou cet enregistrement (fig. 9.2). Des problèmes similaires se produisent lorsque du texte en gras ou en italique est utilisé pour signaler une qualité importante des données. Ce type d'utilisation est bien connu dans les tableaux publiés (par exemple l'utilisation du gras pour indiquer des valeurs statistiquement significatives). Cependant, cette décoration des cellules et du texte pour transmettre de l'information rend très difficile le calcul sur ces valeurs. Il est toujours préférable d'avoir une autre colonne pour capturer cette information, plutôt que de mettre en évidence la cellule ou de formater le texte.

Parmi la quantité d'éléments perturbateurs présents dans les feuilles de calculs on trouve également : les cellules fusionnées, les commentaires de cellule et l'inclusion d'unités de mesure dans une cellule, comme le symbole du pourcentage. Les cellules fusionnées peuvent rendre difficile le tri et le filtrage des données, ainsi que leur transfert dans d'autres programmes pour analyse.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	Ceramic data														
2	Trench	Square	EU	Context	Unit	Counts	Mass	Recorder	Date			Trench	Square	EU	Unit
3	A	SW1	1	1	1	14	27.2	CC	28-Jul-08			A	SW1	1	1
4	A	SW1	1	1	1	1	5.8	CC	28-Jul-08			A	SW2	2	2
5	A	SW1	1	1	1	1	0.5	CC	28-Jul-08			A	SW5	5	9
6	A	SW1	1	1	1	1	0	CC	28-Jul-08			A	SW6	6	12
7	A	SW1	1	1	1	1	0.6	CC	28-Jul-08			A	SW6	6	13
8	A	SW1	1	1	1	1	0.6	CC	28-Jul-08			A	SW9	9	16
9	A	SW1	1	1	1	1	0.5	CC	28-Jul-08			A	SW11	11	17
10	A	SW1	1	1	1	1	1	CC	28-Jul-08			A	SW14	14	19
11	A	SW1	1	1	1	1	0.4	CC	28-Jul-08			A	SW15	15	19
12	A	SW2	2	1	2	1	5.1	HVV	28-Jul-08			A	SW/SE17	18	24
13	A	SW2	2	1	2	51	72.1	HVV	28-Jul-08			A	SW19	19	27
14	A	SW2	2	1	2	1	3.5	HVV	28-Jul-08			A	SW20	20	27
15	A	SW2	2	1	2	1	6.9	HVV	28-Jul-08						
16	A	SW2	2	1	2	1	3.4	HVV	28-Jul-08				= contains complete vessels		

FIG. 9.2. : Cette capture d'écran montre le formatage d'une feuille de calcul. Dans ce cas, le surlignage jaune indique que des récipients complets ont été trouvés dans ce niveau. Une meilleure approche serait d'avoir une colonne appelée 'pots_complets', avec des valeurs indiquant le nombre de récipients complets dans chaque unité.

Enfin, l'endroit approprié pour les commentaires est une colonne nommée "commentaires" et les unités doivent être exprimées dans l'en-tête de colonne, la même unité de mesure devant être utilisée pour chaque ligne d'une colonne. Lorsque vous ajoutez des en-têtes à vos colonnes, veillez à ne pas inclure d'espaces, de chiffres ou de caractères spéciaux. Une bonne alternative aux espaces dans les en-têtes de colonne sont les traits de soulignement ("_"), par exemple,"longueur_mm" est préférable à "Longueur (mm)", pour une colonne de valeurs de longueur exprimées en millimètres. Les noms de champ courts ou abrégés sont souvent souhaitables pendant la collecte des données pour économiser de l'espace à l'écran, mais ils peuvent devenir déroutants plus tard lorsque vous réexaminez les données pour les analyser. Il faut donc veiller à ce que les en-têtes ne soient pas trop ambigus et envisager de conserver la liste des codes utilisés ou un dictionnaire de données qui associe chaque en-tête de colonne à une description de la variable et à la manière dont elle a été mesurée.

L'idée générale est de faire preuve de retenue lorsque vous travaillez avec des feuilles de calcul et de résister à la tentation d'utiliser toutes ses fonctionnalités. Soyez attentif à l'utilisation en aval de vos données, par vous dans R ou par d'autres dans d'autres environnements logiciels. Nous ne pouvons pas optimiser notre collecte de données pour toutes les situations possibles, mais avec quelques règles, nous pouvons en améliorer l'organisation pour en faciliter l'analyse avec d'autres outils.

Attention

De manière générale, méfiez-vous de votre tableur ! Ces derniers ont tendance à réaliser automatiquement des conversions qui ne sont pas sans conséquences lors de l'ouverture d'un fichier.

9.2. Enregistrer ses données

Une fois vos données correctement structurées, choisissez un format de fichier adapté pour les archiver, les diffuser ou les réutiliser (fig. 9.3). De manière générale, conservez vos données dans un fichier texte.

Dans le cas des données tabulaires, utilisez le format CSV. Comme son nom l'indique, un fichier CSV est un fichier texte contenant des valeurs séparées par une virgule. Cette simplicité lui confère plusieurs avantages, ce qui en fait le format idéal pour pérenniser des données :

- Un fichier CSV est une suite de caractères et de retours à la ligne : il est facilement éditable et peut aussi bien être lu par un humain (les données ne sont pas encodées) que par une machine.
- Le format CSV est un format ouvert : son utilisation ne dépend pas d'un éditeur particulier. Un fichier CSV peut ainsi être lu ou écrit par n'importe quel logiciel capable de manipuler des feuilles de calcul (ou par un simple éditeur de texte).
- Le format CSV est un format bien établi (ses origines remontent aux années 1970), il ne risque pas de subir un changement dramatique de spécification.
- Son apparente austérité (il ne contient pas de formatage) oblige a structurer correctement ses données.

La simplicité et la souplesse de ce format nécessitent cependant un peu de prudence au moment de la création d'un fichier CSV. Il est en effet possible d'utiliser n'importe quel caractère en guise de séparateur à la place d'une virgule. Si votre tableur est paramétré en français, par défaut, les données seront séparées un point-virgule pour éviter les confusions avec le séparateur décimal (de même, lors de l'ouverture d'un fichier CSV votre éditeur s'attendra à trouver des valeurs séparées par un point-virgule). Pensez à bien paramétrer votre logiciel lors de l'import ou de l'export d'un fichier CSV dans un tableur (fig. 9.4).

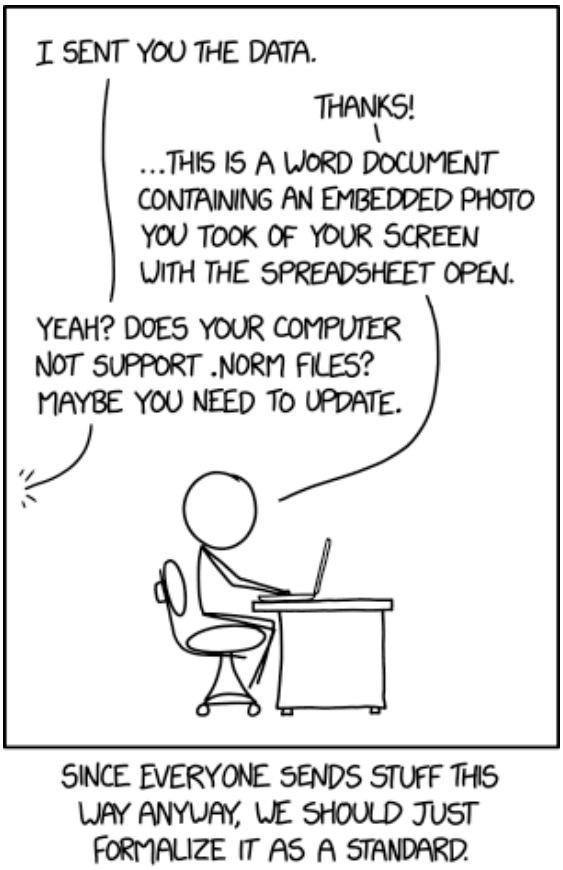


Fig. 9.3. : .NORM Normal File Format. “At some point, compression becomes an aesthetic design choice. Luckily, SVG is a really flexible format, so there’s no reason it can’t support vector JPEG artifacts.” (CC BY-NC 2.0 Munroe, 2019).

FIG. 9.4. : Paramètres lors de l'export d'un fichier au format CSV avec LibreOffice Calc (choisissez l'encodage de caractères UTF-8 et un séparateur de champ adapté).

Troisième partie

Préparation des données

10. Introduction

Dans ce chapitre, nous allons passer en revue quelques-unes des principales méthodes permettant d'importer des données dans R. La méthode la plus courante consiste à utiliser la fonction `read.csv()` avec un fichier CSV. Il s'agit d'une approche recommandée, car la plupart des types de fichiers qui contiennent des données tabulaires peuvent être enregistrés au format CSV. Le format CSV est idéal car il s'agit d'un format ouvert, en texte clair, qui peut être lu et modifié par de nombreux programmes différents. Il convient à l'archivage à long terme, car c'est un format stable. Cependant, il est souvent plus pratique de travailler directement avec le fichier dans lequel les données ont été saisies, et de travailler avec un format avec lequel nos collaborateurs sont à l'aise, comme une feuille de calcul. Il existe plusieurs packages qui permettent de lire différents format de feuilles de calcul, d'autres formats de fichiers de données tabulaires courants et même des bases de données, directement dans R. Une grande partie de ce chapitre est consacrée à l'import dans R de données stockées dans des feuilles de calcul, prêtes à être analysées. Si vous disposez de fichiers de données provenant d'un autre logiciel, tel que SPSS, vous pouvez utiliser les fonctions `read_*()` du paquet *haven* de Hadley Wickham de la même manière que `read_excel()` dans ce chapitre.

R est également capable de gérer des situations plus difficiles, comme le travail avec des centaines ou des milliers de fichiers de données, la lecture de données tabulaires à partir de documents Microsoft Word ou de PDF, et la lecture de données à partir de fichiers de texte brut non structurés. Dans ce chapitre, nous allons brièvement montrer comment gérer ces situations.

Il est rare que les données issues du travail sur le terrain ou obtenues directement d'un instrument soient immédiatement prêtes à être utilisées. Souvent, les données contiennent des incohérences et des erreurs mineures qui doivent être éliminées. L'identification et la suppression de ces contaminants sont rarement enseignées au cours de la formation de premier ou de deuxième cycle, et par conséquent, de nombreux chercheurs passent beaucoup de temps à se battre pour que leurs données soient exploitables. L'objectif de ce chapitre est d'étudier cinq concepts clés pour le nettoyage des données avec R. Ce chapitre vous donnera les outils nécessaires pour identifier les problèmes avec vos données, les corriger rapidement et simplement, et donner à vos données une forme appropriée pour l'analyse et la visualisation.

11. Importer et exporter

```
# read.table()
```

12. Traitement des données

12.1. Nettoyer

```
# abbreviate()
# replace()
# tolower() / toupper()
```

12.2. Selectionner

```
# startsWith()

# endsWith()
```

12.3. Filtrer

```
# subset()
```

12.4. Transformer

```
# transform()
# with()
```

12.5. Fusionner

```
# merge()
```

Quatrième partie

Éléments de statistiques

13. Introduction

Une *variable statistique* désigne une caractéristique mesurable qui peut prendre différentes *valeurs*. On distingue les *variables quantitatives* (ou numériques) qui décrivent des caractéristiques quantifiables et les *variables qualitatives* (ou catégoriques) qui sont des caractéristiques non quantifiables (fig. 13.1). Dans sa forme la plus simple, une variable statistique est représentée par un vecteur atomique.

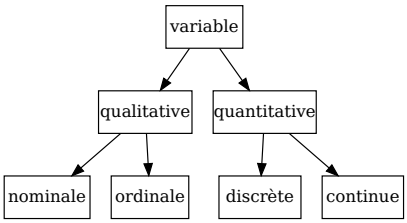


Fig. 13.1. : Les types de variables.

Variables quantitatives Les variables quantitatives prennent des valeurs numériques qui représentent des quantités mesurables. Si les valeurs d'une variable quantitative sont dénombrables et ont un nombre fini de possibilités, cette variable est dite *discrète*. Les valeurs d'une variable discrète sont souvent, mais pas uniquement, des nombres entiers (la population d'un pays par exemple). À l'inverse une variable quantitative est dite *continue* si les valeurs qu'elle peut prendre ne sont pas dénombrables et ont un nombre infini de possibilités dans un intervalle donné (la taille et le poids d'un groupe d'individus par exemple).

Variables qualitatives Les variables qualitatives représentent des caractéristiques non quantifiables, dont les valeurs sont appelées catégories ou *modalités*[1]. Une variable qualitative *nominale* décrit une catégorie sans ordre naturel (le genre, les couleurs sont ainsi des variables nominales). À l'inverse, une variable qualitative est dite *ordinale* s'il existe une relation d'ordre entre les catégories auxquelles

[1] Il peut arriver que des variables qualitatives apparaissent sous une forme numérique dans un jeu de données. Cette dernière résulte du codage des données. Pour ne pas commettre d'erreur d'interprétation il est donc important de consulter les **métadonnées** décrivant chacune des variables du jeu de données utilisé.

peuvent appartenir ses différentes valeurs (une grandeur mesurée sur une échelle petit/moyen/grand est une variable ordinale).

logical = variable qualitative nominale qui ne peut prendre que deux valeurs : `TRUE` (vrai) ou `FALSE` (faux)

14. Statistique descriptive

14.1. Indicateurs de position

14.2. Indicateurs de dispersion

15. Relations entre variables

correlation vs causalité

15.1. Covariance

15.2. Corrélation

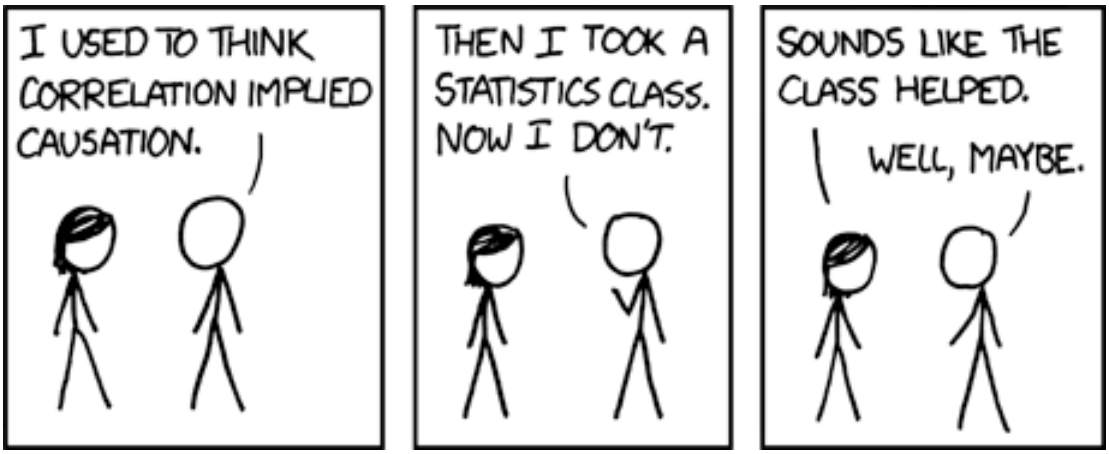


FIG. 15.1. : Correlation. "Correlation doesn't imply causation, but it does waggle its eyebrows suggestively and gesture furtively while mouthing 'look over there'." (CC BY-NC 2.0 MUNROE, 2009).

15.3. Régression

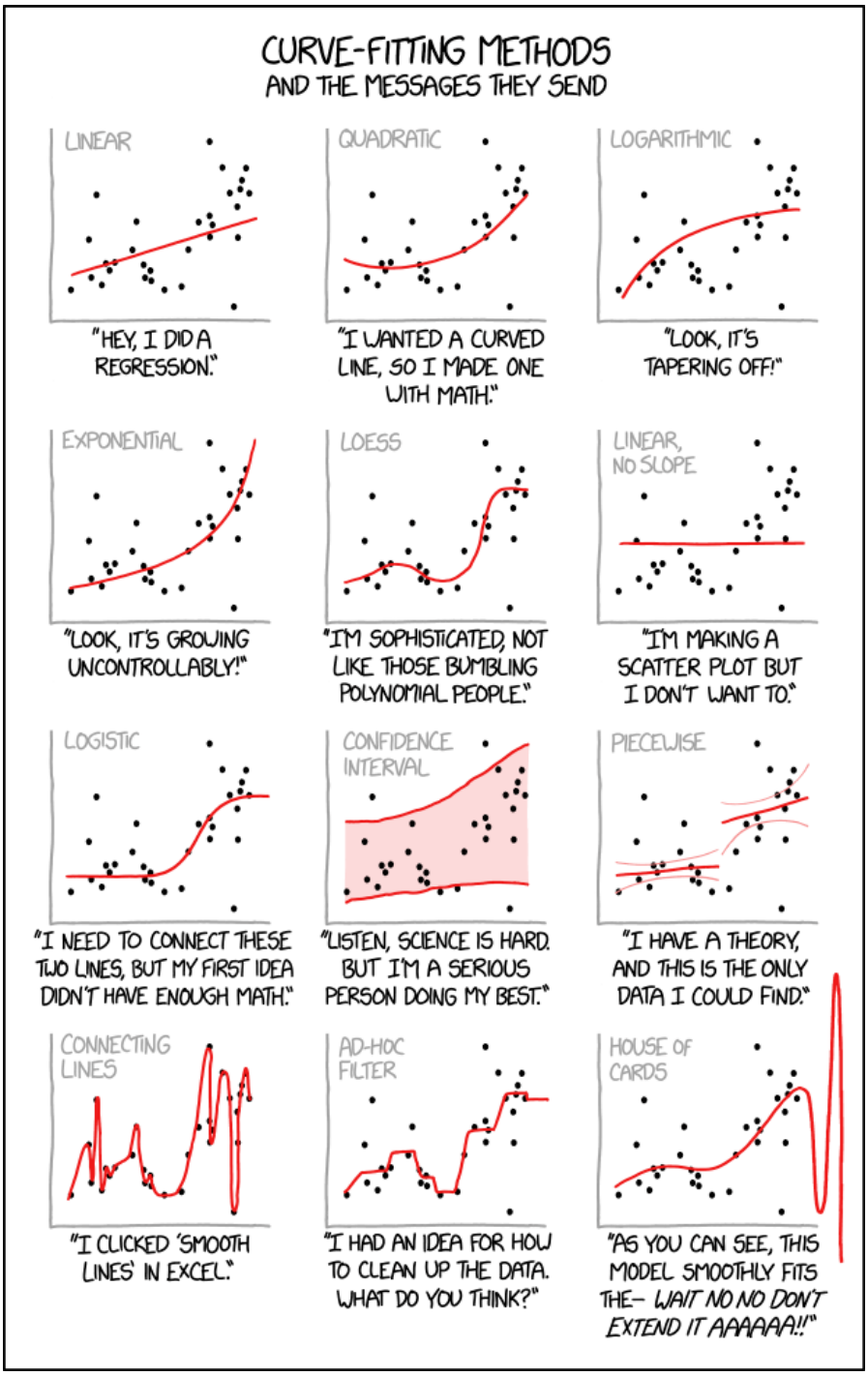


FIG. 15.2. : Curve-Fitting. "Cauchy-Lorentz : 'Something alarmingly mathematical is happening, and you should probably pause to Google my name and check what field I originally worked in.' " (CC BY-NC 2.0 MUNROE, 2018).

Cinquième partie

Visualisation de données

16. Introduction

16.1. Recherche empirique en matière de visualisation de données

Il n'existe pas de principes absolus ou de pratiques idéales en matière de visualisation de données, mais il existe une abondante littérature sur la question. Il résulte de cette dernière quelques principes récurrents, souvent motivés par des préoccupations éthiques et esthétiques. Un exemple de préoccupation éthique est le principe selon lequel seules les données séquentielles peuvent être reliées à un graphique linéaire, car la ligne implique un changement continu entre les points. Les mesures de catégories, telles que les types d'artefacts, les lieux et les méthodes, ne doivent pas être reliées par des lignes (par exemple, il n'y a pas de séquence continue entre un artefact en pierre et un os d'animal). Un principe connexe est que, lorsque les magnitudes absolues des données sont importantes, l'axe vertical doit commencer à zéro (Robbins, 2005 ; Strange, 2007). L'affichage des données le long d'un axe vertical qui ne comprend pas le zéro peut donner une fausse représentation de l'étendue des données et exagérer l'ampleur relative entre les valeurs. Un autre ensemble influent d'idées provient des mantras minimalistes de Tufte (2001). L'un de ses principes les plus fréquemment cités est "maximiser le rapport données-encre, dans la limite du raisonnable" (Tufte, 1990). L'"encre de données" fait référence à l'encre utilisée pour afficher les données, et le rapport entre les données et l'encre est donc la quantité d'encre de données par rapport à l'encre totale utilisée dans une visualisation. Ces principes garantissent que les données ne sont pas cachées ou déformées par de mauvais choix ou des éléments non pertinents sur la visualisation, et que le lecteur peut apprécier les données dans la visualisation sans distraction ni confusion. Les conséquences pratiques de ce conseil sont notamment d'éviter les graphiques tridimensionnels lorsque des graphiques bidimensionnels suffisent (par exemple, les diagrammes à barres et les diagrammes circulaires). De même, le fait d'omettre les éléments superflus — grilles, couleurs et éléments artistiques sur les graphiques — contribue à améliorer le rapport données-encre.

Si ces principes ont un attrait intuitif et sont largement répétés, ils peuvent conduire à des graphiques très minimalistes difficiles à interpréter (Tukey, 1990). Inbar et al. (2007) et Kulla-Mader (2007) ont constaté que les graphiques standard étaient largement préférés aux graphiques minimalistes de style Tufte. Kosslyn et Chabris (1992) et Carswell (1992), ont soulevé la question de savoir comment décider ce qui est de l'encre de données et ce qui ne l'est pas, et ont conclu que cela est souvent très subjectif.

Hullman et al. (2011) ont suggéré que certaines données-encres peuvent profiter aux lecteurs en favorisant l'engagement avec la visualisation. Par exemple, Bateman et al. (2010) ainsi que Li et Moacdieh (2014) ont constaté que les sujets à qui l'on présentait des graphiques avec des éléments graphiques avaient beaucoup plus de chances de comprendre le message du graphique que ceux à qui l'on présentait des graphiques sans éléments graphiques. Kelly (1989) n'a trouvé aucune différence dans le rappel immédiat d'informations provenant de graphiques à encres de données élevées et faibles dans un format de journal. De même, McGurgan (2015) a constaté que les participants ont signalé des niveaux similaires de précision et d'effort mental lorsqu'ils ont répondu à des questions de compréhension de graphiques à l'aide de diagrammes à barres et de boxplots avec des ratios d'encre de données variables. D'autre part, Gillan et Richman (1994) ont trouvé un soutien empirique pour le principe de maximisation de l'encrage des données. Ils ont constaté que le pourcentage d'interprétations correctes des données des graphiques était significativement plus faible pour les graphiques à faible encrage que pour les graphiques à encrage moyen et élevé. Ce bref résumé des recherches sur le ratio data-ink montre qu'il s'agit d'un concept problématique, qui ne bénéficie que d'un soutien empirique équivoque. En somme, il semble que les principes fondés sur des questions subjectives d'esthétique des graphiques, souvent observés dans les conceptions de graphiques de style Tufte, ne conduisent pas toujours aux visualisations les plus efficaces.

Ces résultats mitigés suggèrent que le minimalisme esthétique ne garantit pas toujours que nos visualisations de données soient faciles à interpréter avec précision. Quels sont donc les principes de base pour optimiser la vitesse à laquelle un lecteur peut percevoir les modèles dans les données, et la précision des informations qu'il peut extraire de la visualisation ? Dans le contexte d'une présentation de poster, ces optimisations sont hautement souhaitables, car le lecteur d'un poster espère généralement obtenir des informations beaucoup plus rapidement que s'il était assis à lire un article savant.

Cleveland et McGill (1984) ont mené des expériences en utilisant plusieurs types courants de visualisation de données pour tester la précision avec laquelle les sujets pouvaient lire des valeurs ponctuelles et faire des comparaisons dans les données. Ils ont constaté que les types de graphiques basés sur la longueur (tels que les graphiques à points et les graphiques à barres) étaient lus avec beaucoup plus de précision que les types de graphiques basés sur l'angle, la surface ou le volume (tels que les graphiques circulaires et les graphiques tridimensionnels). En outre, ils ont constaté que les performances sont nettement moins bonnes sur des diagrammes à barres empilées que sur des diagrammes à barres alignées, et que les comparaisons entre barres adjacentes sont plus précises qu'entre barres très éloignées. De nombreuses études ultérieures ont généralement confirmé ces résultats (Heer & Bostock, 2010 ; Kosara & Ziemkiewicz, 2010 ; Talbot et al., 2014). Heer et Bostock (2010) ont reproduit les résultats de base pour la comparaison des tailles entre catégories, et ont également constaté que l'ajout de lignes de quadrillage sur une parcelle améliorait la précision. Kosara et Ziemkiewicz (2010) ont testé des diagrammes de type tarte carrée ou gaufre (un carré divisé en 10 x 10 = 100 cellules) ainsi que des diagrammes de type tarte, barre empilée et beignet, et

ont constaté que les répondants étaient plus confiants dans leur lecture des diagrammes de type tarte carrée et plus précis dans la lecture des valeurs du diagramme, par rapport aux autres types. Zubiaga et MacNamee (2015) ont testé les répondants avec cinq types de graphiques (diagrammes à barres montrant la valeur moyenne de la distribution, diagrammes en essaims, boîtes à moustaches, diagrammes à barres empilées et histogrammes) afin de déterminer leur efficacité relative pour visualiser les distributions de variables. Ils ont constaté que les histogrammes sont les plus complets en termes de détails fournis, et qu'ils constituent le type de graphique qui conduit les répondants à la compréhension la plus précise des données sous-jacentes. Rangecroft (2003) et Schonlau et Peters (2012) ont constaté que les répondants peuvent lire les diagrammes circulaires en 2D avec plus de précision que les diagrammes circulaires en 3D. Zacks et al. (1998) ont trouvé un résultat similaire pour les diagrammes à barres 2D par rapport aux diagrammes à barres 3D.

Des expériences récentes ont jeté un peu de lumière sur la rivalité traditionnelle entre les diagrammes en secteurs et les diagrammes en barres (Spence, 2005). Pour les jugements de comparaison entre catégories, les barres sont jugées plus précisément que les tartes (Feldman-Stewart et al., 2000). Cependant, pour les estimations de la proportion du tout, les diagrammes en camembert sont aussi précis que les diagrammes en bâtons (Simkin & Hastie, 1987 ; Spence, 1990). Pour les comparaisons par paires, les diagrammes circulaires et les diagrammes à barres ont également des performances similaires (Meyer et al., 1997 ; Spence & Lewandowsky, 1991). Ces études montrent que, dans certaines conditions, les graphiques en secteurs peuvent être plus efficaces que les graphiques en barres. Le meilleur choix du type de graphique dépend de l'objectif du graphique (Kosslyn & Chabris, 1992), et les preuves ne permettent pas de faire une prescription universelle sur les types de graphiques.

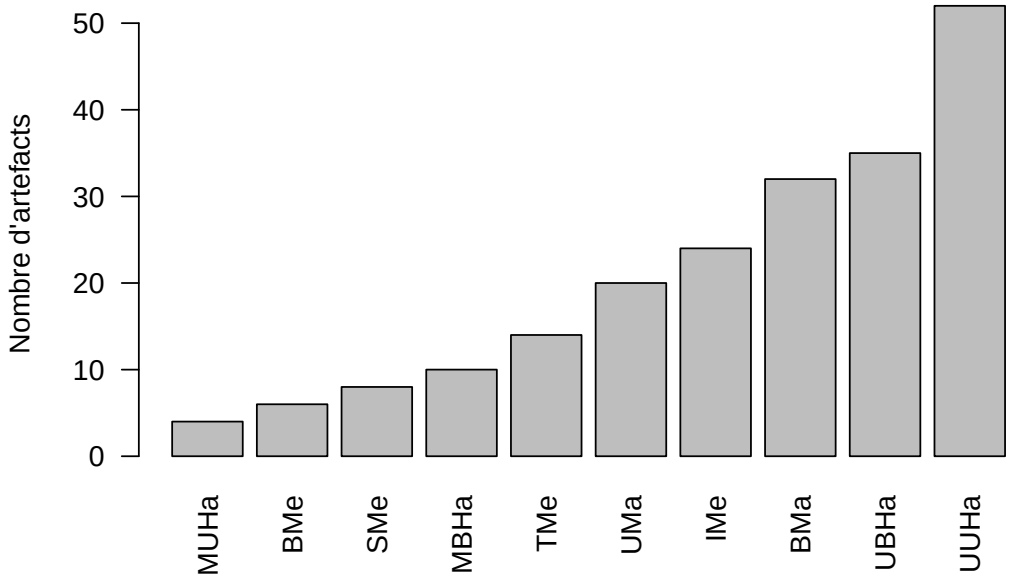
Bien que ces études empiriques démontrent une relation complexe entre le type de graphique et l'efficacité, elles peuvent fournir un classement simple, bien qu'approximatif, des stratégies de visualisation des données. Les diagrammes à points et les diagrammes à barres se situent généralement à l'extrémité supérieure du spectre, ainsi que des styles plus exotiques comme les diagrammes en gaufre et les histogrammes. Parmi les types de graphiques de rang inférieur, on trouve les diagrammes à barres empilées et les diagrammes circulaires. Tout type de graphique en 3D est classé en dernier.

17. Représentations

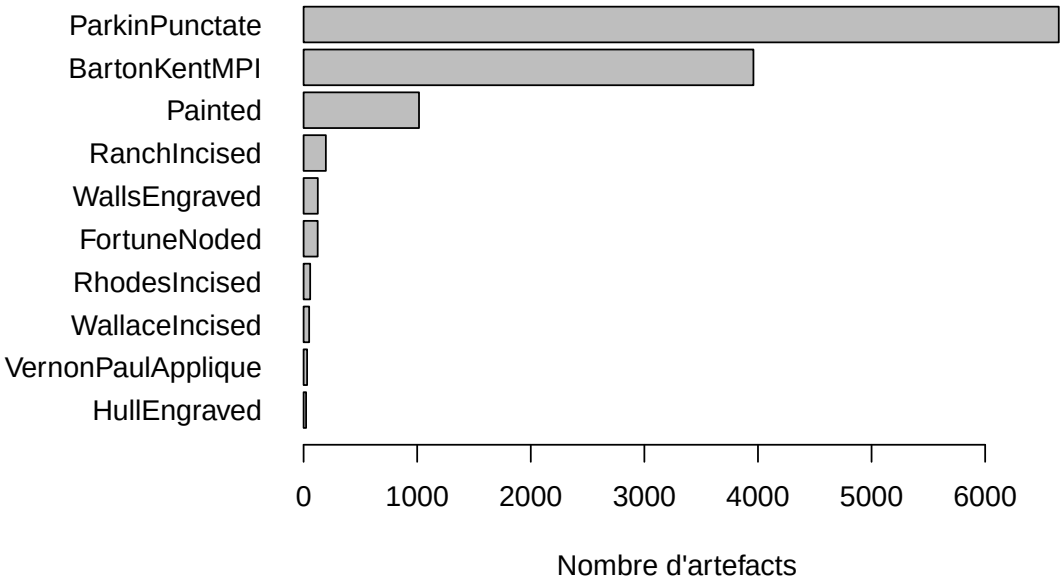
17.1. Représenter des quantités

On peut créer un diagramme en barres à l'aide de la fonction `barplot()`, dont l'argument `height` accepte un vecteur ou une matrice permettant de décrire les barres qui composent le graphique.

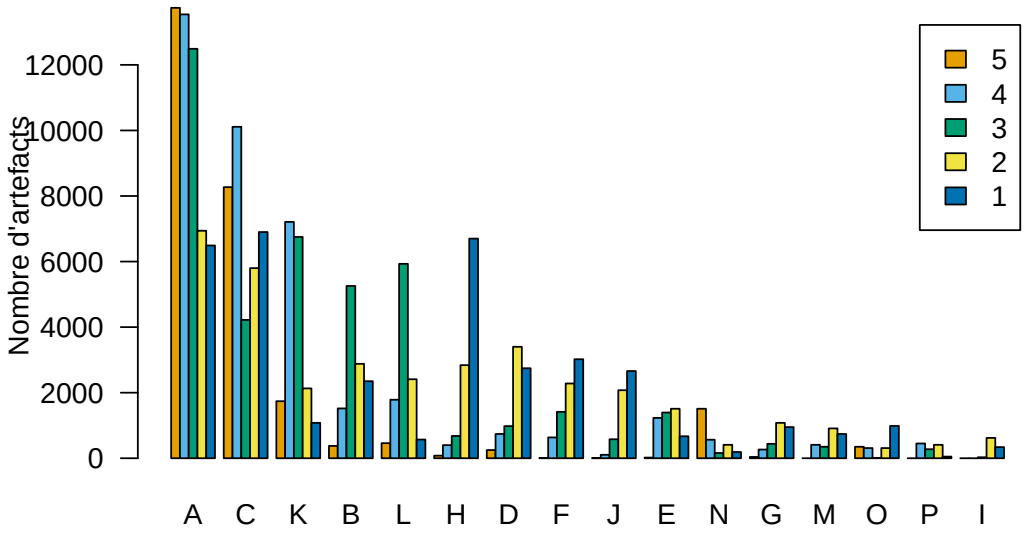
```
# Charger les données depuis le package folio
data(chevelon, package = "folio")

# Calculer la somme de chaque colonne (type d'artefact)
# Trier dans l'ordre croissant
chevelon_total <- sort(colSums(chevelon))

barplot(
  height = chevelon_total,
  ylab = "Nombre d'artefacts",
  las = 2 # étiquettes perpendiculaires aux axes
)
```




```r
# Charger les données depuis le package folio
data(mississippi, package = "folio")

# Calculer la somme de chaque colonne (type d'artéfact)
# Trier dans l'ordre croissant
mississippi_total <- sort(colSums(mississippi))

# Ajuster les marges autour du graphique
# (permet de ne pas tronquer les étiquettes)
par(mar = c(5, 9, 4, 1) + 0.1)
barplot(
  height = mississippi_total,
  horiz = TRUE, # dessiner les barres horizontalement
  xlab = "Nombre d'artefacts",
  las = 1 # étiquettes des axes horizontales
)
```




```r
# Charger les données depuis le package folio
data(compiegne, package = "folio")

# Déterminer l'ordre des colonnes à partir de leurs sommes
# (ici dans l'ordre décroissant)
compiegne_order <- order(colSums(compiegne), decreasing = TRUE)
# Permuter les colonnes
compiegne_sorted <- compiegne[, compiegne_order]

barplot(
  height = as.matrix(compiegne_sorted), # doit être une matrice
  col = c("#E69F00", "#56B4E9", "#009E73", "#F0E442", "#0072B2"),
  legend.text = rownames(compiegne),
  beside = TRUE,
  ylab = "Nombre d'artefacts",
  las = 1
)
```



17.2. Représenter des propotions

17.3. Représenter des distributions

17.4. Réprésenter des relations entre variables

17.5. Représenter des séries temporelles

17.6. Représenter des tendances

17.7. Diagrammes 3D

Ne faites pas ça.

Sixième partie

Données de comptage

18. Introduction

Septième partie

Données de composition

19. Introduction

19.1. Étendue du problème

Soient deux archéologues, Howard et Arthur qui se partagent l'étude de plusieurs ensembles de mobilier. On considère pour l'exemple la situation idéale, où chacun à reçu une moitié identique de chaque ensemble et que tous deux sont parfaitement routiniers de ce type d'étude. Howard quantifie son matériel en distinguant les catégories A, B, C et D. De son côté Arthur ne quantifie que les catégories A, B et C (tab. 19.1). Les résultats d'Arthur constituent ainsi une sous-composition de ceux obtenus par Howard et, en toute logique, les observations faites par les deux archéologues sur les catégories de mobilier qu'ils ont en commun devraient s'accorder.

A partir de leurs résultats, exprimés en pourcentages, les deux archéologues cherchent ensuite à mettre en évidence les relations de dépendance entre certaines catégories de mobilier. Pour cela, tous deux calculent à partir de leurs données le coefficient de corrélation entre les catégories A et B. Howard et Arthur obtiennent 0.5 et -1, respectivement, alors qu'ils étudient le même matériel.

19.2. Définitions

Une composition décrit les parties d'un tout comme des parts mutuellement exclusives et exhaustives (Aitchison, 1982, 1986). Par exemple, la composition élémentaire d'une céramique correspond aux teneurs des différents oxydes la constituant (les teneurs sont exprimées en pourcentages massiques, leur somme est égale à 100%).

Tab. 19.1. : Pourcentages de céramiques appartenant à chaque catéogrie observés par deux archéologues.

Howard				Arthur		
A	B	C	D	A	B	C
10	20	10	60	25.0	50.0	25
20	10	10	60	50.0	25.0	25
30	30	20	20	37.5	37.5	25

Une composition x est donc définie comme un vecteur de longueur D dont tous les **composés** $(x_1, x_2, \dots, x_D)$ sont des réels strictement positifs $(x_1 > 0, \dots, x_D > 0)$:

$$x = [x_1, x_2, \dots, x_D] \, \forall x_i > 0 \tag{19.1}$$

Les composés portent une information relative : seuls les rapports entre ces composés sont d'intérêt, car la somme k des teneurs du vecteur composition est arbitraire. Pour un ensemble de compositions cette somme est constante et est généralement égale à 1, 100, 10^6, selon que l'on considère des parts d'une unité, des pourcentages ou, par exemple, des ppm, en fonction des conventions.

L'opération qui permet la normalisation d'une composition x est appelée **fermeture** des données (notée $\mathscr{C}$ pour *closure*) et consiste à diviser chaque partie par le tout, puis à multiplier le résultat par une constante k (19.2).

$$\mathscr{C}(x) = \frac{k}{\sum_{i=1}^{D} x_i} [x_1, x_2, \dots, x_D] \tag{19.2}$$

Par exemple, dans le cas d'un objet de masse totale m, décomposé en ses différents constituants de masses $m_1, m_2, \dots, m_D$, on a : $x = \frac{1}{m} [m_1, m_2, \dots, m_D]$.

Pour une composition x donnée, une sous-composition y est définie comme un sous-ensemble de x ayant subi une opération de fermeture (19.3).

$$y = [y_1, y_2, \dots, y_C] = \mathscr{C} [x_1, x_2, \dots, x_C] \, \forall C < D \tag{19.3}$$

Remarque

En pratique, seules des sous-compositions sont généralement connues et celles-ci dépendent du problème à traiter et de la capacité à reconnaître les différents composés d'un système et à les mesurer. On ne mesure jamais l'intégralité des éléments chimiques lors d'une analyse, on ne connaît pas l'ensemble des espèces appartenant à un écosystème, certaines catégories d'objets peuvent être ignorées lors d'une étude (ou ne pas avoir été reconnues du fait de la fragmentation du matériel), etc.

Du fait de la fermeture des données, l'univers d'une composition (c'est-à-dire l'ensemble de toutes les valeurs susceptibles d'être prises par ses différents composés) est un espace particulier. Ce dernier possède une dimension de moins que le nombre de composés : bien qu'il y ait D observations pour chaque individus, il n'y a en effet que $D - 1$ valeurs indépendantes. Autrement formulé, si les $D - 1$ teneurs sont connues, sachant que leur somme est constante, la dernière teneur est connue également.

Cet univers (ou espace échantillon) peut être facilement représenté dans le cas de compositions à deux ou trois composés. Lorsque ces compositions sont projetées dans un espace réel, on constate qu'elles appartiennent toutes à un même sous-espace triangulaire (simplexe ; fig. 19.1)[1].

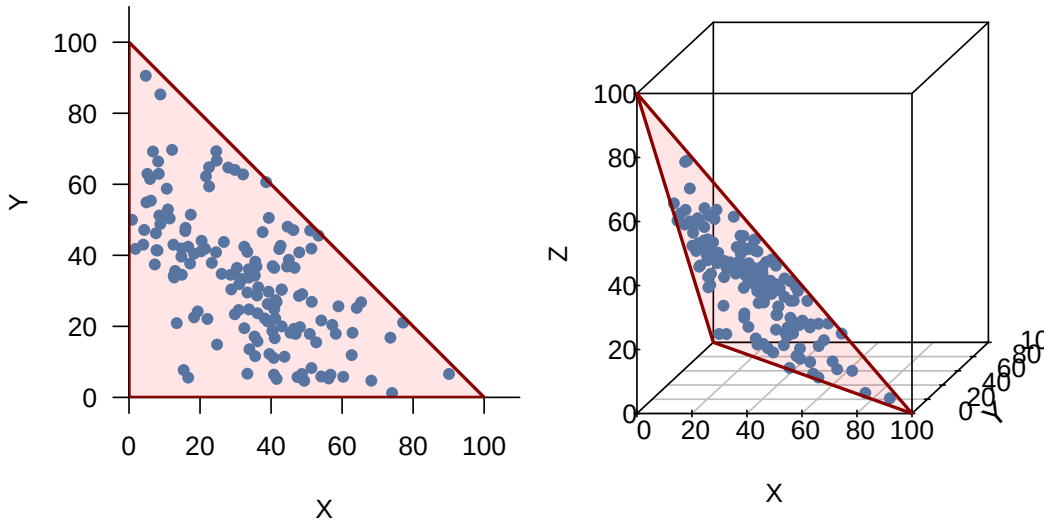


Fig. 19.1. : Projection de compositions à deux (gauche) et trois composés (droite) dans un espace réel à deux et trois dimensions, respectivement. L'ensemble des points appartient à un sous-espace figuré en rouge. Dans le cas d'une composition à trois composés, ce sous-espace correspond au diagramme ternaire des trois composés.

19.3. Principes

L'analyse de données de composition est régie par quatre principes :

Scale invariance La comparaison d'échantillons de tailles différentes est permise par l'opération de fermeture des données qui les force à partager la même somme k. Toute analyse de ces données doit donc conduire à des résultats similaires indépendamment de la valeur de k.

Perturbation invariance Toute composition peut être exprimée à l'aide d'unités différentes selon les grandeurs mesurées (kg, pourcentages massiques, dm^3, pourcentages volumiques, moles, pression partielle, etc.). Le choix des modalités de quantification revient alors à l'expérimentateur et plusieurs approches peuvent être

[1] En termes géométriques, les observations appartiennent à un hyperplan (S^d) de l'espace euclidien à D dimensions $\mathbb{R}^D$ (avec $d = D - 1$).

également considérées comme pertinentes. Les différents composants d'une composition sont qualitativement différents : chacun portant une information qui lui est propre et il peut être utile de souligner une propriété particulière[2]. Malgré le changement d'unité, la composition mesurée reste la même et toute analyse statistique sensible doit permettre d'obtenir des résultats qualitativement identiques, quelle que soit l'unité choisie.

Permutation invariance Le résultat de l'analyse ne doit pas dépendre de l'ordre des composants dans le vecteur composition.

Subcompositional coherence Dans certaines situations, l'utilisation d'une sous-composition peut être pertinente et permet notamment l'inspection des données dans des espaces de moindre dimension.

Ainsi, ni la somme des composants du vecteur composition, ni le choix des unités, ni la séquence des composés ou l'étude d'un sous-ensemble issu d'une composition, ne doivent affecter les résultats de l'analyse.

Les définitions classiques de la covariance et de la corrélation ne respectent pas le principe de *subcompositional coherence* (voir l'encadré ci-après). Ce problème se manifeste particulièrement lorsque les teneurs brutes de deux composés sont représentés au sein d'un diagramme de dispersion[3]. Il n'y a en effet aucune garantie que les projections à partir du jeu de données initial ou d'une sous-composition conduisent à des observations similaires. Bien que ce type de représentation soit particulièrement courant, il est impossible de faire confiance aux tendances ainsi mises en évidence (Aitchison & Ng, 2005)[4].

La plupart des méthodes dédiées aux statistiques multivariées, dont la mise en œuvre repose sur l'exploitation des matrices de variance-covariance, sont donc inapplicables. Le recours à ces dernières requiert une transformation préalable des données, permettant de casser la contrainte de somme constante introduite par la fermeture des données.

[2]Par exemple, lors du passage de la masse au pourcentage volumique en utilisant la densité de chaque composant.

[3]Ou diagramme de Harker (Harker, 1909, p. 119).

[4]Voir la réponse de Cortés (2009).

Remarque

L'approche courante lorsque l'on cherche à explorer les relations de d'interdépendance dans le cas d'un jeu de données multivarié est de s'attacher à la covariance et à la corrélation entre variables. Pour deux variables aléatoires X et Y, la covariance est en effet une mesure de la tendance de ces deux variables à évoluer dans le même sens (indiqué par le signe de la covariance), tandis que la corrélation (covariance normalisée) est une mesure de l'intensité de la relation entre ces deux variables.
Sachant les propriétés de la covariance, pour une composition x de longueur D telle que $x = [x_1, x_2, \dots, x_D]$ avec $\sum_{i=1}^{D} x_i = 1$, on a :

$$\mathrm{Cov}(x_1, x_1 + \cdots + x_D) = 0 \tag{19.4}$$

Soit :

$$\mathrm{Cov}(x_1, x_2) + \cdots + \mathrm{Cov}(x_1, x_D) = -\mathrm{Var}(x_1) \tag{19.5}$$

Ainsi, le membre de droite de l'équation (19.5) est toujours négatif (excepté dans le cas où le premier composé est une constante), si bien qu'au moins une des covariances du membre de gauche est forcement négative (Aitchison, 1986, p. 53-54). Autrement formulé, au moins un élément de chaque ligne de la matrice de covariance doit être négatif (soit au moins D éléments de la matrice).
Dans le cas des données de composition, la covariance entre deux composés est donc contrainte par la somme constante et dépend des autres composés également présents dans le jeu de données. De plus, la matrice de covariance est singulière (van den Boogaart & Tolosana-Delgado, 2013). L'influence de la somme constante au sein de la matrice de covariance se traduit en terme de corrélation : s'agissant de compositions, le coefficient de corrélation n'est pas libre de prendre une valeur dans l'intervalle $[-1; 1]$ mais correspond à une valeur arbitraire[a].
L'exemple d'une composition à deux composés x_1 et x_2 tels que $x_1 + x_2 = 1$ est particulièrement illustratif du problème (Aitchison, 1986, p. 54), on a en effet :

$$\mathrm{Cov}(x_1, x_2) = \mathrm{Cov}(x_1, 1 - x_1) = -\mathrm{Var}(x_1) = -\mathrm{Var}(x_2)$$

Soit :

$$\mathrm{Cov}(x_1, x_2) = \frac{\mathrm{Cov}(x_1, x_2)}{\sqrt{\mathrm{Var}(x_1)\mathrm{Var}(x_2)}} = -1$$

[a] Pearson (1896) a été le premier à relever le risque que représente l'interprétation de ce qu'il nomme alors *spurious correlation* ("corrélation fallacieuse"), mettant alors en évidence que la corrélation entre des rapports présentant des parties communes au dénominateur est arbitraire. Le problème est redécouvert au milieu du XXe siècle par Chayes (1960) mais il faut attendre les premiers travaux de Aitchison (1982) pour qu'une solution formelle soit proposée.

20. Transformations de données

Sachant les contraintes propres aux données de composition, la plupart des outils statistiques et méthodes d'analyse ne peuvent être mises en œuvre sans prendre le risque d'aboutir à des inférences erronées. Néanmoins, il est possible de trouver une transformation permettant de projeter les données de compositions dans un espace réel non contraint. Aitchison (1986) a montré qu'une telle transformation peut être obtenue par le logarithme de rapports de composés.

21. Détection des valeurs aberrantes

La détection de valeurs aberrantes (*outliers*), ou considérées comme inattendues, est une tâche courante lors de l'analyse de données de composition. Cette dernière repose sur la définition d'un seuil ou d'une valeur limite permettant de séparer les valeurs aberrantes du reste des données (Filzmoser et al., 2005 ; Reimann et al., 2005). Ce seuil correspond généralement à une distance limite par rapport au centre du jeu de données, au-delà de laquelle les individus ne sont pas uniquement affectés par la variation naturelle du phénomène étudié, voire relèvent d'un ou de plusieurs processus complètement différents (Filzmoser et al., 2005).

Une telle approche nécessite de bien distinguer les valeurs aberrantes (observations issues d'une ou plusieurs distributions différentes) des valeurs extrêmes (issues de la même distribution malgré leur éloignement du centre) (Reimann et al., 2005). Cette distinction constitue une des principales difficultés : les valeurs aberrantes ne sont pas nécessairement des valeurs extrêmes, en particulier lorsqu'elles résultent de phénomènes secondaires (taphonomie, contaminations…). Ainsi, les approches univariées sont généralement inefficaces pour détecter les valeurs aberrantes (Filzmoser et al., 2005). Les données de composition sont par nature mutlivariées. La détection de valeurs abbérantes doit donc reposer sur la position des données (distance par rapport au centroïde), mais prendre également en compte la forme de ces dernières (Filzmoser et al., 2005).

La distance de Mahalanobis (21.6) permet de déterminer la similarité entre un individu x multivarié et un ensemble d'observations en prenant en compte la position (moyenne μ), ainsi que la forme et la taille (quantifiées par la matrice de covariance Σ) de cet ensemble.

$$d_M = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)} \tag{21.6}$$

Dans le cas de données multivariées à p dimensions distribuées normalement, la distribution du carré des distances de Mahalanobis (d_M^2) de l'ensemble des observations suit approximativement une loi du χ^2 à p degrés de liberté (fig. 21.1).

```
## Simuler 500 observations de deux variables
## Les deux variables ont pour moyenne 0 et écart-type 1
## La corrélation entre les deux variables est fixée à 0.8
sigma <- matrix(c(1, 0.8, 0.8, 1), nrow = 2, ncol = 2) # covariance
z <- MASS::mvrnorm(n = 500, mu = c(0, 0), Sigma = sigma)
```

```r
## Calculer les distances de Mahalanobis
## (chaque observation par rapport à l'ensemble des données)
## NB : mahalanobis() retourne le carré de la distance
d2 <- mahalanobis(x = z, center = colMeans(z), cov = cov(z))

## Paramètres graphiques
par(mfrow = c(1, 2), mar = c(2, 4, 1, 1) + 0.1, las = 1)

## Diagramme de dispersion des observations
plot(x = z, type = "p", xlab = "", ylab = "", asp = 1)
abline(h = 0, v = 0, lty = 2)

## Histogramme des distances
hist(d2, freq = FALSE, main = "", xlab = "", ylab = "Densité")

## Loi du khi-deux à 2 degrés de liberté
curve(dchisq(x, df = 2), from = 0, to = max(d2),
      col = "red", lwd = 2, add = TRUE)
```

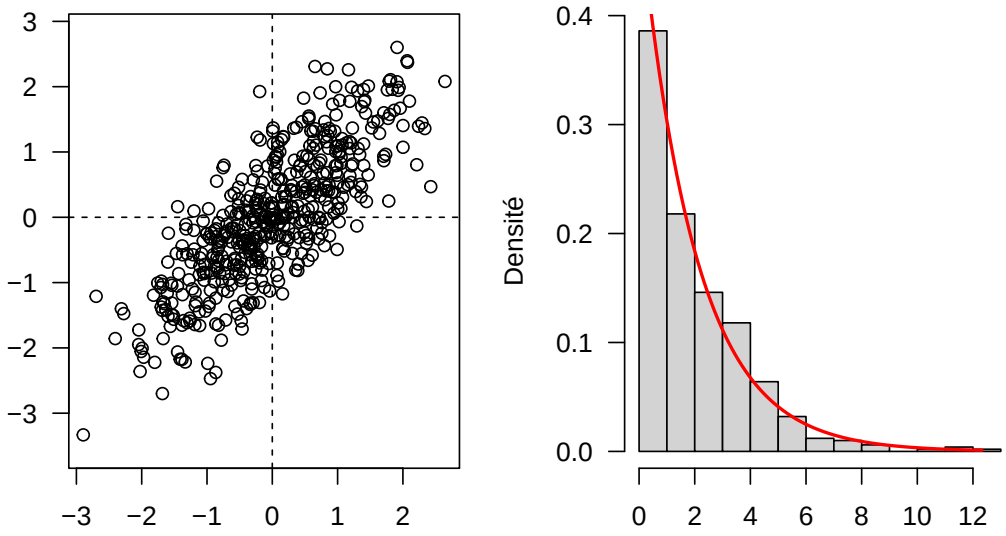


Fig. 21.1. : Gauche : données simulées distribuées normalement. Droite : histogramme des distances de Mahalanobis et densité de probabilité de la loi du χ^2 à 2 degrés de liberté (en rouge).

Une valeur abérrante peut ainsi être définie comme ayant une distance de Mahalanobis très importante par rapport à l'ensemble des observations. De cette façon, une certaine proportion des observations peut être isolée, par exemple : les 2% de valeurs les plus

élevées, soit les valeurs supérieures au quantile d'ordre 0,98 (98e centile) de la loi du χ^2. Pour un seuil donné, on peut ainsi définir des ellipsoïdes ayant la même distance de Mahalanobis par rapport au centre des données (fig. 21.2). Une telle approche soulève cependant quelques difficultés.

D'une part, la distance de Mahalanobis est susceptible d'être fortement affectée par la présence de valeurs aberrantes. Rousseeuw et van Zomeren (1990) recommandent ainsi d'utiliser des méthodes robustes (qui ne sont pas excessivement affectées par la présence de valeurs aberrantes) pour l'estimation de la position et de la dispersion des données[1]. Par exemple, la figure 21.2 représente la masse du cerveau de différents animaux en fonction de leur masse corporelle. Les données suivent une même tendance, à l'exception des dinosaures qui présentent une importante masse corporelle et un petit cerveau. Lorsque des estimateurs classiques sont utilisés, l'ellipse de tolérance est fortement affectée par les dinosaures. Au contraire, l'usage d'un estimateur robuste permet d'exclure les dinosaures qui constituent effectivement des valeurs aberrantes au regard de l'ensemble des données.

```r
## Répliquer la figure 1 de Rousseeuw et van Zomeren (1990)
data("Animals", package = "MASS")
brain <- log10(Animals) # Echelle logarithmique

## Paramètres graphiques
par(mar = c(4, 4, 1, 1) + 0.1, las = 1)

## Diagramme de dispersion
plot(
  x = brain, type = "p", asp = 1,
  xlim = c(-3, 7), ylim = c(-2, 6),
  xlab = "Masse corporelle (kg)",
  ylab = "Masse du cerveau (g)"
)
## Mettre en évidence les dinosaures
dino <- c("Dipliodocus", "Triceratops", "Brachiosaurus")
lines(
  x = brain[rownames(brain) %in% dino, ],
  type = "p", pch = 16
)

## Ellipses de tolérance
## Estimateurs classiques
car::ellipse(
```

[1] L'ellipsoïde de volume minimal (MVE pour *minimum volume ellipsoid*) et le déterminant de covariance minimale (MCD pour *minimum covariance determinant*) sont sans doute les estimateurs les plus couramment utilisés (Filzmoser et al., 2005).

```
  center = colMeans(brain),
  shape = cov(brain),
  radius = sqrt(qchisq(0.975, df = 2)),
  center.pch = NULL, col = "blue", lty = 3
)
## Estimateurs robustes
rob <- MASS::cov.rob(brain, method = "mve") # Ellipsoïde de volume minimal
car::ellipse(
  center = rob$center,
  shape = rob$cov,
  radius = sqrt(qchisq(0.975, df = 2)),
  center.pch = NULL, col = "blue", lty = 1
)
```

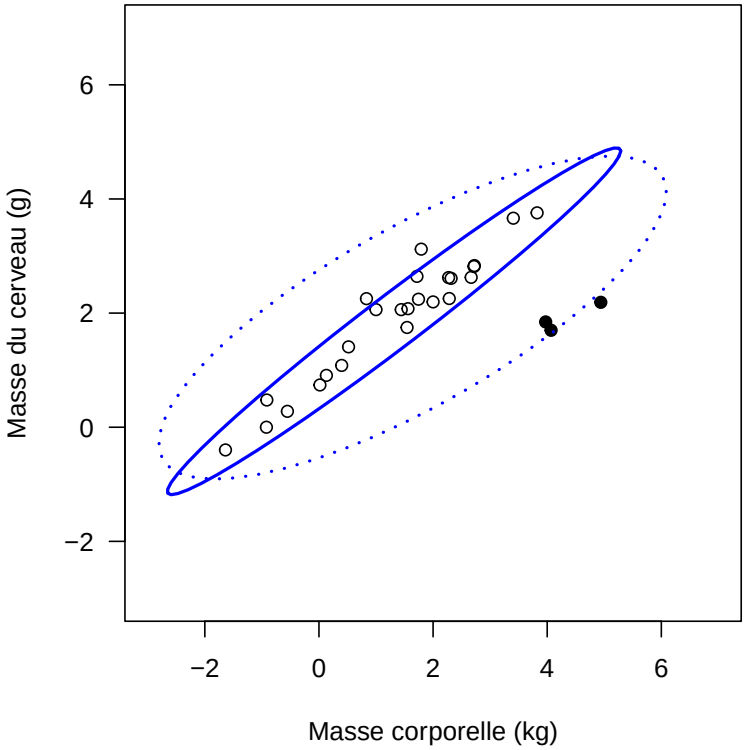


FIG. 21.2. : Masse du cerveau en fonction de la masse corporelle de 28 espèces animales (échelle logarithmique, base 10). Les symboles pleins correspondent aux dinosaures. Ellipses de tolérance (seuil fixé à $\chi^2_{2;0.975}$) calculées à partir de la moyenne et de la covariance classiques (tirets) et à l'aide d'estimateurs robustes (ellipsoïde de volume minimal ; trait plein).

D'autre part, le choix du seuil permettant de caractériser une observation comme aber-

rante doit faire l'objet d'une discussion. Il n'y a en effet aucune raison *a priori* pour qu'un seuil particulier soit applicable à tous les jeux de donnée (Filzmoser et al., 2005). Garrett (1989) propose d'utiliser une méthode graphique en représentant les distances robustes, ordonnées par rang, en fonction des quantiles théoriques de la loi du χ^2 (fig. 21.3). Les valeurs extrêmes sont alors retirées jusqu'à ce que les données s'alignent sur une même droite (fig. 21.3). Il est possible de combiner ces ces approches en fixant un seuil pour identifier de potentielles valeurs aberrantes, puis d'inspecter les données afin d'expliquer les différences observées en faisant appel à l'expérience de l'analyste[2].

```
## Distance de Mahalanobis robuste
d <- sqrt(mahalanobis(brain, center = rob$center, cov = rob$cov))
## Trier par rang
d <- sort(d)

## Paramètres graphiques
par(mar = c(4, 4, 1, 1) + 0.1, las = 1)

## Quantiles théoriques
q <- qchisq(ppoints(length(d)), df = 2)

## Diagramme Quantile-Quantile
plot(x = q, y = d, xlab = "Quantile théorique",
     ylab = "Distance de Mahalanobis robuste")
## Tracer la droite théorique
qqline(y = d, distribution = function(p) qchisq(p, df = 2), col = "blue")
## Mettre en évidence les dinosaures
text(x = q[d > 9], y = d[d > 9], labels = names(d[d > 9]), pos = 2)
```

Enfin, comme nous l'avons vu précédemment (§ 19.3), la définition de la covariance ne respecte pas le principe de *subcompositional coherence*. Dans le cas des données de composition, une transformation préalable des données est donc nécessaire (§ 20). Filzmoser et Hron (2008) ont montré que les distances de Mahalanobis sont les mêmes quelle que soit la transformation utilisée (ALR, CLR ou ILR) si des estimateurs classiques de position et de dispersion sont utilisés. Si des estimateurs robustes sont utilisés, les distances sont les mêmes pour des données transformées ALR ou ILR.

[2] Filzmoser et al. (2005) proposent quant à eux une méthode automatique, permettant de distinguer les valeurs extrêmes issues d'une distribution normale des valeurs aberrantes issues d'une distribution différente.

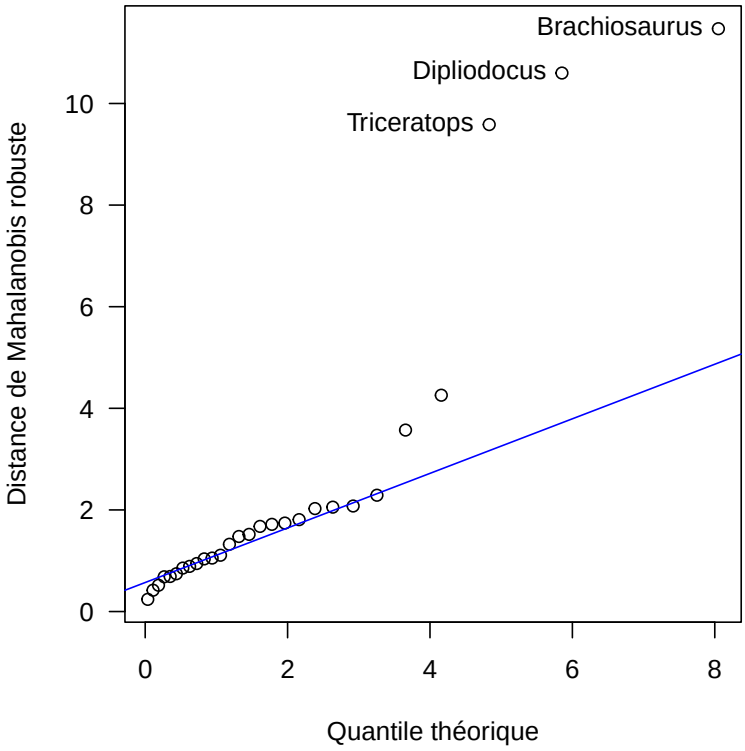


FIG. 21.3. : Masse du cerveau en fonction de la masse corporelle de 28 espèces animales (échelle logarithmique, base 10). Les symboles pleins correspondent aux dinosaures. Ellipses de tolérance (seuil fixé à $\chi^2_{2;0.975}$) calculées à partir de la moyenne et de la covariance classiques (tirets) et à l'aide d'estimateurs robustes (ellipsoïde de volume minimal ; trait plein).

Huitième partie

Chronologie

22. Introduction

En archéologie, le processus de datation consiste au “placement dans le temps d’événements les uns par rapport aux autres ou par rapport à toute échelle établie de mesure temporelle” (Dean, 1978). Cela implique de distinguer les méthodes de datation dites *relatives* (qui fournissent une séquence chronologique ordonnée d’événements) et les méthodes de datation dites *absolues* (qui donnent une indication calendaire et peuvent fournir la durée d’un événement) (O’Brien & Lyman, 2002).

À proprement parler, il n’existe pas de datation absolue, étant donné la façon dont les dates sont produites et que toute date s’inscrit dans un référentiel particulier. La distinction entre temps absolu et temps relatif peut être reformulée plus clairement comme quantifiable ou non quantifiable (O’Brien & Lyman, 2002) : les dates absolues “sont exprimées en points sur des échelles standard de mesure du temps” (Dean, 1978).

Nous garderons ici la distinction entre une *date* et un *âge* telle que formulée par Colman et al. (1987) : “une date est un point précis dans le temps, alors qu’un âge est un intervalle de temps mesuré en arrière à partir du présent”. Traiter des dates en archéologie peut être délicat si l’on ne tient pas compte des sources de l’information chronologique. Dans la plupart des cas, une date représente un *terminus* pour un assemblage archéologique donné. C’est-à-dire une date avant (*terminus ante quem*) ou après (*terminus post quem*) laquelle l’événement étudié a eu lieu. Enfin, on gardera à l’esprit qu’il n’existe pas de datation sans incertitude en archéologie (même si cette dernière ne peut pas toujours être quantifiée).

23. Chronologie qualitative

24. Chronologie quantitative

Neuvième partie

Cartographie

25. Introduction

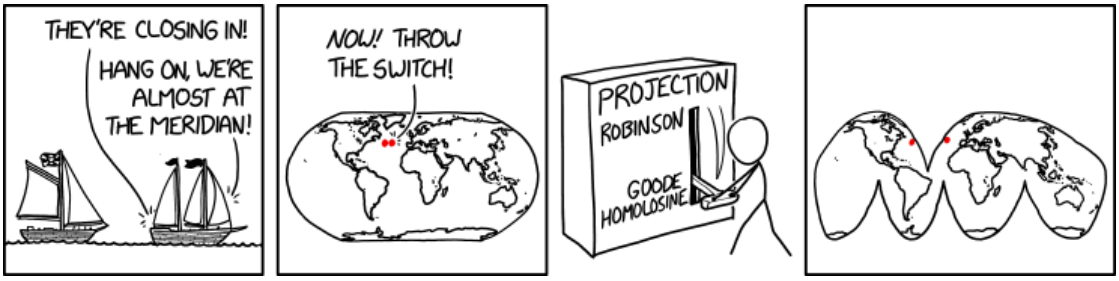


Fig. 25.1. : Sea Chase. "There are two rules on this ship : Never gaze back into the projection abyss, and never touch the red button labeled DYMAXION." (CC BY-NC 2.0 Munroe, 2022).

26. Analyse de données vectorielles

27. Analyse de données raster

Dixième partie

Annexes

A. Installation

A.1. Installer R

R est fonctionne sur les principaux systèmes d'exploitation. Vous pouvez télécharger les fichiers d'installation depuis le site du CRAN. Sous Windows et Mac OS, lancez l'exécutable après l'avoir téléchargé et suivez les étapes d'installation en laissant les options par défaut.

R est disponible dans les dépôts d'Ubuntu. Cependant, les paquets inclus dans les dépôts Ubuntu par défaut sont souvent un peu anciens. Il peut être préférable d'installer R à partir du dépôt du CRAN pour bénéficier des dernières versions.

Pour installer R sur Ubuntu :

```
sudo apt update
sudo apt -y install r-base r-base-dev
```

Important

Des changements importants peuvent subvenir entre les versions de R, entraînant parfois des problèmes de compatibilité. Il est fortement recommandé d'utiliser la dernière version mineure de R et de maintenir son installation à jour.

A.2. Installer RStudio

RStudio est un environnement de développement intégré (*interactive development environment*, IDE)[1]. RStudio *n'est pas* R, pas plus qu'il n'offre une interface graphique à R. Il s'agit d'un logiciel utilitaire qui permet d'éditer et d'exécuter votre code, ainsi que de réaliser des actions connexes telles que l'installation de packages ou l'import de données. RStudio dispose notamment d'un système de gestion de projets qui permet d'organiser son travail.

[1] RStudio n'est pas le seul IDE, des utilisateurs avancés peuvent lui préférer *Emacs Speaks Statistics* (ESS), un plugin pour l'éditeur Emacs.

RStudio Desktop est disponible sous licence *open-source* pour les principaux systèmes d'exploitation et peut être téléchargée depuis le site de la société éditrice.

Pour installer RStudio sur Ubuntu, ouvrez un terminal dans le dossier de téléchargement et lancez la commande suivante (le nom du fichier peut différer en fonction de la version) :

```
sudo dpkg -i rstudio-2021.09.2-382-amd64.deb
```

Important

Après l'installation de RStudio, la première étape consiste à en changer les réglages par défaut, pour éviter qu'il ne conserve des éléments entre deux sessions de travail. Ouvrez les options globales (*Tools* > *Global Options…*) et modifiez les réglages suivants (fig. A.1) :

- Décochez *Restore .RData into workspace at startup.*
- Réglez *Save workspace to .RData on exit* sur *never.*

Un fichier .RData stocke les données et les fonctions que vous créez dans chaque session de travail. Par défaut, RStudio enregistre automatiquement ce fichier à la fermeture, et le recharge lorsque vous démarrez RStudio. C'est une source de problèmes et de confusion. En effet, il peut en résulter que des objets temporaires créés au cours de brèves expériences et explorations s'attardent sur plusieurs sessions car ils sont automatiquement rechargés, contaminant ainsi l'espace de travail. La modification de ces deux paramètres vous épargnera bien des soucis lorsque vous apprendrez à utiliser l'environnement de R, et vous aidera à prendre le contrôle des données et des fonctions que vous créez avec R (en vous évitant de faire débuter tous vos scripts par `rm(list = ls())`). Dans la plupart des cas, il est plus simple et plus efficace de considérer le code que vous écrivez comme le produit le plus important lorsque vous travaillez sur l'analyse, et d'écrire des instructions spécifiques pour exporter (ou enregistrer) uniquement les résultats les plus importants (par exemple, certaines sorties tabulaires sous forme de fichiers CSV, et des graphiques sous forme de fichiers PNG).

L'interface de RStudio se présente sous la forme d'un ensemble d'onglets répartis en quadrants (fig. A.2). Par défaut, de haut en bas et de gauche à droite[2] :

- Le quadrant supérieur gauche est l'éditeur de texte : il contient les documents en cours d'édition (scripts…), permet d'explorer les données et offre un accès rapide à certaines commandes en fonction du contexte (exécuter un script, compiler un document…). C'est là que vous écrivez et éditez le code et le texte. Écrire du code

[2]Pour plus de détails sur l'interface de RStudio, voir Verzani (2012), Gandrud (2018) et Racine (2012)

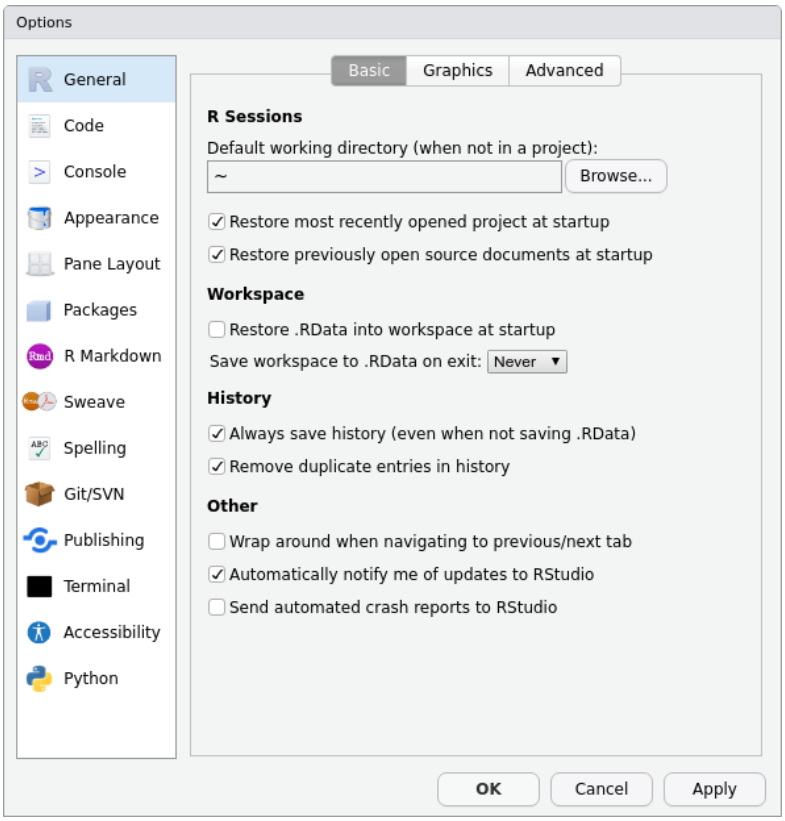


FIG. A.1. : Configuration globale de RStudio Desktop.

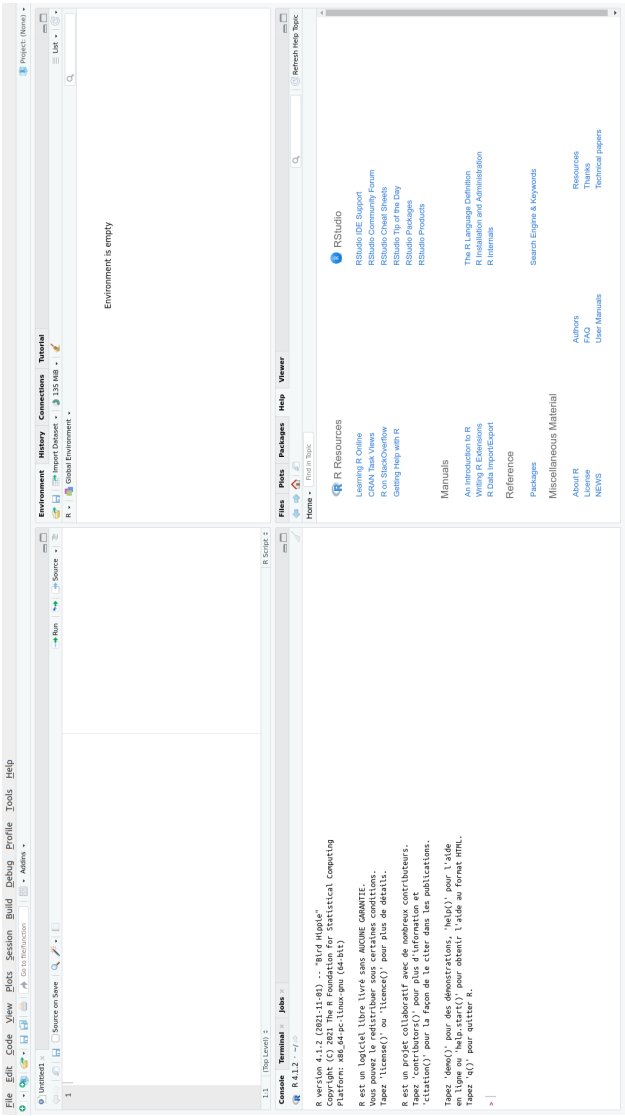


FIG. A.2. : L'interface de RStudio Desktop.

signifie écrire en texte brut, sans gras, soulignement, italique ou autre formatage. L'apparence de l'éditeur de texte peut être modifiée dans RStudio en allant dans le menu déroulant *Tools*, et en sélectionnant *Global Options...*, puis en parcourant les options dans les sections *Code* et *Appearance* sur la liste de gauche. Il peut être utile d'afficher les caractères d'espacement (dans la section *Code*, onglet *Display*), et de vérifier toutes les options de complétion et de diagnostic (section *Code*, onglet *Completion* et *Diagnostics*). Le code dans l'éditeur de texte est le produit le plus important de votre travail, c'est donc une bonne idée de l'enregistrer dans un endroit adapté, avec un nom significatif (généralement les fichiers de code R se terminent par .R, avec un R majuscule après le point), et de l'enregistrer fréquemment pendant que vous travaillez dessus pour minimiser le risque de perte accidentelle.

- Le quadrant supérieur droit peut comporter plusieurs onglets, en fonction de ce que vous faites. La plupart du temps, vous verrez un onglet *Environment* et un onglet *History*. L'onglet *History* contient l'historique des commandes exécutées dans la console. L'onglet *Environment* répertorie les objets qui sont actuellement disponibles dans votre session R. L'environnement est l'endroit où R stocke les résultats des calculs et des fonctions. Par exemple, si vous avez exécuté un code qui lit une feuille de calcul dans R et la stocke en tant qu'objet R, vous verrez cet objet répertorié dans l'onglet *Environment*. Cet onglet est utile car il vous permet d'obtenir en un coup d'œil des informations de base sur la taille des objets que vous créez (par exemple, le nombre de lignes et de colonnes). Il permet de vérifier rapidement le résultat de vos calculs et voir si votre code fonctionne comme prévu. L'environnement est un aspect important de R car il nous offre un système cohérent et flexible pour créer, combiner et manipuler différents types de données pendant notre analyse. Les environnements sont un concept de programmation avancé, et tout ce que nous devons faire ici est de noter qu'ils sont utiles, et qu'il n'y a rien de comparable à eux dans les tableurs. L'onglet *History* contient l'historique des commandes que vous avez précédemment exécutées dans votre session R. Il est vivement recommande de sauvegarder votre code dans un fichier .R, même pour les analyses et les expériences les plus brèves. Si vous travaillez de manière expéditive sans enregistrer, vous pouvez utiliser l'historique pour parcourir vos commandes précédentes afin de les réutiliser et de les modifier. Vous pouvez également enregistrer l'historique de vos commandes R dans un fichier texte. Ce n'est cependant pas une méthode efficace pour garder une trace de votre travail. Une meilleure stratégie consiste à écrire du code dans l'éditeur de texte, entrecoupé de commentaires qui expliquent ce que fait le code, et d'enregistrer ce document sur votre ordinateur.
- Le quadrant inférieur droit permet de naviguer dans les fichiers, de visualiser les graphiques (générés depuis la console ou à l'aide d'un script), de gérer les packages et d'afficher l'aide. Vous pouvez revenir aux graphiques précédents créés dans votre session actuelle. L'onglet *Plots* comporte un bouton *Export*" qui permet d'enregistrer facilement vos graphiques sous forme de fichiers image sur votre ordinateur (dans différents formats) ou de les copier dans le presse-papiers pour un transfert rapide par copier-coller. Ces fonctionnalités sont utiles pour itérer vers un graphique

de qualité publication.

- Le quadrant inférieur gauche contient la console où le code est exécuté. Il n'y a pas de boutons spécifiques à la console, l'interaction directe avec ce volet est donc limitée. La partie la plus importante de ce volet est l'invite de commande, qui débute par un chevron `>`, et lorsque ce volet est actif, un curseur clignotant apparaît.

Conseil

Il existe deux fonctions utiles de la console qui vous feront gagner du temps lorsque vous travaillerez avec R. La première est la touche de déplacement vers le haut de votre clavier, qui vous permet de réexécuter les commandes précédentes. Si vous placez votre curseur à l'invite et appuyez sur la touche de déplacement vers le haut, vous pouvez parcourir les lignes de code précédemment exécutées. Une fois que vous avez trouvé le code que vous voulez réexécuter, vous pouvez le modifier dans la console, si vous le souhaitez, ou simplement appuyer sur Entrée pour l'envoyer à l'interpréteur. La deuxième fonctionnalité pratique du volet console de RStudio est la petite icône en forme de flèche située à droite de la barre de titre de la console. Si vous regardez en haut du panneau de la console, vous verrez le mot *Console*, puis vous verrez un chemin d'accès à un dossier sur votre ordinateur, et à la fin de celui-ci, vous verrez une petite icône en forme de flèche courbée pointant vers la droite. Le chemin sur votre ordinateur dans la barre de titre de la console est votre répertoire de travail. Si vous cliquez sur la petite flèche, le volet des fichiers s'ouvre pour afficher le contenu de votre répertoire de travail.

B. Aide mémoire

B.1. Séquences

```
# Séquence de nombres entiers
1:10
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

```
# Séquence de 1 à 10
seq(1, 10)
#>  [1]  1  2  3  4  5  6  7  8  9 10

# Séquence de 1 à 10 par pas de 2
seq(1, 10, by = 2)
#> [1] 1 3 5 7 9
```

```
# Séquence de 10 entiers
seq_len(10)
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

```
# Séquence de même longueur qu'un objet (ici de 1 à 10)
x <- c(8.6, 4.1, 3.9, 7, 1.8, 6.9, 1.6, 3.8, 2.6, 5.5)
seq_along(x)
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

B.2. Pipes

L'opérateur `|>` (*pipe*) a été introduit dans la version 4.1 de R[1]. Ainsi, `x |> f(y)` est interprété comme `f(x, y)`.

[1] Pour les versions de R antérieures 4.1, on peut utiliser l'opérateur `%>%` du package *magrittr*

```
mtcars |>
  transform(avg = mpg / wt) |>
  subset(avg > 5, select = c(mpg, wt, avg))
#>                    mpg    wt       avg
#> Mazda RX4         21.0 2.620  8.015267
#> Mazda RX4 Wag     21.0 2.875  7.304348
#> Datsun 710        22.8 2.320  9.827586
#> Hornet 4 Drive    21.4 3.215  6.656299
#> Hornet Sportabout 18.7 3.440  5.436047
#> Valiant           18.1 3.460  5.231214
#> Merc 240D         24.4 3.190  7.648903
#> Merc 230          22.8 3.150  7.238095
#> Merc 280          19.2 3.440  5.581395
#> Merc 280C         17.8 3.440  5.174419
#> Fiat 128          32.4 2.200 14.727273
#> Honda Civic       30.4 1.615 18.823529
#> Toyota Corolla    33.9 1.835 18.474114
#> Toyota Corona     21.5 2.465  8.722110
#> Fiat X1-9         27.3 1.935 14.108527
#> Porsche 914-2     26.0 2.140 12.149533
#> Lotus Europa      30.4 1.513 20.092531
#> Ferrari Dino      19.7 2.770  7.111913
#> Volvo 142E        21.4 2.780  7.697842
```

L'utilisation des *pipes* à l'avantage de clarifier et de faciliter la lecture du code, mais peut considérablement complexifier le débuggage en cas de problème. Il est recommandé d'utiliser cet opérateur avec parcimonie.

C. Bonnes pratiques

D. Style de programmation

Cette section propose un style de programmation destiné à accompagner les utilisatrices et utilisateurs débutants. Ce style regroupe des recommandations pour l'écriture du code et permet d'en faciliter la lecture et la compréhension. Ce style est dérivé du *tidyverse style guide* et du livre *rOpenSci Packages*.

D.1. Dénominations

En règle générale, les abréviations doivent être évitées lors de la dénomination de fichiers ou d'objets.

D.1.1. Nommer les fichiers

Les noms de fichiers doivent utiliser l'extension `.R`.

```
# GOOD
plot.R

# BAD
plot
```

Les noms de fichiers doivent être parlants.

```
# GOOD
plot.R

# BAD
Untitled1.R
```

Les noms de fichiers ne doivent pas contenir de `/` et d'espaces. À la place, utilisez un tiret (`-`) ou un trait de soulignement (`_`).

```
# GOOD
read_csv.R
plot-methods.R

# BAD
read csv.R
```

Les noms de fichiers doivent utiliser des lettres du latin de base, et non du supplément latin-1, et doivent être en minuscules.

```
# GOOD
plot.R

# BAD
Plot.R
données.R
```

Utilisez des verbes signifiants pour les noms de fichiers.

```
# GOOD
fit_model.R

# BAD
addition.R
```

Si le fichier ne contient qu'une seule fonction, nommez-le par le nom de la fonction.

D.1.2. Nommer les variables

Les noms des variables doivent être aussi courts que possible.

Les noms des variables doivent être des substantifs évocateurs.

Les noms de variables doivent être en minuscules.

Ne séparez jamais les mots à l'intérieur du nom par `.` (réservé pour les méthodes S3) et n'écrivez pas en PascalCase (réservé pour les définitions de classes S4). Utilisez plutôt un trait de soulignement (`_`).

```
# GOOD
std_dev <- 3

# BAD
std.dev <- 3
StdDev <- 3
```

N'utilisez pas les noms de fonctions et de variables existantes (en particulier celles qui sont propres à R).

```
# GOOD
std_dev <- 3

# BAD
T <- 1
c <- 2 * 2
mean <- 10
```

D.1.3. Nommer les fonctions

Les noms de fonction doivent contenir un verbe qui fait référence à l'action principale de la fonction.

Les noms de fonction doivent être en snake_case. N'utilisez `.` que pour le dispatching des méthodes S3.

Le schéma de nommage `object_verb()` doit être préféré aussi souvent que possible. Ce schéma est facile à lire et à compléter automatiquement.

```
# GOOD
peak_detect()

# BAD
addition()
readFile()
```

Évitez de réemployer des noms de fonctions déjà utilisés par les packages de base ou d'autres packages populaires.

D.1.4. Nommer les classes S4

Les noms des classes doivent être des substantifs en PascalCase avec une majuscule initiale et la première lettre de chaque mot concaténé en majuscule.

D.2. Syntaxe

D.2.1. Longueur de ligne

La longueur maximale des lignes est limitée à 80 caractères.

Ne mettez pas plus d'une déclaration par ligne. N'utilisez pas de point-virgule pour terminer la commande.

```
# GOOD
x <- 1
x <- x + 1

# BAD
x <- 1; x <- x + 1
```

D.2.2. Appel de fonction

Dans un appel de fonction, spécifiez les arguments par leur nom. Ne spécifiez jamais les arguments par un nom partiel et ne mélangez jamais la position et le nom complet.

```
# GOOD
mean(x, na.rm = TRUE)

# BAD
mean(x, na = TRUE)
```

Les arguments obligatoires doivent être placés en premier, suivis des arguments facultatifs.

```
# GOOD
raise_to_power(x, power = 2.7)

# BAD
raise_to_power(power = 2.7, x)
```

L'argument `...` doit se trouver soit au début, soit à la fin.

```
# GOOD
standardize(..., scale = TRUE, center = TRUE)

# BAD
standardize(scale = TRUE, ..., center = TRUE)
```

Définissez les arguments par défaut à l'intérieur de la fonction en utilisant l'idiome `NULL`, et évitez la dépendance entre les arguments.

Validez toujours les arguments dans une fonction.

D.2.3. Espaces de nom

Si vous utilisez ponctuellement une unique fonction d'un package, utilisez `::` pour spécifier le nom du package au lieu de charger tout le package.

```
# GOOD
FactoMineR::PCA()

# OK
library(FactoMineR)
PCA()
```

L'utilisation des espaces de nom permet d'éviter les conflits lorsque deux packages ont une fonction du même nom et explicite l'origine de la fonction utilisée.

D.2.4. Boucles

Utilisez `seq_along()` dans les boucles pour gérer correctement les cas où `x` est vide.

```
# Soit un vecteur numérique de longueur nulle
x <- numeric(0)

# La boucle ne s'exécute pas (OK)
for (i in seq_along(x)) {
  print(i)
}

# La boucle s'exécute pour i=1 puis i=0 (!)
for (i in 1:length(x)) {
  print(i)
}
#> [1] 1
#> [1] 0
```

D.2.5. Affectation

Utilisez `<-` au lieu de `=` pour l'affectation.

```
# GOOD
x <- 1

# BAD
x = 1
1 -> x
```

D.2.6. Espaces

Mettez des espaces autour de tous les opérateurs binaires infixes (`=`, `+`, `*`, `==`, `&&`, `<-`, `%*%`, etc.).

```
# GOOD
x == y
z <- 2 + 1

# BAD
x==y
z<-2+1
```

Mettez des espaces autour de `=` dans les appels de fonction.

```
# GOOD
mean(x = c(1, 2, 3), na.rm = TRUE)

# BAD
mean(x=c(1, 2, NA), na.rm=TRUE)
```

Ne placez pas d'espace pour obtenir des sous-ensembles (`$` et `@`), pour la manipulation des espaces de noms (`::` et `:::`) et pour la génération de séquences (`:`).

```
# GOOD
car$cyl
dplyr::select()
1:10

# BAD
car $cyl
dplyr:: select()
1: 10
```

Mettez un espace après une virgule.

```
# GOOD
mtcars[1, ]
mean(x = c(1, NA, 2), na.rm = TRUE)

# BAD
mtcars[1 ,]
mean(x = c(1,NA,2),na.rm = TRUE)
```

Utilisez un espace avant les parenthèses ouvrantes, sauf dans un appel de fonction.

```
# GOOD
for (element in element_list)
if (total == 5)
sum(1:10)

# BAD
for(element in element_list)
if(total == 5)
sum (1:10)
```

Ne mettez pas d'espaces autour du code entre parenthèses ou entre crochets.

```
# GOOD
if (is_true) message("Hello!")
species["tiger", ]

# BAD
if ( is_true ) message("Hello!")
species[ "tiger" ,]
```

D.2.7. Accolade

Une accolade ouvrante ne doit jamais être placée sur sa propre ligne et doit toujours être suivie d'une nouvelle ligne.

```
# GOOD
if (is_true) {
  # do something
}
```

```
if (is_true) {
  # do something
} else {
  # do something else
}

# BAD
if (is_true)
{
  # do something
}

if (is_true) { # do something }
else { # do something else }
```

Une accolade fermante doit toujours être placée sur sa propre ligne, sauf si elle est suivie de `else`.

```
# GOOD
if (is_true) {
  # do something
} else {
  # do something else
}

# BAD
if (is_true) {
  # do something
}
else {
  # do something else
}
```

Indentez toujours le code à l'intérieur des accolades.

```
# GOOD
if (is_true) {
  # do something
  # and then something else
}

# BAD
if (is_true) {
```

```
  # do something
  # and then something else
}
```

Les accolades et les nouvelles lignes peuvent être évitées, si une déclaration après `if` est très courte.

```
# OK
if (is_true) return(value)
```

D.2.8. Indentation

N'utilisez pas de tabulations ou de mélanges de tabulations et d'espaces pour l'indentation.

Utilisez deux espaces pour l'indentation.

D.2.9. Nouvelle ligne

Dans une définition ou un appel de fonction, les arguments excessifs doivent être indentés sur la ligne de la parenthèse fermante, si deux lignes suffisent.

```
# GOOD
long_function_name <- function(arg1, arg2, arg3, arg4,
                               long_argument_name1 = TRUE)

plot(table(rpois(100, 5)), type = "h", col = "red", lwd = 10,
     main = "rpois(100, lambda = 5)")
```

Sinon, chaque argument peut aller sur une ligne séparée, en commençant par une nouvelle ligne après la parenthèse ouvrante.

```
# GOOD
long_function_name <- function(long_argument_name1 = c("value1", "value2"),
                               long_argument_name2 = TRUE,
                               long_argument_name3 = NULL,
                               long_argument_name4 = FALSE)

list(
  mean = mean(x),
  sd = sd(x),
```

```
  var = var(x),
  min = min(x),
  max = max(x),
  median = median(x)
)
```

Si la condition dans l'instruction `if` s'étend sur plusieurs lignes, alors chaque condition doit se terminer par un opérateur logique, et non commencer par lui.

```
# GOOD
if (some_very_long_name_1 == 1 &&
    some_very_long_name_2 == 1 ||
    some_very_long_name_3 %in% some_very_long_name_4)

# BAD
if (some_very_long_name_1 == 1
    && some_very_long_name_2 == 1
    || some_very_long_name_3 %in% some_very_long_name_4)
```

Si l'instruction qui contient des opérateurs s'étend sur plusieurs lignes, chaque ligne doit se terminer par un opérateur et non commencer par lui.

```
# GOOD
normal_pdf <- 1 / sqrt(2 * pi * d_sigma ^ 2) *
  exp(-(x - d_mean) ^ 2 / 2 / s ^ 2)

# BAD
normal_pdf <- 1 / sqrt(2 * pi * d_sigma ^ 2)
* exp(-(x - d_mean) ^ 2 / 2 / d_sigma ^ 2)
```

Chaque appel de `dplyr` (après `%>%` ou `|>`) ou de `ggplot2` (après `+`) doit commencer sur une nouvelle ligne.

D.3. Commentaires

Il est recommandé d'écrire les commentaires en anglais.

Les commentaires commencent par `#` suivi d'un espace et du texte du commentaire.

```
# This is a comment.
```

Les commentaires doivent expliquer le pourquoi, et non le comment. Les commentaires doivent expliquer l'intention générale de la commande.

```
# GOOD
# define iterator
i <- 1

# BAD
# set i to 1
i <- 1
```

De courts commentaires peuvent être placés sur la même ligne que le code code.

```
plot(price, weight) # Plot a scatter chart of price and weight
```

Il peut être utile de diviser le code source en morceaux logiques par un commentaire suivi de `-` ou `=`.

```
# Read data ------------------------------------------------------------

# Clean data -----------------------------------------------------------
```

Les descriptions des fonctions et des objets doivent respecter les directives de [*roxygen2*] (https://roxygen2.r-lib.org/).

```
#' Add Together Two Numbers
#'
#' @param x A number.
#' @param y A number.
#' @return The sum of x and y.
#' @examples
#' add(1, 1)
#' add(10, 1)
add <- function(x, y) {
  # general comment
  x + y # inline comment
}
```

Bibliographie

Adler, J. (2010). *R in a nutshell*. O'Reilly.

Aitchison, J. (1982). The Statistical Analysis of Compositional Data. *Journal of the Royal Statistical Society : Series B (Methodological)*, *44*(2), 139-160. https://doi.org/10.1111/j.2517-6161.1982.tb01195.x

Aitchison, J. (1986). *The Statistical Analysis of Compositional Data*. Chapman and Hall. http://dx.doi.org/10.1007/978-94-009-4109-0

Aitchison, J., & Ng, K. W. (2005). The role of perturbation in compositional data analysis. *Statistical Modelling : An International Journal*, *5*(2), 173-185. https://doi.org/10.1191/1471082X05st091oa

Allaire, J., Horner, J., Xie, Y., Marti, V., & Porte, N. (2019). *markdown : Render Markdown with the C Library Sundown* [R package version 1.1]. https://github.com/rstudio/markdown

Allaire, J., Xie, Y., McPherson, J., Luraschi, J., Ushey, K., Atkins, A., Wickham, H., Cheng, J., Chang, W., & Iannone, R. (2022). *rmarkdown : Dynamic Documents for R* [R package version 2.14]. https://CRAN.R-project.org/package=rmarkdown

Almiron, M. G., Almeida, E. S., & Miranda, M. N. (2009). The reliability of statistical functions in four software packages freely used in numerical computation. *Brazilian Journal of Probability and Statistics*, *23*(2). https://doi.org/10.1214/08-BJPS017

Barnier, J. (2021). *Introduction à R et au tidyverse*. https://juba.github.io/tidyverse

Bateman, S., Mandryk, R. L., Gutwin, C., Genest, A., McDine, D., & Brooks, C. (2010). Useful junk ? The effects of visual embellishment on comprehension and memorability of charts. *Proceedings of the 28th International Conference on Human Factors in Computing Systems - CHI '10*, 2573. https://doi.org/10.1145/1753326.1753716

Borer, E. T., Seabloom, E. W., Jones, M. B., & Schildhauer, M. (2009). Some Simple Guidelines for Effective Data Management. *Bulletin of the Ecological Society of America*, *90*(2), 205-214. https://doi.org/10.1890/0012-9623-90.2.205

Bryan, J. (2020). *STAT 545 : Data wrangling, exploration, and analysis with R*. https://stat545.com/

Bryan, J. (2021). *Happy Git and GitHub for the useR*. https://happygitwithr.com/

Carswell, C. M. (1992). Choosing Specifiers : An Evaluation of the Basic Tasks Model of Graphical Perception. *Human Factors : The Journal of the Human Factors and Ergonomics Society*, *34*(5), 535-554. https://doi.org/10.1177/001872089203400503

Chang, W. (2012). *R graphics cookbook*. O'Reilly Media.

Chang, W. (2022). *webshot : Take Screenshots of Web Pages* [R package version 0.5.3]. https://CRAN.R-project.org/package=webshot

Chayes, F. (1960). On correlation between variables of constant sum. *Journal of Geophysical Research*, *65*(12), 4185-4193. https://doi.org/10.1029/JZ065i012p04185

Cleveland, W. S., & McGill, R. (1984). Graphical Perception : Theory, Experimentation, and Application to the Development of Graphical Methods. *Journal of the American Statistical Association*, *79*(387), 531-554. https://doi.org/10.1080/01621459.1984.10478080

Colman, S. M., Pierce, K. L., & Birkeland, P. W. (1987). Suggested Terminology for Quaternary Dating Methods. *Quaternary Research*, *28*(2), 314-319. https://doi.org/10.1016/0033-5894(87)90070-6

Cortés, J. A. (2009). On the Harker Variation Diagrams ; A Comment on "The Statistical Analysis of Compositional Data. Where Are We and Where Should We Be Heading ?" by Aitchison and Egozcue (2005). *Mathematical Geosciences*, *41*(7), 817-828. https://doi.org/10.1007/s11004-009-9222-8

Csárdi, G., Müller, K., & Hester, J. (2022). *desc : Manipulate DESCRIPTION Files* [R package version 1.4.1]. https://CRAN.R-project.org/package=desc

de Vries, A. (2022). *miniCRAN : Create a Mini Version of CRAN Containing Only Selected Packages* [R package version 0.2.16]. https://github.com/andrie/miniCRAN

Dean, J. S. (1978). Independent Dating in Archaeological Analysis. In *Advances in Archaeological Method and Theory* (p. 223-255). Elsevier. https://doi.org/10.1016/B978-0-12-003101-6.50013-5

Feldman-Stewart, D., Kocovski, N., McConnell, B. A., Brundage, M. D., & Mackillop, W. J. (2000). Perception of Quantitative Information for Treatment Decisions. *Medical Decision Making*, *20*(2), 228-238. https://doi.org/10.1177/0272989X0002000208

Ferris, N. (1999). What's in a Name ? The Implications of Archaeological Terminology Used in Nonarchaeological Contexts. In R. Williamson & C. Watts (Éd.), *Taming the Taxonomy : Toward a New Understanding of Great Lakes Archaeology* (p. 111-121). Eastendbooks.

Filzmoser, P., Garrett, R. G., & Reimann, C. (2005). Multivariate outlier detection in exploration geochemistry. *Computers & Geosciences*, *31*(5), 579-587. https://doi.org/10.1016/j.cageo.2004.11.013

Filzmoser, P., & Hron, K. (2008). Outlier Detection for Compositional Data Using Robust Methods. *Mathematical Geosciences*, *40*(3), 233-248. https://doi.org/10.1007/s11004-007-9141-5

Fletcher, M., & Lock, G. R. (1991). *Digging numbers : Elementary statistics for archaeologists*. Oxford Univ. Comm. for Archaeology.

Fox, J., Weisberg, S., & Price, B. (2022). *car : Companion to Applied Regression* [R package version 3.0-13]. https://CRAN.R-project.org/package=car

Frerebeau, N. (2022). *folio : Datasets for Teaching Archaeology and Paleontology* [R package version 1.1.0]. https://CRAN.R-project.org/package=folio

Gandrud, C. (2018, septembre 3). *Reproducible Research with R and RStudio* (2e éd.). Chapman and Hall/CRC. https://doi.org/10.1201/9781315382548

Garrett, R. G. (1989). The chi-square plot : A tool for multivariate outlier recognition. *Journal of Geochemical Exploration*, *32*(1-3), 319-341. https://doi.org/10.1016/0375-6742(89)90071-X

Gillan, D. J., & Richman, E. H. (1994). Minimalism and the Syntax of Graphs. *Human Factors : The Journal of the Human Factors and Ergonomics Society*, *36*(4), 619-644. https://doi.org/10.1177/001872089403600405

Gillespie, C. (2016). *Efficient R programming : A practical guide to smarter programming*. O'Reilly Media.

Grolemund, G. (2015). *Hands-on Programming with R*. O'Reilly.

Hampton, S. E., Strasser, C. A., Tewksbury, J. J., Gram, W. K., Budden, A. E., Batcheller, A. L., Duke, C. S., & Porter, J. H. (2013). Big data and the future of ecology. *Frontiers in Ecology and the Environment*, *11*(3), 156-162. https://doi.org/10.1890/120103

Harker, A. (1909). *The Natural History of Igneous Rocks*. Macmillan.

Heer, J., & Bostock, M. (2010). Crowdsourcing graphical perception : Using mechanical turk to assess visualization design. *Proceedings of the 28th International Conference on Human Factors in Computing Systems - CHI '10*, 203. https://doi.org/10.1145/1753326.1753357

Hullman, J., Adar, E., & Shah, P. (2011). Benefitting InfoVis with Visual Difficulties. *IEEE Transactions on Visualization and Computer Graphics*, *17*(12), 2213-2222. https://doi.org/10.1109/TVCG.2011.175

Iannone, R. (2022). *DiagrammeR : Graph/Network Visualization* [R package version 1.0.9]. https://github.com/rich-iannone/DiagrammeR

Inbar, O., Tractinsky, N., & Meyer, J. (2007). Minimalism in information visualization : Attitudes towards maximizing the data-ink ratio. *Proceedings of the 14th European Conference on Cognitive Ergonomics Invent ! Explore ! - ECCE '07*, 185. https://doi.org/10.1145/1362550.1362587

Keeling, K. B., & Pavur, R. J. (2007). A comparative study of the reliability of nine statistical software packages. *Computational Statistics & Data Analysis*, *51*(8), 3811-3831. https://doi.org/10.1016/j.csda.2006.02.013

Keller, G. (2000). *Applied statistics with Microsoft Excel*. Duxbury.

Kelly, J. D. (1989). The Data-Ink Ratio and Accuracy of Newspaper Graphs. *Journalism Quarterly*, *66*(3), 632-639. https://doi.org/10.1177/107769908906600315

Kleindienst, M. R. (2006). On Naming Things : Behavioral Changes in the Later Middle to Earlier Late Pleistocene, Viewed From the Eastern Sahara. In E. Hovers & S. L. Kuhn (Éd.), *Transitions Before the Transition* (p. 13-28). Springer US. https://doi.org/10.1007/0-387-24661-4_2

Kosara, R., & Ziemkiewicz, C. (2010). Do Mechanical Turks dream of square pie charts ? *Proceedings of the 3rd BELIV'10 Workshop on BEyond Time and Errors : Novel evaLuation Methods for Information Visualization - BELIV '10*, 63-70. https://doi.org/10.1145/2110192.2110202

Kosslyn, S. M., & Chabris, C. F. (1992). Minding information graphics : Knowing a little about human perception can help make your graphics a lot more informative. *Folio*, *21*, 69-71.

Kulla-Mader, J. (2007). *Graphs via Ink : Understanding How the Amount of Non-Data-Ink in a Graph Affects Perception and Learning* (mém. de mast.). Department of Information and Library Science, University of North Carolina.

Li, H., & Moacdieh, N. (2014). Is "chart junk" useful ? An extended examination of visual embellishment. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, *58*(1), 1516-1520. https://doi.org/10.1177/1541931214581316

Ligges, U., Maechler, M., & Schnackenberg, S. (2018). *scatterplot3d : 3D Scatter Plot* [R package version 0.3-41]. https://CRAN.R-project.org/package=scatterplot3d

Long, J. D. (2019). *R cookbook : Proven recipes for data analysis, statistics, and graphics*. O'Reilly Media.

Lovelace, R. (2020). *Geocomputation with r*. CRC Press.

Matloff, N. (2019). *An opinionated view of the Tidyverse "dialect" of the R language*. TidyverseSkeptic. Récupérée 12 février 2022, à partir de https://github.com/matloff/TidyverseSkeptic

McCullough, B., & Heiser, D. A. (2008). On the accuracy of statistical procedures in Microsoft Excel 2007. *Computational Statistics & Data Analysis*, *52*(10), 4570-4578. https://doi.org/10.1016/j.csda.2008.03.004

McGurgan, K. (2015). *Data-Ink Ratio and Task Complexity in Graph Comprehension* (mém. de mast.). Department of Psychology, Rochester Institute of Technology.

Meyer, J., Shinar, D., & Leiser, D. (1997). Multiple Factors that Determine Performance with Tables and Graphs. *Human Factors : The Journal of the Human Factors and Ergonomics Society*, *39*(2), 268-286. https://doi.org/10.1518/001872097778543921

Millot, G. (2014). *Comprendre et réaliser les tests statistiques à l'aide de R - Manuel de biostatistique* (troisième édition). De Boeck.

Munroe, R. (2009, mars 6). *Correlation*. xkcd. https://xkcd.com/552/

Munroe, R. (2014, décembre 12). *Documents*. xkcd. https://xkcd.com/1459/

Munroe, R. (2018, septembre 19). *Curve-Fitting*. xkcd. https://xkcd.com/2048/

Munroe, R. (2019, février 25). *.NORM Normal File Format*. xkcd. https://xkcd.com/2116/

Munroe, R. (2020, août 17). *Dependency*. xkcd. https://xkcd.com/2347/

Munroe, R. (2022, février 4). *Sea Chase*. xkcd. https://xkcd.com/2577/

Murrell, P. (2019). *R graphics*. CRC Press, Taylor & Francis Group.

Noble, W. S. (2009). A Quick Guide to Organizing Computational Biology Projects (F. Lewitter, Éd.). *PLoS Computational Biology*, *5*(7), e1000424. https://doi.org/10.1371/journal.pcbi.1000424

O'Brien, M. J., & Lyman, R. L. (2002). *Seriation, Stratigraphy, and Index Fossils : The Backbone of Archaeological Dating*. Springer.

Pearson, K. (1896). Mathematical Contributions to the Theory of Evolution – On a Form of Spurious Correlation Which May Arise When Indices Are Used in the

Measurement of Organs. *Proceedings of the Royal Society of London (1854-1905)*, *60*(1), 489-498. https://doi.org/10.1098/rspl.1896.0076

Pedersen, T. L. (2021). *ggraph : An Implementation of Grammar of Graphics for Graphs and Networks* [R package version 2.0.5]. https://CRAN.R-project.org/package=ggraph

Pedersen, T. L. (2022). *tidygraph : A Tidy API for Graph Manipulation* [R package version 1.2.1]. https://CRAN.R-project.org/package=tidygraph

R Core Team. (2022). *R : A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria. https://www.R-project.org/

Racine, J. S. (2012). RStudio : A Platform-Independent IDE for R and Sweave : SOFTWARE REVIEW. *Journal of Applied Econometrics*, *27*(1), 167-172. https://doi.org/10.1002/jae.1278

Rangecroft, M. (2003). As easy as pie. *Behaviour & Information Technology*, *22*(6), 421-426. https://doi.org/10.1080/01449290310001615437

Reimann, C., Filzmoser, P., & Garrett, R. G. (2005). Background and threshold : Critical comparison of methods of determination. *Science of The Total Environment*, *346*(1-3), 1-16. https://doi.org/10.1016/j.scitotenv.2004.11.023

Ripley, B. (2022). *MASS : Support Functions and Datasets for Venables and Ripley's MASS* [R package version 7.3-57]. http://www.stats.ox.ac.uk/pub/MASS4/

Robbins, N. B. (2005). *Creating more effective graphs*. Wiley-Interscience.

Rousseeuw, P. J., & van Zomeren, B. C. (1990). Unmasking Multivariate Outliers and Leverage Points. *Journal of the American Statistical Association*, *85*(411), 633-639. https://doi.org/10.1080/01621459.1990.10474920

Schmidt, S. C., & Marwick, B. (2020). Tool-Driven Revolutions in Archaeological Science. *Journal of Computer Applications in Archaeology*, *3*(1), 18-32. https://doi.org/10.5334/jcaa.29

Schonlau, M., & Peters, E. (2012). Comprehension of Graphs and Tables Depend on the Task : Empirical Evidence from Two Web-Based Studies. *Statistics, Politics, and Policy*, *3*(2). https://doi.org/10.1515/2151-7509.1054

Sievert, C., & Cheng, J. (2021). *bslib : Custom Bootstrap 'Sass' Themes for shiny and rmarkdown* [R package version 0.3.1]. https://CRAN.R-project.org/package=bslib

Silge, J. (2017). *Text mining with R : A tidy approach*. O'Reilly Media.

Simkin, D., & Hastie, R. (1987). An Information-Processing Analysis of Graph Perception. *Journal of the American Statistical Association*, *82*(398), 454-465. https://doi.org/10.1080/01621459.1987.10478448

Spence, I. (1990). Visual psychophysics of simple graphical elements. *Journal of Experimental Psychology : Human Perception and Performance*, *16*(4), 683-692. https://doi.org/10.1037/0096-1523.16.4.683

Spence, I. (2005). No Humble Pie : The Origins and Usage of a Statistical Chart. *Journal of Educational and Behavioral Statistics*, *30*(4), 353-368. https://doi.org/10.3102/10769986030004353

Spence, I., & Lewandowsky, S. (1991). Displaying proportions and percentages. *Applied Cognitive Psychology*, *5*(1), 61-77. https://doi.org/10.1002/acp.2350050106

Strange, N. (2007). *Smoke and mirrors : How to bend facts and figures to your advantage*. A. & C. Black
OCLC : ocm76798286.

Strasser, C. A., & Hampton, S. E. (2012). The fractured lab notebook : Undergraduates and ecological data management training in the United States. *Ecosphere*, *3*(12), art116. https://doi.org/10.1890/ES12-00139.1

Sutter, R. D., Wainscott, S. B., Boetsch, J. R., Palmer, C. J., & Rugg, D. J. (2015). Practical guidance for integrating data management into long-term ecological monitoring projects. *Wildlife Society Bulletin*, *39*(3), 451-463. https://doi.org/10.1002/wsb.548

Talbot, J., Setlur, V., & Anand, A. (2014). Four Experiments on the Perception of Bar Charts. *IEEE Transactions on Visualization and Computer Graphics*, *20*(12), 2152-2160. https://doi.org/10.1109/TVCG.2014.2346320

Thieme, N. (2018). R generation. *Significance*, *15*(4), 14-19. https://doi.org/10.1111/j.1740-9713.2018.01169.x

Tufte, E. R. (1990). Data-Ink Maximization and Graphical Design. *Oikos*, *58*(2), 130. https://doi.org/10.2307/3545420

Tufte, E. R. (2001). *The visual display of quantitative information* (2nd ed). Graphics Press.

Tukey, J. W. (1990). Data-Based Graphics : Visual Display in the Decades to Come. *Statistical Science*, *5*(3), 327-339.

van den Boogaart, K. G., & Tolosana-Delgado, R. (2013). *Analyzing Compositional Data with R*. Springer-Verlag. https://doi.org/10.1007/978-3-642-36809-7

VanPool, T. L., & Leonard, R. D. (2010, novembre 22). *Quantitative Analysis in Archaeology*. Wiley-Blackwell. https://doi.org/10.1002/9781444390155

Verzani, J. (2012). *Getting started with RStudio* (2. release). O'Reilly.

Volk, C. J., Lucero, Y., & Barnas, K. (2014). Why is Data Sharing in Collaborative Natural Resource Efforts so Hard and What can We Do to Improve it ? *Environmental Management*, *53*(5), 883-893. https://doi.org/10.1007/s00267-014-0258-2

White, E., Baldridge, E., Brym, Z., Locey, K., McGlinn, D., & Supp, S. (2013). Nine Simple Ways to Make It Easier to (Re)Use Your Data. *Ideas in Ecology and Evolution*, *6*(2). https://doi.org/10.4033/iee.2013.6b.6.f

Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, *59*(10), 1-23. https://doi.org/10.18637/jss.v059.i10

Wickham, H. (2015). *R packages*. O'Reilly Media.

Wickham, H. (2016). *R for data science : Import, tidy, transform, visualize, and model data*. O'Reilly Media.

Wickham, H. (2019). *Advanced r*. Taylor & Francis, CRC Press.

Wickham, H. (2021). *downlit : Syntax Highlighting and Automatic Linking* [R package version 0.4.0]. https://CRAN.R-project.org/package=downlit

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen,

T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., … Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

Wickham, H., Chang, W., Flight, R., Müller, K., & Hester, J. (2021). *sessioninfo : R Session Information* [R package version 1.2.2]. https://CRAN.R-project.org/package=sessioninfo

Wickham, H., Chang, W., Henry, L., Pedersen, T. L., Takahashi, K., Wilke, C., Woo, K., Yutani, H., & Dunnington, D. (2022). *ggplot2 : Create Elegant Data Visualisations Using the Grammar of Graphics* [R package version 3.3.6]. https://CRAN.R-project.org/package=ggplot2

Wickham, H., Hester, J., & Ooms, J. (2021). *xml2 : Parse XML* [R package version 1.3.3]. https://CRAN.R-project.org/package=xml2

Wilke, C. O. (2019). *Fundamentals of data visualization : A primer on making informative and compelling figures*. O'Reilly Media, Inc.

Xie, Y. (2016). *bookdown : Authoring Books and Technical Documents with R Markdown* [ISBN 978-1138700109]. Chapman ; Hall/CRC. https://bookdown.org/yihui/bookdown

Xie, Y. (2021). *mime : Map Filenames to MIME Types* [R package version 0.12]. https://github.com/yihui/mime

Xie, Y. (2022a). *bookdown : Authoring Books and Technical Documents with R Markdown* [R package version 0.26]. https://CRAN.R-project.org/package=bookdown

Xie, Y. (2022b). *knitr : A General-Purpose Package for Dynamic Report Generation in R* [R package version 1.39]. https://yihui.org/knitr/

Yalta, A. T. (2008). The accuracy of statistical distributions in Microsoft® Excel 2007. *Computational Statistics & Data Analysis*, *52*(10), 4579-4586. https://doi.org/10.1016/j.csda.2008.03.005

Zacks, J., Levy, E., Tversky, B., & Schiano, D. J. (1998). Reading bar graphs : Effects of extraneous depth cues and graphical context. *Journal of Experimental Psychology : Applied*, *4*(2), 119-138. https://doi.org/10.1037/1076-898X.4.2.119

Zhu, H. (2021). *kableExtra : Construct Complex Table with kable and Pipe Syntax* [R package version 1.3.4]. https://CRAN.R-project.org/package=kableExtra

Zubiaga, A., & MacNamee, B. (2015). Knowing What You Dont Know : Choosing the Right Chart to Show Data Distributions to Non-Expert Users.